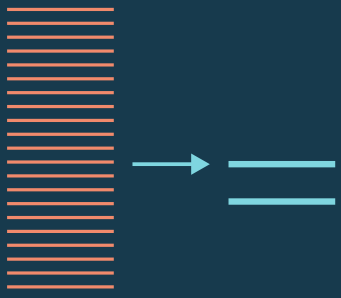
OPTIMISATION BDD · SHOPFLOW · SUP DE VINCI M2

Le chargement groupé de GET /commandes, et le prix d'un cache

UNE PAGE DE 20 COMMANDES



21 requêtes SQL, avant — 2 après

Atelier 10 · Dossier final et restitution

Compétence B4C8 : API et services back-end avec gestion optimisée des données

Octobre 2026 · PostgreSQL 18.6 · Node 22.14 · Redis 8.10.2

21 → 2	×4,2	−73 %	0
requêtes SQL par page de 20 commandes	débit à 20 clients (246,9 → 1032,1 req/s)	latence p95 (98,0 → 26,8 ms)	erreur, résultat identique

Médianes de 3 essais alternés, GET /commandes?limit=20, 20 clients simultanés, même machine, un seul changement (pièces : P-05).

QUESTION DU DOSSIER

Une page de 20 commandes paraît simple, mais devient lente quand plusieurs clients arrivent. **D'où vient ce coût, comment le supprimer sans changer le résultat, et à quel prix ?**

Dans ce dossier

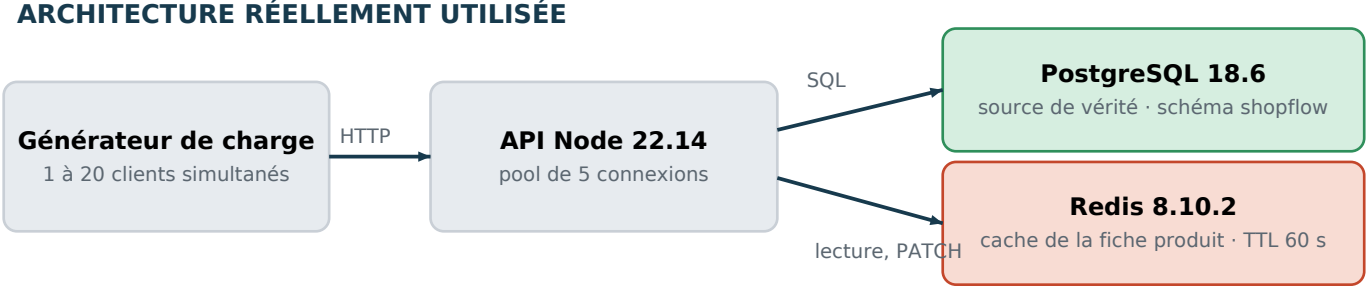
Contexte du projet 2
Organisation du document : comment le lire 3
Inventaire des preuves et choix de l'optimisation 4
A. Besoin et état initial · B. Architecture et stockage 5
C. Intervention principale : 21 requêtes → 2 6
D. Benchmark avant / après, courbe 1 à 20 clients 7
E. Correction et fiabilité : résultat identique, cache Redis 8
F. Décision, variantes rejetées, limites 9
Fiches d'identité des preuves P-01 à P-14 10
Auto-évaluation B4C8 · questions · checklist · formulation finale 11
Pièces jointes : page d'annexe de chaque preuve 12
Sommaire détaillé des ateliers et des annexes, puis annexes (pages 14 à 80) 13

Règle suivie : aucun chiffre sans sortie réelle derrière (sortie brute, plan, trace ou test réellement exécuté).

Contexte du projet

ShopFlow est une boutique en ligne de laboratoire (clients, produits, commandes, lignes). Son API expose l'historique des commandes d'un client (`GET /commandes?limit=20`) et la fiche d'un produit. Le module Optimisation BDD (RNCP 40166, bloc 4, B4C8) demande de mesurer, corriger et défendre une optimisation sur cette API.

ARCHITECTURE RÉELLEMENT UTILISÉE



PgBouncer : testé dans un laboratoire séparé (atelier 7), hors du chemin ci-dessus. Tout tourne sur un portable : i5-10310U, 8 cœurs, 7,6 Go, WSL2 sous Docker.

LES DONNÉES

1 000	200	100 000	300 000	100
clients	produits	commandes	lignes	commandes du client 42

Commandes du 1er janvier au 27 octobre 2026, 3 lignes chacune. Base contrôlée avant et après les tests ; copie jetable pour les mesures lourdes.

LE PARCOURS : CE QUE CHAQUE ATELIER A PROUVÉ

Atelier 1
diagnostic initial des requêtes

Atelier 2
EXISTS 11,3 ms contre jointure 27,6 ms

Atelier 3
index composé : ×2,5 sur la requête de l'API, WAL +35 %

Atelier 4
index partiel : 10,1 → 0,070 ms, utile à son seul prédicat

Atelier 4 bis
migration : 46,3 ms contre 3 324,1 ms sous verrou

Atelier 7
N+1 : 21 → 2 requêtes ; curseur 944,3 → 0,177 ms

Atelier 8
cache Redis : 2,7 → 0,988 ms, copie périmée possible

Jour 5
courbe de charge 1 à 20 clients : ×4,2

Orange : l'optimisation défendue dans ce dossier (ateliers 7 et Jour 5). Vert : la preuve de fraîcheur (atelier 8). Gris : preuves de contexte.

L'HISTOIRE EN QUATRE TEMPS

1 Le symptôme
À 20 clients le débit plafonne à 246,9 req/s dès 5 clients, p95 98,0 ms, sans aucune erreur.

2 La cause
21 allers-retours par page : 0,018 ms vus par PostgreSQL, 0,68 ms vus par l'API.

3 Le remède
Une requête pour toutes les lignes : 2 requêtes, même JSON, 1032,1 req/s.

4 Le prix
Le cache évite des lectures mais peut servir 57.50 quand la base dit 19.90, jusqu'à 60 s.

Comment ce document est organisé

Deux parties. **Le dossier** (pages 1 à 13, dont le sommaire détaillé des ateliers en page 13) répond à la fiche de l'Atelier 10 : il affirme et renvoie. **Les annexes** (pages 14 à 80) contiennent les sorties brutes : elles prouvent. Chaque chiffre du dossier renvoie à une preuve **P-xx** (page 10), qui renvoie à une page d'annexe (page 12).

PARTIE 1 · LE DOSSIER, PAGES 1 À 13

1	**Couverture**	question, résultat en quatre chiffres, plan	
2	**Contexte**	projet, architecture, données, parcours, histoire en quatre temps	
3	**Organisation**	cette page	
4	**D-1 Inventaire**	les preuves déjà produites et le choix de l'optimisation	fiche, partie 2
5	**D-2 A et B**	besoin, état initial, architecture et stockage déployés	partie 3
6	**D-3 C**	hypothèse, un seul changement, trace du mécanisme	partie 3 et 5
7	**D-4 D**	protocole, tableau avant / après, courbes, saturation	partie 3
8	**D-5 E**	résultat identique, droits, fraîcheur du cache Redis	parties 3 et 6
9	**D-6 F**	décision, variantes rejetées, coûts, risques, portée	partie 3
10	**D-7**	fiche d'identité de chaque preuve P-01 à P-14	partie 4
11	**D-8**	auto-évaluation, deux questions, checklist, trois phrases finales	parties 7 à 9
12	**D-9**	pièces jointes : page d'annexe de chaque preuve	partie 9
13	**Sommaire**	contenu détaillé de chaque atelier et de chaque annexe	

PARTIE 2 · LES ANNEXES, PAGES 14 À 80

« annexe p. N » = page portant le numéro N en haut. Numérotation d'origine conservée : les pages 15, 24, 25 et 67 ne sont pas reprises.

Annexe p.	Fichier p.	Contenu
1 à 6	**14-19**	Ateliers 1 et 2 : diagnostic initial, réécritures équivalentes
7 à 14	**20-27**	Ateliers 3 et 4 : historique client, index composé, index spécialisés
16 à 18	**28-30**	Atelier 4 bis : migration de schéma progressive
19 à 21	**31-33**	Atelier 7 : pagination, N+1, PgBouncer
22 à 23	**34-35**	Atelier 8 : cache Redis et fraîcheur
26 à 47	**36-57**	Annexes 1 à 7 : plans et mesures des ateliers 1 à 4
48 à 52	**58-62**	Annexe 8 : preuves de l'atelier 4 bis
53 à 56	**63-66**	Annexe 9 : preuves de l'atelier 7
57 à 59	**67-69**	Annexe 10 : preuves de l'atelier 8
60 à 66	**70-76**	Jour 5 : diagnostic, tuning, courbe de charge, OLTP et analytique
68 à 71	**77-80**	Annexe 11 : preuves du Jour 5, dont les mesures brutes de chaque essai

COMMENT S'EN SERVIR

Lecture rapide	pages 1, 7, 8 et 9 : le résultat, la courbe, le cache et la décision.
Vérifier un chiffre	repérer son P-xx (page 10), lire la page d'annexe (page 12), l'ouvrir via « Fichier p. ».
Naviguer	les signets du PDF reprennent ce plan : couverture, dossier, puis chaque famille d'annexes.

Inventaire des preuves déjà produites et choix de l'optimisation

Chaque chiffre du dossier renvoie à une preuve **P-xx** (fiches en D-7) et à une page des annexes jointes (« annexe p. N », numéro en haut de la page ; voir D-9).

Famille	Résultat mesuré (preuve déjà réalisée)	Où	Rôle ici
Plans et index	Atelier 3 : index composé (client_id, created_at DESC, id DESC) : ×2,5 sur la requête de l'API, ×91 pour un client à 20 100 commandes, WAL +35 % ; retenu, migration écrite et non appliquée au laboratoire. Atelier 4 : index partiel 10,1 → 0,070 ms (×144,6, 1 680 → 3 buffers), inutile hors de son prédicat	annexe p. 7-14	contexte
Réécriture et tuning	Atelier 2 : EXISTS 11,3 ms contre jointure 27,6 ms (la jointure répète le client 42 cent fois). Agrégation mensuelle sur copie : 84,6 → 50,2 ms (work_mem 32 MB) ou 45,5 ms (statistique), même résultat (md5)	annexe p. 1-6, 62	contexte
Migration	Atelier 4 bis : NOT NULL direct = 3 324,1 ms sous verrou exclusif, migration progressive = 46,3 ms ; remplissage 2,7 fois plus long ; retour arrière vérifié	annexe p. 16-18	contexte
API	Atelier 7 : **N+1 21 → 2 requêtes** ; curseur contre OFFSET à 1 000 000 de commandes : 944,3 → 0,177 ms, identique à petite profondeur	annexe p. 19-20	**principal**
Pooling	Atelier 7 : PgBouncer 40 → 5 connexions mais 261 → 94 transactions/s ; Jour 5 : pool 5 / 10 / 20	annexe p. 21, 63 ; P-06, P-07	variante rejetée
Cache	Atelier 8 : miss 2,7 ms, hit 0,988 ms ; 57.50 servi pour 19.90 ; invalidation, expiration, repli, course	annexe p. 22-23	**fraîcheur**
Charge	Jour 5 : courbe 1/5/10/20 clients (N+1 ×4,2), charge ouverte (503 à 130 %), rapport concurrent p95 +64 %	annexe p. 63-64 ; P-05, P-12	**principal**
Donnée moderne	Jour 5 : export journalier CSV (240 lignes, 80 000 commandes payées, montant exact), contrat de données ; Lakehouse et Data Mesh conçus, non déployés	annexe p. 65, 71	contexte
Non réalisé	Réplication, partitionnement, sharding : aucune preuve produite	—	absent

Choix. Une optimisation forte plutôt que plusieurs gains isolés : le chargement groupé de GET /commandes (problème → mécanisme → mesure → limite complets), avec le cache Redis comme preuve de fraîcheur. Les autres familles sont des preuves de contexte ; elles ne sont pas redéfendues ici.

A. Besoin et état initial

Élément	Contenu	Preuve
Endpoint	GET /commandes?limit=20 : écran « mes commandes » du client 42, 20 commandes avec leurs lignes (3 lignes par commande)	P-03
Volume	100 000 commandes, 1 000 clients, 300 000 lignes ; client 42 : 100 commandes (5 pages de 20)	P-03, P-14
Charge	1, 5, 10 et 20 clients simultanés (charge fermée), 10 s de mesure après 3 s d'échauffement, 3 essais alternés	P-05
Problème observable	À 20 clients : 246,9 req/s, p95 98,0 ms, file du pool jusqu'à 15 requêtes, CPU ≈ 50 %, 0 erreur. Le débit plafonne dès 5 clients (239 → 251 → 247 req/s) : le service est saturé sans erreur.	P-01
Métrique de référence	Débit terminé et p95 de GET /commandes à 20 clients, N+1, pool de 5 connexions : 246,9 req/s ; 98,0 ms	P-01

B. Architecture et stockage

Chaîne réellement utilisée (schéma en page 2) : générateur de charge → API Node 22.14 (pool de 5 connexions) → PostgreSQL 18.6, avec Redis 8.10.2 pour la fiche produit.

Composant réellement utilisé	Détail	Preuve
PostgreSQL 18.6	Laboratoire : schéma shopflow (100 000 commandes, 300 000 lignes, 6 index issus des contraintes). L'index composé de l'atelier 3 a été testé puis supprimé, sa migration est écrite mais non appliquée au laboratoire ; la migration progressive de l'atelier 4 bis a été rejouée sur une copie	P-13
Redis 8.10.2	Cache-aside de la fiche produit : miss = 1 SELECT, puis copie avec TTL 60 s ; l'écriture (PATCH) supprime la clé ; politique allkeys-lru	P-08
Pool de l'API	POOL_MAX = 5 sauf série pool ; abandon après 1 s d'attente (connectionTimeoutMillis = 1000, réponse 503)	P-06, P-12
Hors périmètre	Réplication, partitionnement, sharding : non réalisés ; Lakehouse et Data Mesh : conçus, non déployés	—

C. Intervention principale : un seul changement, 21 requêtes → 2

Question de la fiche	Réponse
Hypothèse causale	Le plafond vient du **nombre d'allers-retours** (une requête de lignes par commande : 1 + N = 21), pas de la requête SQL ni de la taille du pool.
Changement (un seul)	relations=n1 → relations=groupe : une requête de lignes pour toute la page, `WHERE commande_id = ANY($1::bigint[])` ; ni index, ni pool, ni base modifiés.

Avant (N+1)

```
rows = SELECT … commandes LIMIT 20;
POUR CHAQUE commande :
  SELECT … FROM lignes
  WHERE commande_id = $1;
-- 1 + 20 = 21 requêtes
```

Après (groupé)

```
rows = SELECT … commandes LIMIT 20;
SELECT … FROM lignes
  WHERE commande_id =
    ANY($1::bigint[]);
-- 2 requêtes, toujours
```

Trace montrant le mécanisme (P-02, P-03)

Observation	N+1	Groupé	Lecture
Requêtes SQL par page de 20 / 50 / 100 commandes	21 / 51 / 101	2 / 2 / 2	croît avec N, contre constant
pg_stat_statements : appels de la requête de lignes (1 200 pages)	20 000 × 0,018 ms	200 × 0,186 ms	le signal est le nombre d'appels
Journal des requêtes lentes (seuil 20 ms)	requête de lignes absente	absente	0 ligne sur 1 249
Temps dans l'API (sqlMs, page de 20)	14,2 ms (≈ 0,68 ms × 21)	2,2 ms	environ 38 fois le temps vu par PostgreSQL (0,018 ms)



Suspects écartés

Suspect	Verdict	Preuve
La requête SQL	innocente	0,018 ms vue par PostgreSQL ; absente du journal lent ; non classée par temps cumulé (P-02)
Le pool de connexions	conséquence	file jusqu'à 15 ; pool de 20 : 276 à 299 req/s, loin de 1 032 (P-06)
Les allers-retours	**cause retenue**	21 par page, chacun environ 38 fois plus cher côté API que côté PostgreSQL (P-02)

Limite du mécanisme. Sur une page isolée le gain de durée est modeste (HTTP 55,3 → 43,9 ms, base locale, aucun autre client) : il devient grand sous concurrence, parce que chaque page occupe une connexion du pool pendant 21 allers-retours (P-03, P-05).

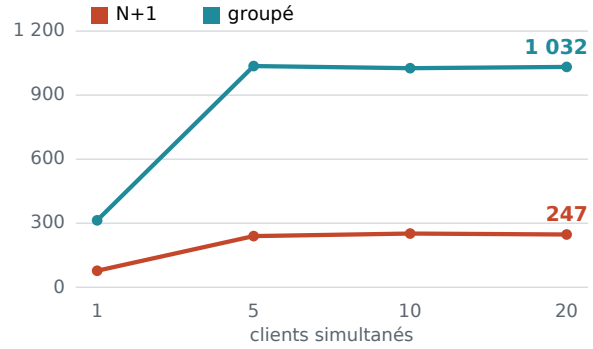
D. Benchmark avant / après (même protocole, un seul changement)

Protocole. Charge fermée sur GET /commandes?limit=20 (mêmes 20 commandes), 1/5/10/20 clients, **10 s de mesure après 3 s d'échauffement**, 3 essais en ordre alterné (N+1, groupé, N+1…), médiane retenue ; API, PostgreSQL et générateur sur la même machine (i5-10310U, 8 cœurs, 7,6 Go, WSL2). Sorties brutes de chaque essai : annexe p. 70-71 (P-05).

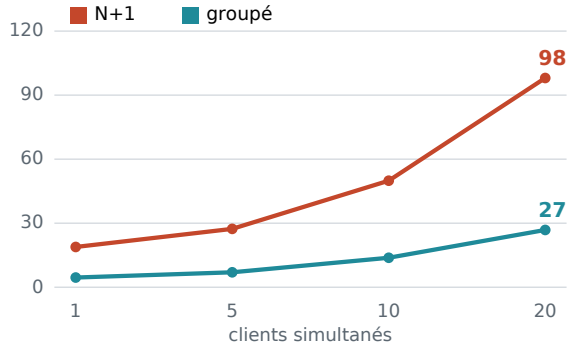
Clients	N+1 req/s	p50 ms	p95 ms	Err.	Groupé req/s	p50 ms	p95 ms	Err.
1	77,0	12,0	18,8	0	313,2	3,0	4,5	0
5	239,4	19,8	27,3	0	1036,1	4,5	7,0	0
10	251,3	38,1	49,9	0	1026,1	9,0	13,8	0
20	246,9	79,0	98,0	0	1032,1	18,2	26,8	0

Médiane de 3 essais par ligne ; erreurs = réponses non 200 (0 partout).

Requêtes terminées par seconde



Latence p95 (ms)



Lecture de la saturation et reproductibilité

Constat à 20 clients	N+1	Groupé	Effet
Débit médian	246,9 req/s	1032,1 req/s	**×4,2**
p95 médian	98,0 ms	26,8 ms	**−73 %**
3 essais (req/s)	238 à 253	978 à 1 082	intervalles disjoints : reproductible
CPU moyen de la machine / erreurs	50,2 % / 0	50,1 % / 0	le gain vient du travail évité, pas d'une ressource en plus

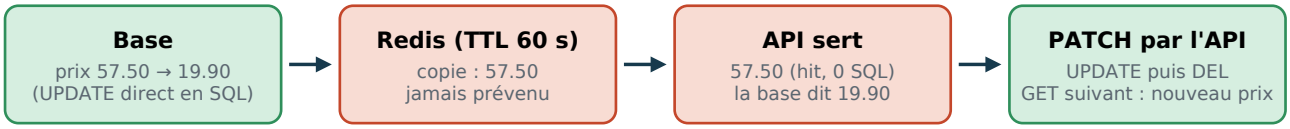
Saturation. Le N+1 plafonne dès 5 clients : les clients supplémentaires n'ajoutent que de l'attente (file du pool jusqu'à 15). Le groupé plafonne plus haut (≈ 1 032 req/s) avec 5 connexions seulement ; à 20 clients pour 5 connexions une file subsiste (jusqu'à 14), mais chaque page garde sa connexion moins longtemps. La charge fermée masque l'effondrement : en charge ouverte à 90 % du débit fermé le comportement est instable (un essai p95 359 ms, l'autre 503), à 130 % presque tout échoue en 503 (P-12, annexe p. 64) ; ces essais sont présentés un par un, sans médiane.

E. Correction et fiabilité

1. Invariant fonctionnel : le résultat est identique (P-03, P-04)

Contrôle	Obtenu	Attendu
JSON complet (commandes et lignes) N+1 = groupé, pages de 5 / 20 / 50 / 100	identique ; 6 / 21 / 51 / 101 SQL contre 2	même contenu
Sans jeton / client_id injecté dans l'URL / curseur falsifié	401 / 400 / 400	refus
Laboratoire après les tests (P-14)	6 index, 0 statistique étendue, md5 des commandes inchangé	inchangé

2. Fraîcheur du cache Redis : ce qu'il gagne, ce qu'il peut faire mentir (P-08 à P-11)



Scénario	Résultat mesuré	Ce que ça établit
Miss puis hit (100 paires, connexion persistante)	miss : 1 SELECT, 2,7 ms ; hit : 0 SQL, 0,988 ms (×2,7)	gain petit (base locale, clé primaire) ; le vrai gain est la charge évitée
UPDATE direct en SQL, puis lecture API	l'API sert 57.50 (hit, 0 SQL), la base a 19.90 ; après DEL : 19.90	le TTL n'actualise pas la copie : elle reste périmée jusqu'au DEL ou à l'expiration
PATCH par l'API (29.90), puis 2 lectures	invalidation = ok ; 1er GET : 29.90 (miss) ; 2e GET : hit, 0 SQL	le chemin d'écriture prévu invalide la copie
Expiration	TTL 60 → −2 après 61 s ; relecture : miss, 1 SELECT	le TTL borne la copie périmée à 60 s
Redis arrêté	200, Cache = indisponible, 1 SELECT par lecture	repli correct, mais toute la charge revient à PostgreSQL
PATCH avec Redis arrêté	invalidation = échouée ; base 34.90 ; copie 29.90 servie au retour de Redis	l'écriture est valide, la copie reste ancienne
Course lecture lente / invalidation (à la main)	A lit 34.90 ; B écrit 39.90 puis supprime la clé ; A range 34.90 : l'API sert 34.90, la base a 39.90	supprimer la clé ne supprime pas toutes les courses (pistes : version, invalidation rejouée)
Rafale : 20 lectures simultanées, clé absente	6 miss, 6 SELECT au lieu d'un ; clé présente : 20 hit, 0 SQL	une mesure, non répétée ; chiffre non reproductible à l'unité près

Garantie retenue. Redis n'est pas la source de vérité : fraîcheur acceptée de quelques secondes à 60 s pour la fiche catalogue ; la validation d'un achat lit prix et stock dans PostgreSQL, jamais dans le cache.

F. Décision et limites

	Contenu	Preuve
Retenu	Chargement groupé : 21 → 2 requêtes, débit ×4,2, p95 −73 %, 0 erreur, réponse identique. Cache de la fiche produit avec TTL 60 s et invalidation par PATCH.	P-02 à P-05, P-08
Variante rejetée n° 1 : agrandir le pool 5 → 20	Supprime l'attente (0 au lieu de 15) mais seulement 276 à 299 req/s contre 1 032 avec le groupé et 5 connexions ; 2 essais, indicatif.	P-06
Variante rejetée n° 2 : PgBouncer (mode transaction)	Connexions 40 → 5 mais débit 260,7 → 93,8 transactions/s, latence 153 → 422 ms : il borne les connexions, il n'accélère rien.	P-07
Coûts	Groupé : aucun index ni écriture en plus ; liste de 100 identifiants au plus par requête (limite de page de l'API) ; regroupement des lignes en mémoire. Cache : copie périmée jusqu'à 60 s, plus de SELECT si Redis tombe.	P-03, P-09, P-10
Risques restants	Course entre lecture lente et invalidation (reproduite à la main, non corrigée) ; rafale de rechargements sur clé absente ; invalidation échouée = copie ancienne jusqu'au TTL.	P-10, P-11
Tests encore nécessaires	Plusieurs instances d'API ; réseau réel (le gain devrait croître avec la latence, non mesuré) ; plus de 20 clients ; mesures de 30 s ; production.	—
Conclusion testable	**Valable ici** : cette machine (8 cœurs, 7,6 Go, WSL2), ce jeu (100 000 commandes, 1 000 clients), GET /commandes?limit=20, 1 à 20 clients, 10 s après 3 s d'échauffement. **Non établi** : un gain en production ; le comportement avec plusieurs instances d'API ; que l'index composé (atelier 3) ou le cache accélèrent « tout PostgreSQL ». La fiche cite 10 s d'échauffement et 30 s de mesure comme exemple : mon protocole est plus court, l'écart entre séries (×4) dépasse l'écart entre essais d'une même série.	—

Fiches d'identité des preuves

Chaque fiche donne : identifiant · question · conditions et action · résultat brut (page de l'annexe) · interprétation · limite.

ID · Question	Conditions · Action	Résultat brut (où)	Interprétation	Limite
P-01 ÉTAT-INITIAL Quel problème observe-t-on sur GET /commandes ?	N+1, pool 5, 20 clients fermés, 10 s après 3 s, médiane de 3	annexe p. 63, 70	246,9 req/s, p95 98,0 ms, file du pool 15, 0 erreur : saturé sans erreur	charge fermée, une machine
P-02 PGSS-N1 Le coût du N+1 est-il dans le SQL ?	copie jetable (PostgreSQL 18.6, md5 identiques au laboratoire), pg_stat_statements ; 1 000 pages N+1 + 200 groupées	annexe p. 68, 20	20 000 appels × 0,018 ms, absente du journal lent ; API : 14,2 ms pour 21 requêtes (≈ ×38 par requête)	le signal est le nombre d'appels, pas la durée
P-03 COMPTAGE-ÉQUIV Le groupé donne-t-il le même résultat avec moins de requêtes ?	client 42, 100 commandes ; mêmes commandes en relations=n1 puis groupe ; comptage par TraceId	annexe p. 19-20, 53	6/21/51/101 SQL contre 2 ; JSON identique ; HTTP 55,3 → 43,9 ms (page isolée)	base locale : gain de durée modeste hors concurrence
P-04 DROITS Les droits sont-ils conservés ?	appels sans jeton, avec client_id dans l'URL, avec curseur falsifié	annexe p. 19, 54	401 / 400 / 400	testé sur cet endpoint seulement
P-05 BENCH-COURBE Le gain tient-il quand la concurrence augmente ?	1/5/10/20 clients, 10 s après 3 s, 3 essais alternés, un seul changement	annexe p. 63, 70-71	N+1 ≈ 250 req/s dès 5 clients ; groupé ≈ 1 032 ; à 20 clients ×4,2, p95 −73 %, essais disjoints (238 à 253 contre 978 à 1 082)	générateur et API sur la même machine ; 10 s (fiche : 30 s en exemple)
P-06 POOL Agrandir le pool suffit-il ?	N+1, 20 clients, POOL_MAX 5 / 10 / 20, 2 essais par taille	annexe p. 63, 70	pool 20 : attente 0, 276 à 299 req/s contre 137 à 216 à pool 5 ; loin de 1 032	2 essais, pool 5 instable : indicatif
P-07 PGBOUNCER PgBouncer accélère-t-il ?	laboratoire séparé, 5 connexions serveur, mode transaction, 40 clients persistants, médiane de 3	annexe p. 21, 55	connexions 40 → 5 ; 260,7 → 93,8 tps ; latence 153 → 422 ms	campagnes de 20 s ; test à 10 clients instable (ordre inversé)
P-08 CACHE-HITMISS Combien de travail le cache évite-t-il ?	produit 42, clé shopflow:produit:v1:42, TTL 60 s, 100 paires miss/hit, connexion persistante	annexe p. 23, 57-58	miss : 1 SELECT, 2,7 ms ; hit : 0 SQL, 0,988 ms (×2,7)	gain absolu petit : base locale, accès par clé primaire
P-09 CACHE-STALE Que sert l'API après une modification SQL directe ?	UPDATE prix = 19.90 en SQL sans passer par l'API, puis lecture	annexe p. 22-23, 58	l'API sert 57.50 (hit, 0 SQL) alors que la base a 19.90 ; après DEL : 19.90	un produit, un processus API
P-10 CACHE-INVALID Invalidation, expiration et repli fonctionnent-ils ?	PATCH 29.90 ; attente 61 s ; Redis arrêté ; PATCH 34.90 avec Redis arrêté	annexe p. 22-23, 58	invalidation ok puis miss ; TTL 60 → −2 ; Redis arrêté : 200 et 1 SELECT ; invalidation échouée : base 34.90, copie 29.90	seul TTL testé : 60 s
P-11 CACHE-COURSE Supprimer la clé après l'écriture suffit-il ?	course lecture lente / invalidation reproduite à la main ; rafale de 20 lectures	annexe p. 23, 59	course : l'API sert 34.90, la base a 39.90 ; rafale : 6 miss et 6 SELECT	course non reproduite sous charge ; rafale non répétée
P-12 CHARGE-OUVERTE La charge fermée masque-t-elle la saturation ?	N+1, pool 5, arrivées à cadence fixe : 50 %, 90 %, 130 % du débit fermé, 2 essais	annexe p. 64, 71	50 % : tient ; 90 % : un essai p95 359 ms, l'autre 503 ; 130 % : presque tout en 503 (pool 1 s)	comportement bimodal : pas de médiane
P-13 STOCKAGE Quels éléments de gestion de la donnée sont réellement déployés ?	ateliers 3, 4 bis, 8 : DDL, index testés, migration progressive, Redis ; état du laboratoire vérifié avant et après chaque test	annexe p. 7-14, 16-18, 48-52	laboratoire : 6 index de contraintes, cache Redis en service (P-08) ; index composé et migration par lots testés, non appliqués au laboratoire	pas de réplication, partitionnement ni sharding
P-14 MÉTHODE-ENV Un autre poste peut-il reproduire la mesure ?	versions, copie jetable, empreintes md5, scripts de charge	annexe p. 68, 71	PostgreSQL 18.6, Node 22.14, md5 commandes 7505112bb794 et lignes e989d5bec39b identiques laboratoire / copie ; campagne : 16 min	WSL2, 8 cœurs ; journaux pgbench par transaction non conservés

Auto-évaluation B4C8 avant remise

Critère	Preuve suffisante si…	Statut	Référence
Besoin	endpoint, volume et problème définis	directe	P-01, P-03
Stockage	un élément de gestion de la donnée réellement déployé	directe (PostgreSQL, Redis en service ; index testés, non appliqués)	P-13, P-08
Correction	résultat, droits et invariants valides après l'intervention	directe	P-03, P-04
Méthode	protocole reproductible : versions, jeu, charge, durée, commandes	directe ; 10 s après 3 s, plus court que l'exemple de la fiche	P-14, P-05
Intervention	le mécanisme explique le gain	directe (appels, temps par requête)	P-02, P-03
Mesure	avant/après comparable, p95/débit/erreurs, sorties brutes	directe ; journaux pgbench non conservés (séries SQL)	P-05
Fiabilité / fraîcheur	panne, reprise, cache ou migration vérifié	directe (cache : stale, invalidation, expiration, panne, course)	P-09, P-10, P-11
Limites	coûts, risques et conditions de généralisation exposés	directe	P-06, P-07, P-12 · section F

Les deux questions

Reproductibilité : « Si je vous donne un autre poste, comment reproduisez-vous exactement votre mesure ? »

1	Créer la base avec 01_schema.sql et 02_donnees.sql (PostgreSQL 18.6) et vérifier les empreintes md5 : commandes 7505112bb794…, lignes e989d5bec39b… (P-14).
2	Démarrer l'API (Node 22.14, pool de 5 connexions, port 3001) puis lancer la campagne de charge décrite en annexe p. 68 et 71 (≈ 16 min : 1/5/10/20 clients, 10 s après 3 s, 3 essais alternés).
3	Comparer l'ordre de grandeur et les intervalles, pas la valeur exacte : plafond du N+1 ≈ 250 req/s et du groupé ≈ 1 030 req/s, essais disjoints. Un autre matériel donne d'autres valeurs absolues.

Conséquence pour l'API : « Qu'est-ce que votre décision change ? »

Contrat	inchangé : même JSON, mêmes droits (401 / 400 / 400) (P-03, P-04)
Disponibilité	2 allers-retours par page au lieu de 21 : à 5 connexions le débit est ×4,2 ; Redis arrêté : l'API répond 200 en repli sur PostgreSQL (P-10)
Fraîcheur	fiche produit : copie jusqu'à 60 s, périmée après une modification SQL directe (57.50 pour 19.90) ; l'achat lit PostgreSQL (P-09)
Ressources	aucun index ni écriture en plus ; au plus 100 identifiants par requête de lignes ; en cas de repli, une lecture SQL par appel (P-10)

Checklist de remise

Vérification	OK	Où
Besoin et état initial compréhensibles sans explication orale	✔	A
Architecture et stockage réellement déployés visibles	✔	B, P-13
Chaque chiffre annoncé renvoie à une mesure réelle et à son protocole	✔	P-01 à P-14
Benchmark avant/après aux mêmes conditions sauf le changement étudié	✔	D, P-05
p95, débit et erreurs présents ; unités indiquées	✔	D
Résultat fonctionnel vérifié, pas seulement la vitesse	✔	E, P-03, P-04
Preuve de fiabilité ou de fraîcheur documentée	✔	E, P-09 à P-11
Variante rejetée et risque restant expliqués	✔	F, P-06, P-07
Conclusion limitée à la machine, au jeu et à la charge testés	✔	F
Sorties brutes, plans, traces ou TraceId conservés	✔	annexes, D-9

Formulation finale

1. Observé. À 20 clients, GET /commandes?limit=20 en N+1 plafonne à 246,9 req/s (p95 98,0 ms, file du pool jusqu'à 15, CPU 50 %, 0 erreur) pour 21 requêtes SQL par page.

2. Pourquoi. Chaque page paie 21 allers-retours, environ 0,68 ms chacun côté API contre 0,018 ms côté PostgreSQL ; le chargement groupé les ramène à 2 sans changer la réponse, d'où 1032,1 req/s et un p95 de 26,8 ms (3 essais disjoints). Agrandir le pool ou ajouter PgBouncer ne produit pas cet effet.

3. Conditions. Je recommande le chargement groupé sur cette machine, ce jeu et 1 à 20 clients ; le cache de la fiche produit est acceptable avec un TTL de 60 s car il peut servir une valeur périmée (57.50 pour 19.90) ; il reste à tester plusieurs instances d'API, un réseau réel, des mesures de 30 s et la production.

Pièces jointes : où trouver la sortie brute de chaque preuve

Ce fichier contient la couverture, le contexte, l'organisation, le dossier (D-1 à D-9), puis les annexes : pages 1 à 71 d'origine, sauf les pages 15, 24, 25 et 67. **« annexe p. N »** désigne la page portant le numéro **N** en haut ; le sommaire (page 13) donne sa place dans ce fichier (colonne « Fichier p. »).

Preuve	Sortie brute dans ce fichier (page de l'annexe)
P-01 P-05 P-06 P-12	p. 63-64 (courbes, séries C et D) ; p. 70-71 (11.7 : chaque essai)
P-02	p. 68 (11.2 pg_stat_statements, 11.3 journal lent) ; p. 20 et 53 (sqlMs par TraceId)
P-03 P-04	p. 19-20 (comptage, droits) ; p. 53-54 (9.3 à 9.6)
P-07	p. 21 ; p. 55 (9.8 campagnes brutes)
P-08	p. 23 ; p. 57-58 (10.3, 10.6)
P-09 P-10	p. 22-23 ; p. 58 (10.4, 10.5)
P-11	p. 23 ; p. 59 (10.7 rafale)
P-13	p. 7-14 et 16-18 (ateliers 3, 4 et 4 bis) ; p. 48-52 (annexe 8)
P-14	p. 68 (11.1 environnement) ; p. 71 (11.8 rejouer)

Ce qui n'est pas dans les pièces

Réplication, partitionnement et sharding : aucune preuve produite. Journaux de latence par transaction de pgbench (séries SQL) : non conservés, seules leurs statistiques le sont (annexe p. 68).

Sommaire détaillé des ateliers et des annexes

« Annexe p. N » = numéro en haut de la page (numérotation d'origine, pages 15, 24, 25 et 67 non reprises). « Fichier p. » = place dans ce PDF.

Chapitre	Annexe p.	Fichier p.	Contenu	Lié à
ATELIERS : CHAPITRES				
Atelier 1 · Diagnostic initial	1-2	**14-15**	deux requêtes (historique du client 42, indicateurs mensuels), résultats vérifiés, 5 mesures, nœud coûteux et hypothèse	annexe 1
Atelier 2 · Réécriture équivalente	3-6	**16-19**	jointure contre EXISTS (27,6 → 11,3 ms), total par commande, optimisations testées sur copie (work_mem, statistique), bilan	annexes 2 et 3
Atelier 3 · Historique client	7-10	**20-23**	index simple, composé, couvrant ; requête réelle de l'API ; piège de l'alias dans ORDER BY ; coût d'écriture ; tableau de décision	annexes 4 et 5
Atelier 4 · Index spécialisés	11-14	**24-27**	index partiel (10,1 → 0,070 ms), GIN, GiST ; à quelle requête répond chacun et à quel prix	annexes 6 et 7
Atelier 4 bis · Migration de schéma	16-18	**28-30**	colonne ajoutée sans bloquer : remplissage par lots, verrous A, B, C, retour arrière (46,3 ms contre 3 324,1 ms)	annexe 8
Atelier 7 · Pagination, N+1, PgBouncer	19-21	**31-33**	21 → 2 requêtes, curseur contre OFFSET (944,3 → 0,177 ms), dates identiques, droits, PgBouncer	annexe 9
Atelier 8 · Cache Redis et fraîcheur	22-23	**34-35**	cache-aside, miss et hit (2,7 → 0,988 ms), copie périmée, invalidation, repli, course	annexe 10
PREUVES DES ATELIERS 1 À 4 BIS, 7 ET 8				
Annexe 1 · Plans de l'atelier 1	26	**36**	plans EXPLAIN (ANALYZE, BUFFERS) complets	
Annexes 2 et 3 · Atelier 2	27-29	**37-39**	plans de la jointure, de EXISTS, des optimisations O1 et O2	
Annexes 4 et 5 · Atelier 3	30-39	**40-49**	plans de la requête de l'API, preuves des mesures	
Annexes 6 et 7 · Atelier 4	40-47	**50-57**	plans de chaque index, preuves des mesures	
Annexe 8 · Atelier 4 bis	48-52	**58-62**	traces SQL, volumes, contrainte, verrous, retour arrière	
Annexe 9 · Atelier 7	53-56	**63-66**	SQL émis, traces par TraceId, 30 tours alternés, OFFSET et curseur, PgBouncer brut	
Annexe 10 · Atelier 8	57-59	**67-69**	code du cache, paires miss / hit, TTL, expiration, rafale	
JOUR 5 : MONITORER ET PROUVER LES GAINS				
Jour 5 · Chapitre	60-66	**70-76**	tableau de bord et requêtes coûteuses ; tuning (work_mem, cache, connexions, VACUUM) ; courbe de charge 1 à 20 clients ; lecture avant / après ; OLTP et analytique ; conclusion limitée	annexe 11
Annexe 11 · Jour 5	68-71	**77-80**	11.1 environnement · 11.2 pg_stat_statements · 11.3 journal lent · 11.4 à 11.6 tuning · 11.7 mesures brutes de chaque essai · 11.8 export et reproductibilité	

Atelier 1 : diagnostic initial

ShopFlow · PostgreSQL 18.6 · résultats et interprétation

Contexte. PostgreSQL 18.6 (Docker postgres:18, WSL2), base et schéma shopflow, chargés par 01_schema.sql et 02_donnees.sql : 1 000 clients, 200 produits, 100 000 commandes, 300 000 lignes. work_mem = 4 MB, shared_buffers = 128 MB. Aucun index ajouté : seuls ceux des contraintes (clés primaires, UNIQUE). Mesures à chaud, le 05/10/2026.

1. Les deux requêtes

Q1 : historique du client 42 (page d'API, très sélective)

```
SELECT id, created_at, total
FROM shopflow.commandes
WHERE client_id = 42
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

Q2 : indicateurs mensuels (lit toute la table)

```
SELECT date_trunc('month', created_at) AS mois,
       statut,
       count(*)   AS nb_commandes,
       sum(total) AS chiffre_affaires
FROM shopflow.commandes
GROUP BY mois, statut
ORDER BY mois, statut;
```

2. Résultats fonctionnels vérifiés

On vérifie l'exactitude avant de mesurer : ces chiffres sont la référence qui doit rester identique, au centime, après toute optimisation (slides 6 et 24).

Req.	Lignes	Total	Contrôle de référence	OK
Q1	20	14 400,00 €	Le client 42 a 100 commandes (47 760,00 €), la page en garde 20	✔
Q2	30 groupes	29 775 001,25 €	sum(total) sur toute la table : 100 000 commandes, 29 775 001,25 €. Écart 0	✔

3. Mesures : 5 répétitions après échauffement

3 exécutions d'échauffement, puis 5 avec EXPLAIN (ANALYZE, BUFFERS). Execution Time en ms. Les 5 plans de chaque requête sont identiques.

Req.	#1	#2	#3	#4	#5	Médiane	Buffers
Q1	0,207	0,267	0,234	0,208	0,229	**0,229**	hit=99
Q2	84,3	84,1	80,5	86,2	78,7	**84,1**	hit=1 680, temp 380/381

Froid ou chaud
Première exécution de Q2, à froid : 1 096,8 ms (shared read=1330). À chaud : ≈ 84 ms (shared read=0). Même plan, 13 fois plus rapide : l'état du cache doit toujours être précisé (slide 11).

4. Nœud coûteux et hypothèse

Q2 : agrégation mensuelle

Nœud (lecture de bas en haut)	Temps propre	Part
Seq Scan on commandes (100 000 lignes)	≈ 25 ms	33 %
Sort : external merge, Disk 3 040 kB	**≈ 37 ms**	**48 %**
GroupAggregate (30 groupes)	≈ 15 ms	20 %

Nœud coûteux : le Sort. Il trie 100 000 lignes pour produire 30 groupes et déborde sur disque (381 pages temporaires). Le Seq Scan est normal, puisque toute la table est nécessaire.

Hypothèse : work_mem (4 MB) est trop petit pour ce tri, **et** le planificateur estime 80 048 groupes au lieu de 30, faute de statistique sur date_trunc('month', created_at). Il choisit donc de trier plutôt que de faire un HashAggregate en mémoire. **Testé ensuite sur une copie :** SET work_mem = '32MB' et une statistique sur l'expression, voir « Optimisations testées » (hypothèse confirmée, gain ≈ ×1,7 à ×1,9).

Q1 : historique du client 42

Déjà rapide (≈ 0,23 ms) et l'estimation est exacte (100 lignes prévues, 100 lues). L'accès passe par l'index unique (client_id, cle_idempotence) issu d'une contrainte. **Nœud le plus coûteux : le Bitmap Heap Scan**, qui lit 88 pages pour 100 commandes dispersées, puis un tri n'en garde que 20. **Hypothèse :** le coût croît avec le nombre de commandes du client, parce que l'index ne contient pas created_at. Testé à l'Atelier 3 : un index sur (client_id, created_at DESC, id DESC) (×2,5 sur la requête de l'API, WAL +35 % à l'écriture).

Atelier 2 : une réécriture équivalente

Mêmes données, mêmes règles : on prouve l'équivalence du résultat avant de comparer les durées

Principe (slide 31). Réécrire une requête n'est valable que si le résultat pertinent est identique. Une version plus rapide qui perd ou duplique des lignes est **rejetée**. Cas de test choisis : le **client 42** (100 commandes) et la **commande 42** (3 lignes).

A. Jointure qui duplique les clients, contre EXISTS

Question métier : quels clients ont au moins une commande payée ? Une ligne du résultat doit identifier **un client** (slides 24 et 27).

Avant : jointure (A1)

```
SELECT cl.id, cl.nom
FROM shopflow.clients cl
JOIN shopflow.commandes c ON c.client_id = cl.id
WHERE c.statut = 'payee';
```

Après : EXISTS (A3)

```
SELECT cl.id, cl.nom
FROM shopflow.clients cl
WHERE EXISTS (SELECT 1 FROM shopflow.commandes c
              WHERE c.client_id = cl.id AND c.statut = 'payee');
```

Variante	Lignes	Clients distincts	Verdict
A1 jointure	80 000	800	**Rejetée** : le client 42 apparaît 100 fois
A2 jointure + DISTINCT	800	800	Correcte, mais construit 80 000 lignes puis déduplique
A3 EXISTS	800	800	Correcte, une ligne par client

Contrôle du contenu : la différence symétrique entre A2 et A3 (EXCEPT dans les deux sens) renvoie **0 ligne**. Les deux listes de clients sont donc identiques.

Variante	#1	#2	#3	#4	#5	Médiane
A1 jointure	27,8	27,4	27,6	28,4	26,4	**27,6**
A2 + DISTINCT	49,2	40,2	35,3	37,7	42,7	**40,2**
A3 EXISTS	11,3	13,3	11,7	11,3	10,8	**11,3**

Durées en ms, 5 répétitions après 3 exécutions d'échauffement.

Interprétation
A3 est la plus rapide (≈ 11 ms contre ≈ 27 ms) et la seule à répondre directement à la question. PostgreSQL la transforme en Nested Loop Semi Join : pour chaque client, il s'arrête dès qu'il trouve une commande payée (0,8 ligne lue en moyenne par client). La jointure construit 80 000 lignes (Hash Join après un Seq Scan de toute la table). Avec DISTINCT, il faut en plus un HashAggregate : c'est la plus lente. Une sous-requête corrélée n'implique donc pas une exécution complète par ligne (slide 34, question 6).

B. Total par commande sans multiplier commandes.total

Piège (slide 24) : joindre commandes à lignes donne une ligne par **ligne de commande**, pas par commande. Chaque commande a 3 lignes, donc sum(c.total) compte chaque total 3 fois.

Avant : faux (B1)

```
SELECT c.id, sum(c.total) AS total          -- FAUX
FROM shopflow.commandes c
JOIN shopflow.lignes l ON l.commande_id = c.id
WHERE c.client_id = 42
GROUP BY c.id;
```

Après : somme des lignes (B2)

```
SELECT c.id, sum(l.qte * l.prix_unitaire) AS total   -- CORRECT
FROM shopflow.commandes c
JOIN shopflow.lignes l ON l.commande_id = c.id
WHERE c.client_id = 42
GROUP BY c.id;
```

Contrôle (client 42)	Nb cmd	Total	Verdict
Référence : commandes.total	100	47 760,00 €	—
B1 sum(c.total)	100	143 280,00 €	**Rejetée** : total × 3
B2 sum(qte × prix_unitaire)	100	47 760,00 €	Correcte : égale à la référence

Exemple sur la commande 42 : B1 donne **720,00 €** (3 × 240), B2 donne **240,00 €** (58,75 + 80,00 + 101,25), et commandes.total vaut 240,00 €. Comparaison commande par commande de B2 avec commandes.total : **0 écart** sur les 100 commandes.

Variante	#1	#2	#3	#4	#5	Médiane
B1 (faux)	1,15	1,10	1,12	1,10	1,01	**1,10**
B2 (correct)	2,21	2,01	2,02	2,17	2,05	**2,05**

Durées en ms, 5 répétitions après échauffement.

Interprétation
B1 est plus rapide (≈ 1,1 ms contre ≈ 2,1 ms), mais son résultat est faux : on la rejette. B1 n'a besoin que de la clé de la ligne : PostgreSQL utilise un Index Only Scan sur lignes, sans lire la table. B2 doit lire qte et prix_unitaire, donc un Index Scan avec accès à la table, puis un Sort avant le regroupement. Le surcoût de B2 (≈ 1 ms) est le prix de l'exactitude : on ne choisit pas une requête uniquement sur sa durée (slide 6).

Conclusion de l'atelier

1. **EXISTS** remplace avantageusement la jointure quand on veut un résultat par client : correct et 2,4 fois plus rapide ici.
2. Pour un total par commande, on **agrège à la bonne granularité** (les lignes), jamais commandes.total après une jointure un-à-plusieurs.
3. Dans les deux cas, l'équivalence a été prouvée par un contrôle de contenu **avant** de comparer les temps. Les plans des variantes sont en annexe.

Optimisations testées : le bilan

Chaque optimisation est mesurée avec le même protocole, avec un résultat identique à l'original

Cadre. L'atelier 1 interdit d'ajouter un index ou une statistique dans la base du laboratoire. Les pistes ont donc été testées sur une **copie** (shopflow_opt, mêmes 100 000 commandes), **un seul changement à la fois**, avec 3 exécutions d'échauffement puis 5 mesures (médiane). La base shopflow du laboratoire n'a pas été touchée. **Équivalence prouvée :** pour chaque variante, l'empreinte md5 du résultat complet est identique à celle de la requête d'origine.

Q2 : agrégation mensuelle (le problème était le tri qui déborde sur le disque)

Optimisation	Ce qui change dans le plan	Médiane	Gain
Référence (copie)	Sort external merge sur disque ; estimation 80 738 groupes	84,6 ms	—
O1 : SET work_mem = '32MB'	HashAggregate en mémoire ; le tri ne porte plus que sur 30 lignes ; estimation toujours fausse (80 738)	50,2 ms	×1,7
O2 : CREATE STATISTICS sur date_trunc, puis ANALYZE	Estimation corrigée : 30 groupes, HashAggregate de 40 kB	45,5 ms	×1,9
O1 + O2	Même plan que O2	47,9 ms	×1,8 (pas mieux que O2 seule)

Mon hypothèse de l'atelier 1 est confirmée. Les deux causes jouaient chacune : donner plus de mémoire ou corriger l'estimation suffit à remplacer le gros tri par un HashAggregate. **O2 corrige la cause** (PostgreSQL se trompait sur le nombre de groupes), **O1 contourne le symptôme** (le tri n'avait plus à déborder). Les deux gains sont du même ordre (≈ 1,7 à 1,9 fois). **Coût :** O1 augmente la mémoire autorisée par opération de tri ou de hachage et par connexion : à réserver à une session ou une requête précise, pas à tout le serveur. O2 a un coût faible (une statistique à maintenir lors des ANALYZE), mais je ne l'ai pas mesuré. **Recommandation : O2.**

Q1 : historique du client 42

L'optimisation de cette requête (index) est traitée dans l'**Atelier 3**, plus bas : mesurée sur la vraie table, avec la requête exacte de l'API, plusieurs clients, un client très actif et le coût d'écriture. Les mesures préliminaires faites sur la copie y sont remplacées.

Gains obtenus par réécriture (atelier 2), sans aucun coût d’index

Réécriture	Avant	Après	Gain
Clients ayant une commande payée : EXISTS au lieu de la jointure	27,6 ms (80 000 lignes, doublons)	11,3 ms (800 lignes)	×2,4
Total par commande : somme des lignes au lieu de commandes.total	total faux (× 3)	total correct	exactitude

Atelier 3 : l’historique client

Quel index pour l’historique d’un client, et à quel prix ? Comparaison mesurée de quatre états de la base

Consigne (slide 12). Comparer la base initiale, un index simple sur client_id, l’index composé et sa variante couvrante, sur plusieurs clients dont le 42, comparer un lot d’insertions, expliquer le gain ou l’absence de gain, et livrer un tableau de décision. **Documentation complète, scripts et mesures brutes : dossier atelier3/ ; preuves chiffrées : annexe 5.**

Protocole. Les variantes sont créées sur la vraie table du laboratoire puis supprimées une par une (jamais deux ensemble). 3 échauffements puis 5 mesures par requête et par client (75 combinaisons), plus une **mesure renforcée en alternance** (51 tours, une table de travail par variante) car les durées sub-milliseconde sont bruitées : la même requête a pris 0,23 ms à l’atelier 1 et ≈ 0,55 ms aujourd’hui. **Le laboratoire a été vérifié avant et après : il a retrouvé exactement son état initial** (6 index, aucune statistique). Résultat fonctionnel identique (empreinte md5) pour les 75 combinaisons, les 51 tours et le client très actif.

A. Les requêtes testées : le SQL réellement émis par l’application

Slide 13 : la compatibilité d’un index se teste avec le SQL émis par l’application. J’ai donc lu 01_server/api/commandes.mjs et testé, en plus de la requête du cours, la requête exacte de l’API (première page, 21 lignes) et sa page suivante par curseur.

```
SELECT id::text AS id,
       to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS created_at,
       statut, total::text AS total
FROM shopflow.commandes AS commandes
WHERE client_id = 42
ORDER BY commandes.created_at DESC, commandes.id DESC      -- qualificatif indispensable (voir F)
LIMIT 21;
```

La page suivante ajoute AND (created_at, id) < ('<date de la 20e ligne>', <id>).

B. Les variantes (index créés puis supprimés)

Variante	Définition	Taille	Part de la table
Index unique existant	(client_id, cle_idempotence), issu de la contrainte	1 864 kB	13,9 %
Simple	(client_id)	688 kB	5,1 %
Composé	(client_id, created_at DESC, id DESC)	3 984 kB	29,7 %
Couvrant (statut, total)	composé INCLUDE (statut, total)	5 792 kB	43,2 %
Couvrant (total)	composé INCLUDE (total)	4 864 kB	36,3 %

Table commandes : 100 000 lignes 1 674 pages 13 392 kB. Chaque client a exactement 100 commandes.

C. Lecture pour le client 42 (51 tours en alternance, médiane en ms)

Variante	Plan (requête API)	Buffers	Cours	API	Curseur	Gain API
Initiale	Bitmap Heap Scan + Sort	96	0,546	0,690	0,690	—
Simple (client_id)	Bitmap Heap Scan + Sort	96	0,548	0,729	0,707	≈ ×1 (aucun)
Composé	Index Scan, sans Sort	27	0,161	0,274	0,342	×2,5
Couvrant (statut, total)	Index Only Scan, sans Sort	7	0,094	0,199	0,201	×3,5
Couvrant (total)	Index Scan, sans Sort	27	0,086	0,267	0,303	×2,6

Index simple : aucun gain. La base a déjà un index qui commence par client_id (l'index unique). PostgreSQL utilise le nouvel index, plus petit (688 kB contre 1 864 kB), mais **le plan est identique** : il retrouve les commandes du client sans pouvoir les donner dans l'ordre, lit les 100 et les trie pour en garder 20. **Composé :** l'égalité sur client_id borne le groupe, puis l'index donne déjà l'ordre demandé : 21 entrées lues, plus de Sort. **Couvrant (statut, total) :** toutes les colonnes sont dans l'index, donc plus aucune lecture de table (7 buffers). **Couvrant (total) :** bon pour la requête du cours, mais l'API renvoie aussi statut : il faut retourner à la table, donc **aucun gain de plus que le composé** pour le SQL réellement émis.

D. Cinq clients (protocole du cours, 5 mesures, requête API, médiane en ms)

Variante	Client 1	Client 42	Client 250	Client 500	Client 1000	Buffers
Initiale	0,799	0,786	0,803	0,884	0,771	99 à 109
Simple (client_id)	0,766	0,847	0,701	0,771	0,769	99 à 109
Composé	0,256	0,321	0,281	0,358	0,273	27
Couvrant (statut, total)	0,160	0,161	0,172	0,168	0,203	7
Couvrant (total)	0,297	0,257	0,309	0,276	0,269	27

Les 75 combinaisons (5 variantes × 3 requêtes × 5 clients) renvoient exactement le même résultat que la base initiale. Tous les clients ayant 100 commandes, les écarts entre clients sont du bruit : le jeu ne contient aucun client très actif.

E. Un client très actif (client 7 porté à 20 100 commandes, requête API)

Variante	Médiane	Buffers	Gain
Initiale	23,3 ms	287	—
Simple (client_id)	23,3 ms	287	—
Composé	0,256 ms	27	×91,1
Couvrant (statut, total)	0,189 ms	7	×123,3
Couvrant (total)	0,276 ms	27	×84,5

Sans index adapté, le coût croît avec le nombre de commandes du client : 96 buffers pour 100 commandes, 287 pour 20 100, 23 ms. Avec le composé, PostgreSQL lit toujours 21 entrées : le coût **ne dépend plus du volume**. C'est l'argument principal du choix : à 100 commandes le gain est de 0,4 ms, il devient de 23 ms pour un client actif.

F. Le piège de l’alias dans ORDER BY

Variante	API qualifiée	API sans qualificatif	Buffers	Plan sans qualificatif
Initiale	0,690 ms	0,877 ms	96 / 93	Bitmap Heap Scan + Sort
Composé	0,274 ms	0,783 ms	27 / 93	Sort, index composé non utilisé
Couvrant (statut, total)	0,199 ms	0,382 ms	7 / 11	Index Only Scan puis Sort

Les colonnes de sortie de l’API s’appellent aussi id et created_at (des textes formatés). Dans ORDER BY, un nom sans qualificatif désigne d’abord la colonne **de sortie** : le tri porterait sur les textes et aucun index ne peut le fournir. C’est pourquoi l’API écrit commandes.created_at. Le résultat est le même, mais **l’index composé ne sert plus à rien** (0,783 ms, 93 buffers, comme sans index). Un bon index peut être rendu inutile par une réécriture anodine du SQL.

G. Coût d’écriture : 20 000 insertions dans une transaction annulée (15 tours en alternance)

Variante	Index	WAL (octets)	Écart WAL	Enreg. WAL	Durée médiane
Initiale	2	4 623 674	—	60 322	126,8 ms
Simple (client_id)	3	5 927 578	+28,2 %	80 327	180,9 ms
Composé	3	6 240 760	+35,0 %	80 322	194,0 ms
Couvrant (statut, total)	3	6 570 048	+42,1 %	80 322	219,0 ms
Couvrant (total)	3	6 400 760	+38,4 %	80 323	187,8 ms

Lot mesuré, dans une transaction de laboratoire annulée (une table de travail par variante : mêmes 100 000 lignes, mêmes contraintes) :

```
BEGIN;
EXPLAIN (ANALYZE, BUFFERS, WAL)
INSERT INTO a3_tmp.t_<variante>
SELECT id + 1000000, client_id, created_at, statut, total, NULL
FROM a3_tmp.t_<variante> WHERE id <= 20000;          -- 20 000 lignes
ROLLBACK;                                            -- rien n'est conservé
```

Le volume de WAL est la mesure fiable. Chaque index ajouté ajoute environ 20 000 enregistrements, un par ligne insérée (60 322 = 3 × 20 000 + 322 avec la table, la clé primaire et l’index unique ; ≈ 80 322 avec un index de plus). **Les durées sont très bruitées** : sur trois exécutions complètes du protocole, la surcharge de l’index composé a varié de +23 % à +62 %. Je retiens donc : tout index ajoute du travail à chaque insertion, de l’ordre de **+30 à +40 % de WAL** ici, et plus l’index est large, plus c’est cher.

H. Un index couvrant garantit-il zéro accès à la table ? (slide 9)

Étape (table jetable)	Plan	Heap Fetches	Buffers
Après VACUUM	Index Only Scan	0	4
Après UPDATE des 100 commandes du client 42, sans VACUUM	Index Only Scan	**40**	48
Après un nouveau VACUUM	Index Only Scan	0	8

Non. Dès que des lignes sont modifiées, PostgreSQL doit visiter la table jusqu’au prochain VACUUM. Sur la vraie table Heap Fetches vaut 0 parce qu’elle était entièrement « visible » avant l’atelier. Les 40 visites pour 20 lignes renvoyées s’expliquent probablement par les anciennes versions encore présentes dans l’index (hypothèse non vérifiée).

I. Tableau de décision

Critère	Simple	Composé	Couvrant (statut, total)	Couvrant (total)
Supprime le Sort	non	**oui**	oui	slide : oui ; API : non
Gain API, client 42	aucun	×2,5	×3,5	×2,6
Gain, client à 20 100 commandes	aucun	×91	×123	×84
Résultat identique	oui	oui	oui	oui
Taille de l'index	688 kB	3 984 kB	5 792 kB	4 864 kB
Surcoût d'écriture (WAL)	+28 %	+35 %	+42 %	+38 %
Dépend de VACUUM	non	non	oui	oui
Décision	**Rejeté**	**Retenu**	**Option**	**Rejeté**

Pourquoi le composé et pas le couvrant ? Le couvrant gagne encore 0,075 ms par appel (0,274 → 0,199 ms), soit moins que le gain du composé lui-même (0,416 ms). En échange il est 45 % plus gros, coûte plus en écriture et perd de son intérêt quand la table est modifiée. **Pourquoi rejeter le couvrant (total) ?** Il est inutile pour le SQL réellement émis. **Conditions de remise en cause :** si les créations de commandes dominent nettement, retirer l'index ; si l'endpoint devient un point chaud avec peu de mises à jour, envisager le couvrant. La charge réelle est inconnue : la décision est argumentée, pas définitive.

Migration retenue (testée sur une base jetable : montée, rejeu sans erreur, contrôle de validité, retour arrière)

```
-- 001 UP (labo)
SET lock_timeout = '5s';
CREATE INDEX IF NOT EXISTS idx_hist_client
  ON shopflow.commandes (client_id, created_at DESC, id DESC);
-- Déploiement : CREATE INDEX CONCURRENTLY, hors BEGIN/COMMIT, puis contrôle de indisvalid.

-- 001 DOWN (retour arrière)
DROP INDEX IF EXISTS shopflow.idx_hist_client;
```

La migration n'est **pas appliquée** au laboratoire (l'atelier demande une décision, et la base sert aux ateliers suivants). Les scripts (laboratoire, déploiement, vérification, retour arrière) sont dans atelier3/migration/.

Atelier 4 : index spécialisés

Index partiel, GIN et GiST : à quelle requête répond chacun, et à quel prix

Consigne (slide 26). Créer les variantes partielle et GIN, les comparer avec un filtre qui correspond au prédicat et un filtre qui n'y correspond pas, observer la taille et l'usage des index ; sur la petite table produits un Seq Scan peut rester rationnel (l'expliquer et proposer un test à plus grand volume) ; pour GiST, deux périodes qui se chevauchent et une disjointe. Livrable : choix d'index, opérateurs, résultats et conditions. **Documentation complète : dossier atelier4/ ; preuves chiffrées : annexe 7.**

Protocole. Même protocole que l'atelier 3 : variantes créées sur la vraie table puis supprimées une par une (5 mesures après 3 échauffements), mesures alternées sur des tables de travail (51 tours, sans parallélisme), lots d'écriture annulés (15 tours, volume de WAL). **Le laboratoire a été vérifié avant et après : état initial retrouvé.** Deux défauts de mesure ont été détectés puis corrigés pendant l'atelier : mon premier script ne lisait que les buffers « hit » (un plan affichait hit=841 read=1554, soit 2 395 pages), et PostgreSQL avait choisi un Parallel Seq Scan sur une table et un Seq Scan simple sur une autre, deux plans non comparables. Les chiffres ci-dessous viennent de l'exécution corrigée. L'API n'émet aucune de ces requêtes : ce sont celles du cours.

Situation	ShopFlow grossit. L'équipe veut accélérer trois traitements **sans ralentir les créations de commandes** : la file des commandes à traiter, le filtre du catalogue sur les attributs JSONB et, plus tard, une fonction de réservation. On teste **un index par besoin**, on mesure le gain **et** son coût, et on garde le moins d'index possible (slide 44 : « on peut garder moins d'index que dans le laboratoire »).

A. Index partiel : la file des commandes en attente

Situation	Un opérateur traite chaque jour les 100 plus anciennes commandes en_attente. Elles sont 10 000 sur 100 000 (10 %). Sans index, PostgreSQL lit toute la table (1 680 buffers, 10,1 ms) puis trie.
Solution choisie et pourquoi	**Index partiel** (created_at, id) WHERE statut = 'en_attente'. **Pourquoi :** (1) le besoin ne concerne qu'un statut, il est inutile d'indexer les 90 % de lignes restantes ; (2) l'index donne déjà l'ordre demandé : PostgreSQL lit les 100 premières entrées sans trier ; (3) il est 12,4 fois plus petit que l'index complet (328 kB contre 4 072 kB) et ajoute beaucoup moins de WAL à l'insertion (+3,2 % contre +36,5 %). **Écartée :** l'index complet (statut, created_at, id), qui fonctionne aussi mais pour un besoin limité à un statut.

```
SELECT id FROM shopflow.commandes
WHERE statut = '<statut>'      -- 'en_attente' correspond au prédicat ; 'payee' et 'annulee' non
ORDER BY created_at, id LIMIT 100;

CREATE INDEX a4_idx_partiel ON shopflow.commandes (created_at, id) WHERE statut = 'en_attente';
CREATE INDEX a4_idx_complet ON shopflow.commandes (statut, created_at, id);   -- comparaison
```

Variante (5 mesures, médiane en ms / buffers)	en_attente	payee	annulee
Initiale	10,1 / 1 680 (Seq Scan)	17,4 / 1 720 (Seq Scan)	10,9 / 1 680 (Seq Scan)
Partiel	0,070 / 3 (Index Only Scan)	14,7 / 1 720 (Seq Scan)	10,0 / 1 680 (Seq Scan)
Complet	0,078 / 4 (Index Only Scan)	0,095 / 4 (Index Only Scan)	0,106 / 4 (Index Only Scan)

Mesure renforcée (51 tours, médiane / p95 en ms)	en_attente	payee	Buffers
Initiale	9,9 / 11,9	22,2 / 26,2	846
Partiel	**0,071 / 0,141** (×139,4)	21,9 / 25,1 (×1,0)	3 ; 846
Complet	0,084 / 0,201 (×117,8)	0,082 / 0,140 (×271,0)	4 ; 4

Le partiel ne sert que son prédicat. Pour en_attente il lit 3 buffers au lieu de 1 680 ; pour payee et annulee le plan est **identique** à la base initiale (Seq Scan + tri), l'écart de durée est du bruit. Le complet sert tous les statuts. **Taille :** partiel 328 kB (2,4 % de la table), complet 4 072 kB (30,4 %), soit 12,4 fois plus. **Usage** (pg_stat_user_indexes.idx_scan) : partiel 9 scans, uniquement la requête en_attente (3 + 1 + 5 exécutions) ; complet 27 scans, les trois statuts (3 × 9).

Le piège de la requête paramétrée (slide 13). Avec PREPARE et statut = $1 : plan **générique** forcé → l'index partiel **n'est pas utilisé** (Seq Scan + tri, 1 680 buffers) ; plan spécifique forcé → Index Only Scan, 3 buffers. L'index complet marche dans les deux cas. PostgreSQL ne peut pas prouver qu'un paramètre inconnu implique le prédicat : l'application doit envoyer le littéral. **Prédicat avec now() (slide 14) :** refusé, « functions in index predicate must be marked IMMUTABLE ».

Coût d'écriture (WAL, 15 tours)	Initiale	Partiel	Complet
Insertion de 20 000 lignes (2 000 en_attente)	4 621 038	4 768 498 (**+3,2 %**)	6 305 626 (**+36,5 %**)
2 000 lignes quittent la file (en_attente → payee)	605 083	605 083 (**+0,0 %**)	771 943 (**+27,6 %**)
2 000 lignes entrent dans la file (payee → en_attente)	615 355	769 824 (**+25,1 %**)	797 436 (**+29,6 %**)

À l'insertion le partiel n'écrit que pour les lignes en_attente (environ 2 000 enregistrements de plus, le complet environ 20 000). Une ligne qui quitte la file ne crée aucune entrée dans le partiel (WAL identique à la base initiale) ; l'ancienne entrée n'est probablement retirée que par VACUUM (hypothèse non vérifiée). Une ligne qui entre dans la file en crée une.

Est-ce que ça fonctionne ?	**Oui, pour son périmètre.** Gain ×144,6 (10,1 → 0,070 ms), 1 680 → 3 buffers, résultat identique à la base initiale (empreinte md5), usage prouvé (idx_scan = 9). **Limites vérifiées :** il ne sert ni payee ni annulee (Seq Scan, même plan qu'initialement) ; il n'est **pas** utilisé avec un plan générique (1 680 buffers) : l'application doit envoyer le littéral 'en_attente'. **Verdict : retenu**, à condition que la requête contienne le littéral et que la file reste une petite part de la table.

B. GIN : le filtre JSONB sur produits.attributs

Situation	Le catalogue stocke des attributs variables en JSONB (categorie, couleur). Un filtre par catégorie est envisagé (slide 16). Aujourd'hui : 200 produits sur 3 pages ; à terme le catalogue peut grossir.
Solution choisie et pourquoi	**Décision en deux temps.** Aujourd'hui : aucun index, car lire 3 pages (6 buffers, ≈ 0,1 ms) coûte moins que passer par un index, qui ne serait jamais utilisé. À volume, si le filtre porte toujours sur categorie : index d'expression ou colonne typée, plus rapides (×28,0 et ×30,4) et moins coûteux en écriture (+34,6 % et +38,9 % de WAL à l'insertion contre +120,7 % pour le GIN). **Écartée :** le GIN, réservé aux filtres sur des clés variables que l'on ne peut pas prévoir (slide 17).

```
SELECT id, nom FROM shopflow.produits WHERE attributs @> '{"categorie":"livre"}'::jsonb ORDER BY id;
CREATE INDEX ... USING gin (attributs);                      -- jsonb_ops par défaut
CREATE INDEX ... ((attributs->>'categorie'));                -- index d'expression, pour comparaison
```

Sur la vraie table (200 lignes, 3 pages) : un Seq Scan rationnel. Les 4 requêtes testées (livre, materiel, absent, ->>) donnent le **même plan** dans les trois variantes : Seq Scan, 6 buffers, ≈ 0,1 ms. Les index GIN et d'expression pèsent 16 kB chacun (la table : 24 kB) et n'ont **jamais été utilisés** (idx_scan = 0 et 0). Lire 3 pages coûte moins que passer par un index. En interdisant le Seq Scan (diagnostic de session, jamais laissé) on voit ce que chaque index sait faire : le GIN peut servir @> (valeur absente : coût 12,81 contre 5,51 pour le Seq Scan) mais pas ->> ; l'index d'expression peut servir ->> et pas @>.

Test à plus grand volume (proposé par la slide, réalisé) : 200 000 produits par variante, categorie : 50 % materiel, 25 % accessoire, 24,9 % livre, 0,1 % rare (200 lignes) ; une 4e variante avec une colonne typée categorie. 51 tours, sans parallélisme.

Médiane en ms / buffers	Initiale	GIN	Expression	Colonne typée
@> valeur à 24,9 %	48,7 / 2 855 (Seq Scan)	24,3 / 947 (Bitmap (index))	53,2 / 2 855 (Seq Scan)	49,1 / 3 086 (Seq Scan)
@> valeur rare (0,1 %)	43,9 / 2 855 (Seq Scan)	2,6 / 257 (Bitmap (index))	46,3 / 2 855 (Seq Scan)	43,4 / 3 086 (Seq Scan)
->> 'categorie' = 'rare'	30,9 / 2 855 (Seq Scan)	32,8 / 2 855 (Seq Scan)	1,1 / 203 (Bitmap (index))	1,0 / 203 (Bitmap (index)) (categorie = 'rare')

Le GIN sert @>, selon la rareté de la valeur : ×17,2 pour 0,1 % des lignes, seulement ×2,0 pour 24,9 % (buffers divisés par 3). Il est **inutile pour ->>**. Pour une propriété connue, l'index d'expression (×28,0) et la colonne typée (×30,4) sont plus rapides que le GIN. **Tailles :** GIN 0,93 MiB, expression 1,37 MiB, colonne typée 1,37 MiB (+ 1,80 MiB dans la table).

Coût d'écriture (WAL, 20 000 lignes)	Initiale	GIN	Expression	Colonne typée
Insertion	4 097 786	9 044 567 (**+120,7 %**)	5 515 841 (**+34,6 %**)	5 690 863 (**+38,9 %**)
Modification de attributs	5 547 001	10 120 555 (**+82,5 %**)	6 964 137 (**+25,5 %**)	7 153 583 (**+29,0 %**)

Le GIN est le plus petit des trois index mais le plus coûteux à maintenir : l'insertion écrit plus de deux fois le WAL de la base initiale.

Est-ce que ça fonctionne ?	**Le GIN fonctionne techniquement** : il sert @> (×17,2 sur une valeur à 0,1 %, ×2,0 à 24,9 %), mais **pas pour ce besoin** : sur les 200 produits actuels il n'est jamais utilisé (idx_scan = 0), il ne sert pas ->>, et il écrit +120,7 % de WAL à l'insertion. L'index d'expression et la colonne typée fonctionnent pour le filtre sur categorie (×28,0 et ×30,4). Résultats identiques dans toutes les variantes. **Verdict : GIN rejeté pour le catalogue actuel** ; à reconsidérer si le catalogue atteint des dizaines de milliers de produits avec des filtres sur des clés variables.

C. GiST : réservations et chevauchement

Situation	Une fonction de réservation doit détecter si une période en chevauche une autre. **Ce besoin n'existe pas encore dans ShopFlow** : l'exemple de la slide 18 est indépendant du fil rouge.
Solution choisie et pourquoi	**GiST sur un tstzrange**, avec l'opérateur de chevauchement **&&**. **Pourquoi :** la slide 18 prévoit GiST pour les intervalles, et « l'index doit prendre en charge l'opérateur employé » ; je l'ai vérifié en interdisant le Seq Scan, l'index sert bien &&. Je n'ai testé aucun autre type d'index pour ce besoin.

Table indépendante du fil rouge (id, periode tstzrange), intervalles semi-ouverts [), fenêtre testée 14h-16h : **1** (14h-16h) et **2** (15h-17h) se chevauchent ; **3** (18h-19h) est disjointe ; **4** (16h-17h) est contiguë : sa borne est exclue, elle ne chevauche pas. Résultat de periode && fenêtre : **ids 1, 2**. Sur 4 lignes PostgreSQL choisit un Seq Scan (coût 1,08) ; si on l'interdit, le GiST peut servir l'opérateur && (coût 8,21).

200 000 périodes, fenêtre de 2 h (17 chevauchements)	Sans index	GiST
Lecture : médiane / p95 en ms (buffers)	27,5 / 33,6 (1 474)	**0,168 / 0,294** (8) : **×163,6**
Taille de l'index	n/a	6,95 MiB (60,5 % de la table)
Insertion de 20 000 lignes : WAL	3 029 330	4 669 330 (**+54,1 %**)
Insertion : durée médiane (minimum) en ms	55,6 (32,7)	434,9 (343,6) : **×7,8**

Le GiST rend la recherche de chevauchement quasi instantanée, mais ralentit beaucoup les insertions : le choix se justifie par la fréquence des recherches face à celle des écritures.

Est-ce que ça fonctionne ?	**Oui.** Sur les 4 périodes de test, && retourne exactement les ids 1, 2 (la disjointe et la contiguë sont exclues). Sur 200 000 périodes : gain ×163,6, 1 474 → 8 buffers, résultat identique (empreinte md5). **Coût :** WAL +54,1 %, insertions ×7,8 plus lentes. **Verdict : retenu si le besoin de détection de conflits existe** et si les recherches sont bien plus fréquentes que les insertions.

D. Livrable : choix d’index, opérateurs, résultats et conditions

Besoin	Index et opérateur	Résultat	Condition	Décision
File des commandes en attente	Partiel B-tree (created_at, id) WHERE statut = 'en_attente' ; égalité littérale	×144,6, 3 buffers, 328 kB, WAL +3,2 %	littéral dans la requête, file peu volumineuse	**Retenu**
Même file, tous statuts	Complet B-tree (statut, created_at, id)	4 072 kB, WAL +36,5 %	besoin réel de plusieurs statuts	Rejeté
Attributs (200 produits)	GIN jsonb_ops ; @>	jamais utilisé	table de 3 pages	**Rejeté**
Attributs (volume)	GIN jsonb_ops ; @> seulement	×17,2 (rare), WAL +120,7 %	clés ou valeurs variables	Conditionnel
Propriété connue (categorie)	Expression (attributs->>'categorie') ou colonne typée	×28,0 et ×30,4	filtre toujours sur cette propriété	**Préférable**
Chevaucheme nt de périodes	GiST (periode) ; &&	×163,6, WAL +54,1 %	recherches fréquentes, peu d’écritures	Retenu si le besoin existe

Conditions de remise en cause. Partiel : filtres sur plusieurs statuts, part des commandes en attente devenue importante, ou statut passé en paramètre sans plan spécifique. GIN : catalogue de plusieurs dizaines de milliers de produits avec filtres sur des attributs variables. GiST : insertions de réservations devenues le point chaud. Tous : le coût d’écriture se paie à chaque insertion (slide 44 : « import fréquent : limiter les index »).

Migration retenue (002), testée sur une base jetable : montée, rejeu sans erreur, contrôles, retour arrière, CONCURRENTLY refusé dans une transaction

```
-- 002 UP (labo)                                        -- 002 DOWN (retour arrière)
SET lock_timeout = '5s';                                DROP INDEX IF EXISTS shopflow.idx_attente;
CREATE INDEX IF NOT EXISTS idx_attente ON shopflow.commandes (created_at, id)
  WHERE statut = 'en_attente';
-- Déploiement : CREATE INDEX CONCURRENTLY hors BEGIN/COMMIT, puis contrôle de indisvalid.
-- Contrôle : un plan générique (statut = $1) n'utilise PAS l'index : voir 002_..._verifier.sql.
```

La migration n’est **pas appliquée** au laboratoire. GIN et GiST ne sont pas migrés : GIN est rejeté pour le catalogue actuel, GiST concerne une table absente de ShopFlow. Scripts dans atelier4/migration/.

Atelier 4 bis : migration de schéma compatible

Ajouter newsletter_ok aux clients sans casser l'application et sans bloquer les utilisateurs

Consigne (slides 33 à 43). Sur une copie de shopflow.clients : ajouter la colonne, simuler l'ancien et le nouveau code, remplir les anciennes lignes par lots, renforcer la contrainte, tester l'attente de verrou avec deux sessions, puis exécuter le retour arrière. Fin attendue : champs historiques identiques, aucun NULL, retour arrière fonctionnel. **Les fichiers 01 à 09 du dossier Atelier_4bis ont été rejoués tels quels** (script atelier4bis/run_atelier.py) ; preuves chiffrées : annexe 8 ; documentation complète : dossier atelier4bis/. Le temps de 60 minutes n'est pas respecté (le script rejoue tout en quelques secondes) : la forme et les preuves le sont. **L'ajout de la colonne ne démontre pas un gain de performance** (LIRE_EN_PREMIER.txt) : le gain mesuré plus bas concerne le blocage des autres utilisateurs pendant la migration.

Situation	ShopFlow veut mémoriser le consentement newsletter de chaque client. L'application (ancienne version) lit et écrit id, email et nom ; la nouvelle version lira aussi newsletter_ok. Les deux versions tournent en même temps pendant le déploiement, et la table clients ne doit jamais être indisponible : un ALTER TABLE demande le verrou ACCESS EXCLUSIVE qui bloque même les simples lectures (slides 30 et 31).
Solution choisie et pourquoi	**Migration progressive en 4 temps** (slide 27) : (1) colonne **nullable**, instantanée, l'ancienne version continue de fonctionner ; (2) défaut false pour les écritures suivantes ; (3) remplissage par **lots de 200**, chacun validé seul ; (4) **CHECK … NOT VALID**, puis VALIDATE, puis SET NOT NULL. **Pourquoi :** chaque étape garde un verrou court ou léger. Un SET NOT NULL direct lirait toute la table sous ACCESS EXCLUSIVE ; avec un CHECK déjà validé, PostgreSQL saute ce scan. **lock_timeout = 2 s** fait échouer la migration plutôt que de bloquer la file.
Est-ce que ça fonctionne ?	**Oui, tous les contrôles attendus sont obtenus :** 1 000 NULL après l'ajout ; lots 200, 200, 200, 200, 200 puis 0 ; CHECK validé, colonne NOT NULL, défaut false ; écriture de NULL refusée (23502) ; **0 écart** sur id, email, nom (EXCEPT ALL dans les deux sens, empreinte md5 aec47d5bdba0 identique à la table clients) ; verrou : B échoue en 55P03 après 2,0 s, puis réussit après le COMMIT de A ; retour arrière : colonne absente, 0 écart.

Déroulement et preuves, étape par étape

Étape	Commande (extrait)	Ce que j'ai observé
01 Préparation	CREATE TABLE … (LIKE clients INCLUDING ALL) ; copie ; table de référence	1 000 clients dans la copie et dans la référence ; clé primaire et UNIQUE(email) conservées
02 Ajout	SET lock_timeout='2s' ; ADD COLUMN newsletter_ok boolean	1 000 NULL ; ALTER en 6,3 ms ; la table clients d'origine n'a pas la colonne
03 Compatibilité	ancien lecteur, nouveau lecteur COALESCE(…, false), insertions de test, ROLLBACK, SET DEFAULT false	ancienne insertion → NULL, nouvelle → false ; lignes de test annulées ; **le défaut ne remplit pas les 1 000 lignes existantes** (toujours NULL)
04 Lots	WITH lot AS (… LIMIT 200) UPDATE … RETURNING	200, 200, 200, 200, 200, puis 0 ; NULL restants 800, 600, 400, 200, 0 ; relancé après 0 : 0 ligne modifiée
05 Contrainte	ADD CONSTRAINT … CHECK (newsletter_ok IS NOT NULL) NOT VALID	convalidated = false ; une insertion avec NULL est **déjà refusée** (23514)
06 Validation	VALIDATE CONSTRAINT ; SET NOT NULL	convalidated = true ; is_nullable = NO ; message DEBUG1 : « existing constraints … are sufficient to prove that it does not contain nulls » (pas de second scan)
07 Contrôles	insertion sans la colonne ; UPDATE … = NULL ; comptages ; EXCEPT ALL	insertion → false ; UPDATE NULL → erreur 23502 ; 1 000 / 0 NULL / 1 000 false ; 0 écart
08 Verrous	A : BEGIN + SELECT ; B : lock_timeout 2 s + ALTER	B : 55P03 après 2,005 s ; COMMIT de A ; même ALTER relancé : réussi ; nettoyage
09 Retour arrière	DROP CONSTRAINT ; DROP NOT NULL / DEFAULT ; DROP COLUMN	colonne et valeurs encore là après le premier bloc ; colonne absente après le DROP ; 0 écart

Le remplissage lot par lot (mesures de l'exécution)

Lot	Lignes modifiées	Durée (ms)	WAL (octets)	NULL restants
1	200	4,2	66 560	800
2	200	5,3	65 352	600
3	200	41,5	60 872	400
4	200	38,5	57 296	200
5	200	39,2	44 816	0
6	0	0,553	0	0

Les durées sont bruitées (4,2 à 41,5 ms pour des lots identiques) : elles ne servent pas à comparer. Ce qui est fiable : le nombre de lignes, les NULL restants et le volume de WAL (≈ 0,31 Mo au total, soit ≈ 305 octets par ligne modifiée). Une valeur explicite déjà présente survit au remplissage (test ajouté, annulé par ROLLBACK : l'id 7 mis à true reste true, et le dernier lot ne traite que 199 lignes).

Verrous : A, B et C (chronologie réelle)

Temps	Session	Événement
0,003 s	A	BEGIN + SELECT : verrou ACCESS SHARE détenu, transaction laissée ouverte
0,004 s	B	ALTER TABLE … ADD COLUMN test_verrou : demande ACCESS EXCLUSIVE et attend
pendant l'attente	—	pg_locks : A détient AccessShareLock (accordé), B demande AccessExclusiveLock (non accordé)
2,009 s	B	**55P03** : canceling statement due to lock timeout (le serveur mesure 2001,8 ms)
2,178 s	A	COMMIT : verrou libéré
2,182 s	B	même ALTER relancé : **réussi** (2,3 ms) ; puis DROP COLUMN test_verrou et RESET lock_timeout

Ajout : la file de la slide 31. Avec B qui attend (lock_timeout 4 s) et une lecture simple C lancée 1 s plus tard, **C est bloquée 3,0 s** alors que A ne détient qu'un verrou de lecture compatible avec C : elle attend derrière B, et repart à la seconde où B abandonne (instantané : B et C bloquées, A non). C'est le risque réel d'un ALTER TABLE sur une table très lue, et la raison d'être de lock_timeout.

Ancienne version → nouvelle version à plus grand volume

Sur une table de travail de 3 000 000 lignes (hors laboratoire), même résultat final (empreinte md5 b227f2377b62 identique, colonne NOT NULL, défaut false) :

Ce qui bloque les autres	Ancienne (naïve)	Nouvelle (atelier 4 bis)
Temps sous verrou ACCESS EXCLUSIVE pour poser NOT NULL	**3324,1 ms** (SET NOT NULL lit toute la table)	**46,3 ms** (CHECK NOT VALID + SET NOT NULL sans scan)
Lecture la plus longue bloquée pendant cette étape	3 323 ms	38,9 ms (pendant VALIDATE, verrou léger)
Écriture la plus longue bloquée pendant le remplissage	30 989 ms (un seul UPDATE de 3 000 000 lignes)	416 ms (lots de 50 000)
Coût de la méthode : durée totale du remplissage	57 043 ms	156 265 ms (≈ ×2,7 plus long)

La nouvelle méthode ne va **pas plus vite** au total : elle remplit plus lentement mais **ne bloque presque personne**. C'est ce qu'on optimise ici : le blocage, pas la vitesse brute. Une seule exécution (les durées exactes varient) ; les écarts sont de plusieurs ordres de grandeur. Détails : annexe 8 et Optimisations.pdf.

Atelier 7 : la page et ses relations, puis PgBouncer

Pagination, N+1 et pool de connexions sur l'API ShopFlow

Consigne (slide 13 et fiche étudiant du kit, Jour 4). Comparer OFFSET et curseur, tester des commandes qui ont la même date, mesurer le nombre de requêtes d'une page qui charge ses lignes, corriger N+1 par chargement groupé (partie A) ; puis mesurer ce que mutualise PgBouncer (partie B). Livrable : contrat, SQL, comptage, trace et contrôle des résultats. **L'API est déjà codée : le travail consiste à l'expérimenter et à expliquer.** Les appels HTTP passent par l'aide du kit (jour4_http.mjs), les scripts SQL du kit sont rejoués tels quels (script atelier7/run_partie_a.py), PgBouncer par le pilote du kit (pooling.mjs). Documentation complète : dossier atelier7/ ; preuves : annexe 9. Le temps prévu (2 h 15) n'est pas respecté ; les preuves le sont.

Situation	L'écran « mes commandes » du client 42 affiche 20 commandes avec leurs lignes. Deux coûts distincts menacent l'API : le **nombre de requêtes** (N+1 : une requête de lignes par commande) et le **travail de pagination** (OFFSET écarte d'abord toutes les lignes d'avant). Et sous charge, chaque instance de l'API ouvre ses propres connexions à PostgreSQL.
Solution choisie et pourquoi	**Chargement groupé** (une requête de lignes pour toute la page : commande_id = ANY(…)) : le nombre de requêtes ne dépend plus de la taille de la page. **Curseur** sur la paire (created_at, id) au lieu d'OFFSET : la page suivante reprend au repère au lieu de réécarter les lignes d'avant ; l'id départage les dates identiques. **PgBouncer** en mode transaction : il borne les connexions serveur. **Pourquoi ces trois, et pas un cache ou un index de plus :** ils traitent trois problèmes différents (requêtes, lignes parcourues, connexions) qu'on corrige séparément (slide 3).
Est-ce que ça fonctionne ?	**Oui, avec des nuances à connaître.** Chargement groupé : **21 → 2 requêtes** pour 20 commandes, réponses identiques, durée HTTP médiane 55,3 → 43,9 ms. Curseur : page 2 identique à celle d'OFFSET ; à 1 000 000 de commandes, **944,3 ms → 0,177 ms** pour la même page. Dates identiques : 2042, 1042 et 42 sortent une seule fois. **PgBouncer ne rend rien plus rapide** : il limite les connexions (40 → 5) mais, avec 5 serveurs pour 40 clients, le débit passe de 261 à 94 transactions/s.

Partie A : déroulement et preuves

Étape	Ce qu'on fait	Ce que j'ai observé
1 Mise en route	API démarrée, client 42	100 commandes (5 pages de 20), 3 lignes chacune
2 N+1 / groupé	mêmes 20 commandes, deux modes	21 contre 2 SQL ; réponses **identiques** (lignes comprises) ; N+1 : 1 + N pour N = 5, 20, 50, 100 (6, 21, 51, 101) contre 2 ; page vide : 1 requête
3 OFFSET / curseur	page 2 par les deux méthodes	réponses **identiques**, 2 SQL chacune ; contrat : data / hasNextPage / nextCursor, limite 20 par défaut, 100 maximum (101 et 0 : erreur 400)
4 Dates identiques	07_dates_identiques.sql, pages de 2	page 1 : 2042, 1042 ; page 2 : 42, 86042 : les trois ids traversent la frontière sans perte
5 Parcours et droits	5 pages de 20, refus, restauration	5 pages, 100 ids, 100 uniques, même ordre que PostgreSQL ; 401 sans jeton, 400 avec client_id, 400 avec un curseur falsifié ; dates restaurées (3), empreinte md5 identique

N+1 contre chargement groupé : requêtes et durées

Commandes par page	SQL N+1	SQL groupé	HttpMs N+1 (médiane)	HttpMs groupé (médiane)	sqlMs N+1	sqlMs groupé
20	21	2	55,3	43,9	14,2	2,2
50	51	2	68,5	44,2	29,0	2,6

Pages de 5 et de 100 commandes (réponses identiques aussi) : 6 et 101 SQL contre 2 ; durées non relevées. 30 tours alternés après 5 échauffements, durées en ms. **HttpMs** (côté client) inclut le traitement HTTP du client Node.js et le transfert ; **sqlMs** (journal de l'API, rattaché par TraceId) n'inclut que le SQL et l'attente du pool : ce ne sont pas les mêmes durées. Le gain en durée est faible pour 20 commandes (11,4 ms) (base locale : chaque aller-retour coûte peu ; sur un réseau réel l'écart devrait grandir, ce que je n'ai pas mesuré). **Une baisse du nombre de requêtes n'est pas, à elle seule, un gain mesuré** : ici il l'est, et il est modeste.

OFFSET contre curseur à grande profondeur (table de travail, 1 000 000 de commandes)

Avec 100 commandes pour le client 42, la fiche interdit d'annoncer un gain sur de grands décalages (l'OFFSET dépasserait le nombre de résultats). J'ai donc comparé la **même page** sur une table jetable (un client, 1 000 000 de commandes, dates en double, index de l'atelier 3), avec les colonnes de l'API ; pages identiques à chaque profondeur (empreinte md5).

Décalage (page de 20)	OFFSET : ms	OFFSET : pages lues	Curseur : ms	Curseur : pages lues	Même page
0	0,171	24	0,154	24	oui
20	0,180	44	0,177	24	oui
1 000	1,2	1 029	0,174	25	oui
20 000	21,8	20 122	0,167	24	oui
100 000	99,7	100 516	0,168	24	oui
500 000	466,3	502 487	0,195	24	oui
900 000	831,6	904 457	0,164	24	oui
999 000	944,3	1 003 945	0,177	24	oui

Médianes de 7 mesures après 3 échauffements. L'OFFSET lit environ une ligne par ligne écartée ; le curseur lit toujours le même nombre de pages. **À petite profondeur il n'y a aucun gain** (pages 1 et 2 : 0,15 à 0,18 ms dans les deux cas) : le curseur n'apporte rien tant que le décalage est petit.

Dates identiques : pourquoi le curseur contient la date et l'id

Les commandes 2042, 1042 et 42 ont la même date (2026-09-13 23:54:02). Page de 2 : la première page se termine sur 1042. La page suivante par created_at < date seule renvoie **86042,83042** (la commande 42 est **perdue**) ; par la paire (created_at, id) < (date, 1042) elle renvoie **42,86042** (42 est bien là).

Partie B : PgBouncer (laboratoire séparé, 5 connexions serveur, mode transaction)

Campagne	Connexions PostgreSQL (pic échantillonné) direct / pool	TPS direct / pool (médiane de 3)	Latence moyenne direct / pool (ms)	Lecture
40 clients persistants	40 / 5	260,7 / 93,8	153 / 422	moins de connexions, **plus lent** : des clients attendent
10 clients, reconnexion (campagne 1)	10 / 5	22,6 / 29,7	301 / 258	instable : l'ordre s'inverse d'une campagne à l'autre
10 clients, reconnexion (campagne 2)	10 / 5	36,5 / 33,1	206 / 246	instable : l'ordre s'inverse d'une campagne à l'autre
80 clients, saturation	refus en direct / 5	non mesurable / 90,7	non mesurable / 860	direct : FATAL « remaining connection slots are reserved » ; pool : tous servis, avec attente

Avec 5 serveurs et 50 ms d'occupation par transaction, le pool ne peut pas dépasser 5 / 0,05 s = 100 transactions par seconde (mesuré : ≈ 94) ; 40 clients qui attendent chacun leur tour donnent une latence d'environ 40 / 94 ≈ 0,43 s (mesuré : 422 ms). **PgBouncer encadre les connexions : il ne corrige ni le N+1 ni les index, et il ne prouve aucun gain de temps de réponse.**

Atelier 8 : cache Redis et fraîcheur

Cache-aside d’une fiche catalogue : ce qu’il fait gagner, et ce qu’il peut faire mentir

Consigne (slides 23 à 35 et fiche étudiant, Jour 4). Vérifier le cache-aside de la fiche du produit 42 (clé shopflow:produit:v1:42), sa fraîcheur après modification et le repli quand Redis est indisponible. PostgreSQL reste la source de vérité ; l’API contient déjà le cache et l’invalidation, le travail consiste à **prouver leur comportement** (valeurs, comptage SQL, durées, traces). Script atelier8/run_atelier8.py (aide du kit jour4_http.mjs, journal de l’API rattaché par TraceId) ; preuves : annexe 10 ; documentation : dossier atelier8/. **Écart avec la fiche :** le TTL de 60 s est passé à l’API par variable d’environnement au lieu de modifier .env, qui reste intact (empreinte vérifiée avant et après). Le temps prévu (1 h 15) n’est pas respecté.

Situation	La fiche d’un produit est lue très souvent et change rarement. Chaque lecture déclenche un SELECT sur PostgreSQL. On veut éviter ce travail, **sans que le catalogue affiche durablement un prix faux** et sans que la validation d’un achat dépende d’une copie.
Solution choisie et pourquoi	**Cache-aside** avec Redis (slide 24) : l’API lit Redis ; sur miss elle lit PostgreSQL puis range une copie avec un **TTL de 60 s** ; l’écriture (PATCH) fait l’UPDATE puis supprime la clé. **Pourquoi :** la fiche catalogue est publique, simple à clé unique (shopflow:produit:v1:42, version de format incluse) et tolère un court retard ; l’historique privé d’un client changerait trop souvent et demanderait un périmètre par droits (slide 35). **Pourquoi pas read-through :** il demanderait une couche de chargement en plus pour un seul endpoint (slide 25). La confirmation de stock garde son contrôle dans la source.
Est-ce que ça fonctionne ?	**Oui, avec une limite qu’on a prouvée.** Miss : **1 SELECT** ; hit : **0 SQL**, contenu identique. Mesuré sur une connexion persistante (100 paires) : durée de la requête **2,7 → 0,988 ms** (×2,7). **Mais la copie peut être périmée :** après un UPDATE direct dans PostgreSQL, l’API a renvoyé l’ancien prix (57.50, hit, 0 SQL) alors que la base disait 19.90. L’expiration (TTL 60 → −2 après 61 s), le repli (Redis arrêté : réponse correcte, 1 SELECT) et la restauration du prix (57.50) sont vérifiés.

Déroulement et preuves, étape par étape

Étape	Ce qu’on fait	Ce que j’ai observé
1 Préparation	TTL 60 s, prix initial, prix cible SQL	prix initial 57.50, cible 19.90 ; TTL de la clé après chargement : 60 s (le TTL de 60 s est bien pris en compte)
2 Miss et hit	DEL, deux lectures ; 5 paires	miss : Cache=miss, **1 SELECT** ; hit : Cache=hit, **0 SQL** ; data identiques ; TTL après lecture : 60 s ; 5 paires vérifiées
3 UPDATE SQL direct	UPDATE du prix dans PostgreSQL, puis lecture, DEL, lecture	UPDATE → 42|19.90 ; l’API renvoie **57.50** (hit, 0 SQL) ; après DEL : **19.90** (miss, 1 SELECT)
4 PATCH par l’API	PATCH 29.90 puis 2 GET	corps : invalidation = **ok**, 1 UPDATE ; 1er GET : 29.90 et miss ; 2e GET : hit, 0 SQL
5 Expiration	attendre 61 s	TTL 60 → **-2** ; relecture : miss, 1 SELECT, prix 29.90
5 Panne de Redis	arrêter Redis, lire	réponse 200, Cache=**indisponible**, 1 SELECT, prix 29.90 : l’API reste disponible ; 30 appels en repli : durée de requête médiane 42,2 ms
6 Restauration	PATCH du prix initial	« Prix restauré » : 57.50, invalidation = ok ; table produits identique à l’état initial (md5)

Miss contre hit : ce que le cache fait vraiment gagner

Mesure	Miss (médiane)	Hit (médiane)	Rapport
Durée de la requête, connexion persistante, 100 paires (ms)	2,7	0,988	×2,7
Temps dans l'API (durationMs du journal, 100 paires)	2,1	0,671	×3,1
Temps SQL (sqlMs du journal)	0,768	0	—
Durée vue par l'aide du kit (un processus Node par appel, 30 paires)	43,0	42,0	×1,03 (non significatif)

Avec l'aide du kit, chaque appel lance un processus Node dont le coût fixe (≈ 40 ms) **noie** la différence : le gain n'y est pas visible, pourtant le temps dans l'API passe de 2,4 à 0,979 ms. J'ai donc ajouté une mesure sur connexion persistante. Le gain absolu reste **petit** (≈ 1,7 ms) : la base est locale et la requête est un simple accès par clé primaire ; il serait plus grand pour une requête coûteuse ou une base distante, ce que je n'ai pas mesuré. **Le gain principal est la charge évitée sur PostgreSQL** : un hit ne fait aucune lecture SQL.

La fraîcheur : ce que le cache peut faire mentir

Situation	Résultat mesuré	Ce que ça montre
UPDATE direct dans PostgreSQL, sans passer par l'API	l'API sert 57.50 (hit, 0 SQL) alors que la base a 19.90	le TTL n'actualise pas la copie après une écriture : elle reste périmée jusqu'au DEL ou à l'expiration
PATCH par l'API	invalidation = ok, GET suivant : nouveau prix (miss)	le chemin d'écriture prévu invalide la copie
PATCH pendant que Redis est arrêté	invalidation = **echouee**, PostgreSQL = 34.90 ; après le retour de Redis, l'API sert encore **29.90** (hit)	une invalidation échouée n'annule pas l'écriture : le prix est enregistré, la copie reste ancienne (jusqu'au TTL)
Course de la slide 30 (reproduite à la main)	A lit 34.90 ; B écrit 39.90 puis supprime la clé ; A remet son ancienne copie : l'API sert **34.90** (hit), PostgreSQL a **39.90**	supprimer la clé après l'écriture ne supprime pas toutes les courses

Fraîcheur acceptée pour le catalogue de ce laboratoire : de quelques secondes à une minute (le TTL est de 60 s). **La validation d'un achat n'utilise pas le cache** : le prix et le stock sont contrôlés dans PostgreSQL, dans la transaction de l'achat.

Analyse de la course (slide 30). A lit l'ancien prix dans PostgreSQL (miss) ; B valide le nouveau prix puis supprime la clé ; A, plus lent, range sa copie ancienne dans Redis. Redis a alors l'ancien prix alors que la base a le nouveau, jusqu'à l'expiration. Le DEL de B a eu lieu avant le SET de A : il ne pouvait pas l'annuler. **Pistes :** (1) une **version** (numéro ou date de modification) stockée avec la copie : A n'écrit dans Redis que si sa version est au moins aussi récente (SET conditionnel) ; (2) une **invalidation rejouée** après un court délai (ou via une outbox) pour supprimer une copie remise en retard ; (3) un TTL court si la fraîcheur exigée est forte. On choisit selon la fraîcheur exigée et la complexité acceptable.

Panne, rafale et repli

Repli. Redis arrêté : l'API répond correctement (200, prix de PostgreSQL, Cache=indisponible, 1 SELECT par lecture) : **chaque lecture devient un SELECT**, donc la charge sur la base augmente comme si le cache n'existait pas (30 appels : 30 SELECT contre 0). Sous forte charge, ce repli peut créer une seconde panne : limites de concurrence, priorités, circuit breaker (slide 32). **Rafale (slide 31).** 20 lectures simultanées, clé absente : **6 miss** et 6 SELECT au total (au lieu d'un seul) ; avec la clé présente : 20 hit, 0 SQL. Chaque requête qui trouve la clé absente recharge : un chargement unique par clé ou un TTL avec variation limiterait cette rafale.

Annexe 1 : plans EXPLAIN (ANALYZE, BUFFERS) complets (mesure n°5)

Q1 : historique du client 42

```
Limit  (cost=329.84..329.89 rows=20 width=22) (actual time=0.183..0.186 rows=20.00 loops=1)
  Buffers: shared hit=99
  ->  Sort  (cost=329.84..330.09 rows=100 width=22) (actual time=0.182..0.184 rows=20.00 loops=1)
        Sort Key: created_at DESC, id DESC
        Sort Method: top-N heapsort  Memory: 27kB
        Buffers: shared hit=99
        ->  Bitmap Heap Scan on commandes  (cost=5.07..327.18 rows=100 width=22) (actual
              time=0.039..0.147 rows=100.00 loops=1)
              Recheck Cond: (client_id = 42)
              Heap Blocks: exact=88
              Buffers: shared hit=93
              ->  Bitmap Index Scan on commandes_client_id_cle_idempotence_key  (cost=0.00..5.04
                    rows=100 width=0) (actual time=0.022..0.023 rows=100.00 loops=1)
                    Index Cond: (client_id = 42)
                    Index Searches: 1
                    Buffers: shared hit=5
Planning:
  Buffers: shared hit=125
Planning Time: 0.485 ms
Execution Time: 0.229 ms
```

Q2 : agrégation mensuelle

```
GroupAggregate  (cost=13279.82..15730.54 rows=80048 width=54) (actual time=54.107..77.900 rows=30.00
      loops=1)
  Group Key: (date_trunc('month'::text, created_at)), statut
  Buffers: shared hit=1680, temp read=380 written=381
  ->  Sort  (cost=13279.82..13529.82 rows=100000 width=20) (actual time=53.828..62.580 rows=100000.00
        loops=1)
        Sort Key: (date_trunc('month'::text, created_at)), statut
        Sort Method: external merge  Disk: 3040kB
        Buffers: shared hit=1680, temp read=380 written=381
        ->  Seq Scan on commandes  (cost=0.00..2924.00 rows=100000 width=20) (actual time=0.760..25.395
              rows=100000.00 loops=1)
              Buffers: shared hit=1674
Planning:
  Buffers: shared hit=152
Planning Time: 0.490 ms
Execution Time: 78.664 ms
```

Annexe 2 : plans de l’atelier 2 (mesure n°5)

A1 : jointure

```
Hash Join  (cost=32.50..3168.22 rows=80317 width=18) (actual time=0.986..23.875 rows=80000.00 loops=1)
  Hash Cond: (c.client_id = cl.id)
  Buffers: shared hit=1684
  ->  Seq Scan on commandes c  (cost=0.00..2924.00 rows=80317 width=8) (actual time=0.721..11.527
        rows=80000.00 loops=1)
        Filter: (statut = 'payee'::text)
        Rows Removed by Filter: 20000
        Buffers: shared hit=1674
  ->  Hash  (cost=20.00..20.00 rows=1000 width=18) (actual time=0.249..0.250 rows=1000.00 loops=1)
        Buckets: 1024  Batches: 1  Memory Usage: 58kB
        Buffers: shared hit=10
        ->  Seq Scan on clients cl  (cost=0.00..20.00 rows=1000 width=18) (actual time=0.009..0.095
              rows=1000.00 loops=1)
              Buffers: shared hit=10
Planning:
  Buffers: shared hit=248
Planning Time: 0.655 ms
Execution Time: 26.422 ms
```

A2 : jointure + DISTINCT

```
HashAggregate  (cost=3569.81..3579.81 rows=1000 width=18) (actual time=42.510..42.599 rows=800.00
      loops=1)
  Group Key: cl.id, cl.nom
  Batches: 1  Memory Usage: 89kB
  Buffers: shared hit=1684
  ->  Hash Join  (cost=32.50..3168.22 rows=80317 width=18) (actual time=1.102..27.041 rows=80000.00
        loops=1)
        Hash Cond: (c.client_id = cl.id)
        Buffers: shared hit=1684
        ->  Seq Scan on commandes c  (cost=0.00..2924.00 rows=80317 width=8) (actual time=0.695..12.879
              rows=80000.00 loops=1)
              Filter: (statut = 'payee'::text)
              Rows Removed by Filter: 20000
              Buffers: shared hit=1674
        ->  Hash  (cost=20.00..20.00 rows=1000 width=18) (actual time=0.387..0.389 rows=1000.00 loops=1)
              Buckets: 1024  Batches: 1  Memory Usage: 58kB
              Buffers: shared hit=10
              ->  Seq Scan on clients cl  (cost=0.00..20.00 rows=1000 width=18) (actual
                    time=0.008..0.220 rows=1000.00 loops=1)
                    Buffers: shared hit=10
Planning:
  Buffers: shared hit=173
Planning Time: 0.617 ms
Execution Time: 42.705 ms
```

A3 : EXISTS

```
Nested Loop Semi Join  (cost=0.29..568.98 rows=1000 width=18) (actual time=0.363..10.758 rows=800.00
      loops=1)
  Buffers: shared hit=21707
  ->  Seq Scan on clients cl  (cost=0.00..20.00 rows=1000 width=18) (actual time=0.008..0.081
        rows=1000.00 loops=1)
        Buffers: shared hit=10
  ->  Index Scan using commandes_client_id_cle_idempotence_key on commandes c  (cost=0.29..9.92 rows=80
        width=8) (actual time=0.010..0.010 rows=0.80 loops=1000)
        Index Cond: (client_id = cl.id)
        Filter: (statut = 'payee'::text)
        Rows Removed by Filter: 20
        Index Searches: 1000
        Buffers: shared hit=21697
Planning:
  Buffers: shared hit=248
Planning Time: 0.662 ms
Execution Time: 10.832 ms
```

B1 : sum(c.total), faux

```
HashAggregate  (cost=767.18..768.43 rows=100 width=40) (actual time=0.911..0.930 rows=100.00 loops=1)
  Group Key: c.id
  Batches: 1  Memory Usage: 56kB
  Buffers: shared hit=394
  ->  Nested Loop  (cost=5.49..765.68 rows=300 width=14) (actual time=0.055..0.803 rows=300.00 loops=1)
        Buffers: shared hit=394
        ->  Bitmap Heap Scan on commandes c  (cost=5.07..327.18 rows=100 width=14) (actual
              time=0.040..0.156 rows=100.00 loops=1)
              Recheck Cond: (client_id = 42)
              Heap Blocks: exact=88
              Buffers: shared hit=93
              ->  Bitmap Index Scan on commandes_client_id_cle_idempotence_key  (cost=0.00..5.04
                    rows=100 width=0) (actual time=0.027..0.027 rows=100.00 loops=1)
                    Index Cond: (client_id = 42)
                    Index Searches: 1
                    Buffers: shared hit=5
        ->  Index Only Scan using lignes_pkey on lignes l  (cost=0.42..4.35 rows=3 width=8) (actual
              time=0.006..0.006 rows=3.00 loops=100)
              Index Cond: (commande_id = c.id)
              Heap Fetches: 0
              Index Searches: 100
              Buffers: shared hit=301
Planning:
  Buffers: shared hit=179
Planning Time: 0.593 ms
Execution Time: 1.007 ms
```

B2 : sum(lignes), correct

```
GroupAggregate  (cost=1854.26..1859.26 rows=100 width=40) (actual time=1.877..1.995 rows=100.00 loops=1)
  Group Key: c.id
  Buffers: shared hit=696
  ->  Sort  (cost=1854.26..1855.01 rows=300 width=18) (actual time=1.866..1.880 rows=300.00 loops=1)
        Sort Key: c.id
        Sort Method: quicksort  Memory: 36kB
        Buffers: shared hit=696
        ->  Nested Loop  (cost=5.49..1841.92 rows=300 width=18) (actual time=0.060..1.757 rows=300.00
              loops=1)
              Buffers: shared hit=693
              ->  Bitmap Heap Scan on commandes c  (cost=5.07..327.18 rows=100 width=8) (actual
                    time=0.037..0.167 rows=100.00 loops=1)
                    Recheck Cond: (client_id = 42)
                    Heap Blocks: exact=88
                    Buffers: shared hit=93
                    ->  Bitmap Index Scan on commandes_client_id_cle_idempotence_key  (cost=0.00..5.04
                          rows=100 width=0) (actual time=0.023..0.024 rows=100.00 loops=1)
                          Index Cond: (client_id = 42)
                          Index Searches: 1
                          Buffers: shared hit=5
              ->  Index Scan using lignes_pkey on lignes l  (cost=0.42..15.12 rows=3 width=18) (actual
                    time=0.010..0.015 rows=3.00 loops=100)
                    Index Cond: (commande_id = c.id)
                    Index Searches: 100
                    Buffers: shared hit=600
Planning:
  Buffers: shared hit=182
Planning Time: 0.558 ms
Execution Time: 2.051 ms
```

Annexe 3 : plans des optimisations O1 et O2 (copie, mesure n°5)

Q2 référence (copie)

```
GroupAggregate  (cost=12445.82..14906.89 rows=80738 width=54) (actual time=54.555..78.490 rows=30.00
      loops=1)
  Group Key: (date_trunc('month'::text, created_at)), statut
  Buffers: shared hit=846, temp read=380 written=381
  ->  Sort  (cost=12445.82..12695.82 rows=100000 width=20) (actual time=54.310..63.319 rows=100000.00
        loops=1)
        Sort Key: (date_trunc('month'::text, created_at)), statut
        Sort Method: external merge  Disk: 3040kB
        Buffers: shared hit=846, temp read=380 written=381
        ->  Seq Scan on commandes  (cost=0.00..2090.00 rows=100000 width=20) (actual time=0.014..24.472
              rows=100000.00 loops=1)
              Buffers: shared hit=840
Planning:
  Buffers: shared hit=136
Planning Time: 0.350 ms
Execution Time: 79.261 ms
```

Q2 avec O1 : work_mem = 32MB

```
Sort  (cost=10881.60..11083.45 rows=80738 width=54) (actual time=50.533..50.535 rows=30.00 loops=1)
  Sort Key: (date_trunc('month'::text, created_at)), statut
  Sort Method: quicksort  Memory: 26kB
  Buffers: shared hit=846
  ->  HashAggregate  (cost=3090.00..4301.07 rows=80738 width=54) (actual time=50.148..50.475 rows=30.00
        loops=1)
        Group Key: date_trunc('month'::text, created_at), statut
        Batches: 1  Memory Usage: 2089kB
        Buffers: shared hit=840
        ->  Seq Scan on commandes  (cost=0.00..2090.00 rows=100000 width=20) (actual time=0.019..26.247
              rows=100000.00 loops=1)
              Buffers: shared hit=840
Planning:
  Buffers: shared hit=136
Planning Time: 0.329 ms
Execution Time: 51.517 ms
```

Q2 avec O2 : statistique sur date_trunc

```
Sort  (cost=3091.19..3091.26 rows=30 width=54) (actual time=45.145..45.147 rows=30.00 loops=1)
  Sort Key: (date_trunc('month'::text, created_at)), statut
  Sort Method: quicksort  Memory: 26kB
  Buffers: shared hit=846
  ->  HashAggregate  (cost=3090.00..3090.45 rows=30 width=54) (actual time=45.101..45.108 rows=30.00
        loops=1)
        Group Key: date_trunc('month'::text, created_at), statut
        Batches: 1  Memory Usage: 40kB
        Buffers: shared hit=840
        ->  Seq Scan on commandes  (cost=0.00..2090.00 rows=100000 width=20) (actual time=0.015..23.250
              rows=100000.00 loops=1)
              Buffers: shared hit=840
Planning:
  Buffers: shared hit=147
Planning Time: 0.407 ms
Execution Time: 45.225 ms
```

Annexe 4 : plans de l’atelier 3, requête API (première page), client 42 (vraie table, mesure n°5)

Ces plans sont ceux d’**une seule mesure** (la n°5 de chaque série). Leur durée est bruitée et ne sert pas à comparer (ici le plan « Simple » affiche 1,306 ms alors que sa médiane sur 51 tours est de 0,729 ms) : ce qu’ils prouvent, ce sont les **nœuds** et les **buffers**, qui sont stables. Les médianes comparables sont dans la section C.

Initiale

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS
      created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
      ORDER BY commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit  (cost=331.38..331.43 rows=21 width=118) (actual time=0.674..0.678 rows=21.00 loops=1)
  Buffers: shared hit=99
  ->  Sort  (cost=331.38..331.63 rows=100 width=118) (actual time=0.673..0.675 rows=21.00 loops=1)
        Sort Key: created_at DESC, id DESC
        Sort Method: top-N heapsort  Memory: 30kB
        Buffers: shared hit=99
        ->  Bitmap Heap Scan on commandes  (cost=5.07..328.68 rows=100 width=118) (actual
              time=0.116..0.619 rows=100.00 loops=1)
              Recheck Cond: (client_id = 42)
              Heap Blocks: exact=88
              Buffers: shared hit=93
              ->  Bitmap Index Scan on commandes_client_id_cle_idempotence_key  (cost=0.00..5.04
                    rows=100 width=0) (actual time=0.029..0.030 rows=100.00 loops=1)
                    Index Cond: (client_id = 42)
                    Index Searches: 1
                    Buffers: shared hit=5
Planning:
  Buffers: shared hit=137
Planning Time: 0.409 ms
Execution Time: 0.711 ms
```

Simple (client_id)

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS
      created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
      ORDER BY commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit  (cost=331.38..331.43 rows=21 width=118) (actual time=1.247..1.252 rows=21.00 loops=1)
  Buffers: shared hit=99
  ->  Sort  (cost=331.38..331.63 rows=100 width=118) (actual time=1.245..1.247 rows=21.00 loops=1)
        Sort Key: created_at DESC, id DESC
        Sort Method: top-N heapsort  Memory: 30kB
        Buffers: shared hit=99
        ->  Bitmap Heap Scan on commandes  (cost=5.07..328.68 rows=100 width=118) (actual
              time=0.280..1.153 rows=100.00 loops=1)
              Recheck Cond: (client_id = 42)
              Heap Blocks: exact=88
              Buffers: shared hit=93
              ->  Bitmap Index Scan on a3_idx_simple  (cost=0.00..5.04 rows=100 width=0) (actual
                    time=0.129..0.130 rows=100.00 loops=1)
                    Index Cond: (client_id = 42)
                    Index Searches: 1
                    Buffers: shared hit=5
Planning:
  Buffers: shared hit=150
Planning Time: 0.854 ms
Execution Time: 1.306 ms
```

Composé

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS
      created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
      ORDER BY commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit  (cost=0.42..84.25 rows=21 width=118) (actual time=0.154..0.291 rows=21.00 loops=1)
  Buffers: shared hit=27
  ->  Index Scan using a3_idx_compose on commandes  (cost=0.42..399.64 rows=100 width=118) (actual
        time=0.152..0.286 rows=21.00 loops=1)
        Index Cond: (client_id = 42)
        Index Searches: 1
        Buffers: shared hit=27
Planning:
  Buffers: shared hit=160
Planning Time: 0.812 ms
Execution Time: 0.321 ms
```

Couvrant (statut, total)

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS
      created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
      ORDER BY commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit  (cost=0.42..1.94 rows=21 width=118) (actual time=0.168..0.190 rows=21.00 loops=1)
  Buffers: shared hit=7
  ->  Index Only Scan using a3_idx_couvrant on commandes  (cost=0.42..7.67 rows=100 width=118) (actual
        time=0.166..0.187 rows=21.00 loops=1)
        Index Cond: (client_id = 42)
        Heap Fetches: 0
        Index Searches: 1
        Buffers: shared hit=7
Planning:
  Buffers: shared hit=167
Planning Time: 0.580 ms
Execution Time: 0.217 ms
```

Couvrant (total)

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS
      created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
      ORDER BY commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit  (cost=0.42..84.25 rows=21 width=118) (actual time=0.108..0.226 rows=21.00 loops=1)
  Buffers: shared hit=27
  ->  Index Scan using a3_idx_couvrant_total on commandes  (cost=0.42..399.64 rows=100 width=118)
        (actual time=0.107..0.223 rows=21.00 loops=1)
        Index Cond: (client_id = 42)
        Index Searches: 1
        Buffers: shared hit=27
Planning:
  Buffers: shared hit=163
Planning Time: 0.512 ms
Execution Time: 0.248 ms
```

Annexe 4 (suite) : plans de la requête du cours, client 42 (vraie table, mesure n°5)

Initiale

```
-- SELECT id, created_at, total FROM shopflow.commandes WHERE client_id = 42 ORDER BY created_at DESC,
      id DESC LIMIT 20
Limit  (cost=329.84..329.89 rows=20 width=22) (actual time=0.559..0.562 rows=20.00 loops=1)
  Buffers: shared hit=99
  ->  Sort  (cost=329.84..330.09 rows=100 width=22) (actual time=0.557..0.559 rows=20.00 loops=1)
        Sort Key: created_at DESC, id DESC
        Sort Method: top-N heapsort  Memory: 27kB
        Buffers: shared hit=99
        ->  Bitmap Heap Scan on commandes  (cost=5.07..327.18 rows=100 width=22) (actual
              time=0.055..0.513 rows=100.00 loops=1)
              Recheck Cond: (client_id = 42)
              Heap Blocks: exact=88
              Buffers: shared hit=93
              ->  Bitmap Index Scan on commandes_client_id_cle_idempotence_key  (cost=0.00..5.04
                    rows=100 width=0) (actual time=0.035..0.035 rows=100.00 loops=1)
                    Index Cond: (client_id = 42)
                    Index Searches: 1
                    Buffers: shared hit=5
Planning:
  Buffers: shared hit=125
Planning Time: 0.437 ms
Execution Time: 0.606 ms
```

Simple (client_id)

```
-- SELECT id, created_at, total FROM shopflow.commandes WHERE client_id = 42 ORDER BY created_at DESC,
      id DESC LIMIT 20
Limit  (cost=329.84..329.89 rows=20 width=22) (actual time=0.489..0.492 rows=20.00 loops=1)
  Buffers: shared hit=99
  ->  Sort  (cost=329.84..330.09 rows=100 width=22) (actual time=0.488..0.490 rows=20.00 loops=1)
        Sort Key: created_at DESC, id DESC
        Sort Method: top-N heapsort  Memory: 27kB
        Buffers: shared hit=99
        ->  Bitmap Heap Scan on commandes  (cost=5.07..327.18 rows=100 width=22) (actual
              time=0.061..0.443 rows=100.00 loops=1)
              Recheck Cond: (client_id = 42)
              Heap Blocks: exact=88
              Buffers: shared hit=93
              ->  Bitmap Index Scan on a3_idx_simple  (cost=0.00..5.04 rows=100 width=0) (actual
                    time=0.041..0.041 rows=100.00 loops=1)
                    Index Cond: (client_id = 42)
                    Index Searches: 1
                    Buffers: shared hit=5
Planning:
  Buffers: shared hit=138
Planning Time: 0.468 ms
Execution Time: 0.534 ms
```

Composé

```
-- SELECT id, created_at, total FROM shopflow.commandes WHERE client_id = 42 ORDER BY created_at DESC,
      id DESC LIMIT 20
Limit  (cost=0.42..79.96 rows=20 width=22) (actual time=0.058..0.174 rows=20.00 loops=1)
  Buffers: shared hit=26
  ->  Index Scan using a3_idx_compose on commandes  (cost=0.42..398.14 rows=100 width=22) (actual
        time=0.057..0.169 rows=20.00 loops=1)
        Index Cond: (client_id = 42)
        Index Searches: 1
        Buffers: shared hit=26
Planning:
  Buffers: shared hit=148
Planning Time: 0.583 ms
Execution Time: 0.222 ms
```

Couvrant (statut, total)

```
-- SELECT id, created_at, total FROM shopflow.commandes WHERE client_id = 42 ORDER BY created_at DESC,
      id DESC LIMIT 20
Limit  (cost=0.42..1.57 rows=20 width=22) (actual time=0.053..0.057 rows=20.00 loops=1)
  Buffers: shared hit=7
  ->  Index Only Scan using a3_idx_couvrant on commandes  (cost=0.42..6.17 rows=100 width=22) (actual
        time=0.052..0.055 rows=20.00 loops=1)
        Index Cond: (client_id = 42)
        Heap Fetches: 0
        Index Searches: 1
        Buffers: shared hit=7
Planning:
  Buffers: shared hit=155
Planning Time: 0.528 ms
Execution Time: 0.099 ms
```

Couvrant (total)

```
-- SELECT id, created_at, total FROM shopflow.commandes WHERE client_id = 42 ORDER BY created_at DESC,
      id DESC LIMIT 20
Limit  (cost=0.42..1.57 rows=20 width=22) (actual time=0.053..0.057 rows=20.00 loops=1)
  Buffers: shared hit=7
  ->  Index Only Scan using a3_idx_couvrant_total on commandes  (cost=0.42..6.17 rows=100 width=22)
        (actual time=0.052..0.054 rows=20.00 loops=1)
        Index Cond: (client_id = 42)
        Heap Fetches: 0
        Index Searches: 1
        Buffers: shared hit=7
Planning:
  Buffers: shared hit=151
Planning Time: 0.491 ms
Execution Time: 0.099 ms
```

Annexe 4 (suite) : plans de la requête API, page suivante par curseur, client 42 (vraie table, mesure n°5)

Initiale

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS
      created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
      AND (created_at, id) < ('2026-09-12 08:04:02+00'::timestamptz, 29042::bigint) ORDER BY
      commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit  (cost=331.25..331.30 rows=21 width=118) (actual time=0.703..0.707 rows=21.00 loops=1)
  Buffers: shared hit=99
  ->  Sort  (cost=331.25..331.46 rows=85 width=118) (actual time=0.701..0.703 rows=21.00 loops=1)
        Sort Key: created_at DESC, id DESC
        Sort Method: top-N heapsort  Memory: 29kB
        Buffers: shared hit=99
        ->  Bitmap Heap Scan on commandes  (cost=5.06..328.95 rows=85 width=118) (actual
              time=0.169..0.625 rows=80.00 loops=1)
              Recheck Cond: (client_id = 42)
              Filter: (ROW(created_at, id) < ROW('2026-09-12 08:04:02+00'::timestamp with time zone,
                    '29042'::bigint))
              Rows Removed by Filter: 20
              Heap Blocks: exact=88
              Buffers: shared hit=93
              ->  Bitmap Index Scan on commandes_client_id_cle_idempotence_key  (cost=0.00..5.04
                    rows=100 width=0) (actual time=0.063..0.064 rows=100.00 loops=1)
                    Index Cond: (client_id = 42)
                    Index Searches: 1
                    Buffers: shared hit=5
Planning:
  Buffers: shared hit=136
Planning Time: 0.446 ms
Execution Time: 0.755 ms
```

Simple (client_id)

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS
      created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
      AND (created_at, id) < ('2026-09-12 08:04:02+00'::timestamptz, 29042::bigint) ORDER BY
      commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit  (cost=331.25..331.30 rows=21 width=118) (actual time=0.891..0.896 rows=21.00 loops=1)
  Buffers: shared hit=99
  ->  Sort  (cost=331.25..331.46 rows=85 width=118) (actual time=0.889..0.892 rows=21.00 loops=1)
        Sort Key: created_at DESC, id DESC
        Sort Method: top-N heapsort  Memory: 29kB
        Buffers: shared hit=99
        ->  Bitmap Heap Scan on commandes  (cost=5.06..328.95 rows=85 width=118) (actual
              time=0.159..0.835 rows=80.00 loops=1)
              Recheck Cond: (client_id = 42)
              Filter: (ROW(created_at, id) < ROW('2026-09-12 08:04:02+00'::timestamp with time zone,
                    '29042'::bigint))
              Rows Removed by Filter: 20
              Heap Blocks: exact=88
              Buffers: shared hit=93
              ->  Bitmap Index Scan on a3_idx_simple  (cost=0.00..5.04 rows=100 width=0) (actual
                    time=0.042..0.042 rows=100.00 loops=1)
                    Index Cond: (client_id = 42)
                    Index Searches: 1
                    Buffers: shared hit=5
Planning:
  Buffers: shared hit=149
Planning Time: 0.494 ms
Execution Time: 0.955 ms
```

Composé

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS
      created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
      AND (created_at, id) < ('2026-09-12 08:04:02+00'::timestamptz, 29042::bigint) ORDER BY
      commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit  (cost=0.42..84.16 rows=21 width=118) (actual time=0.114..0.334 rows=21.00 loops=1)
  Buffers: shared hit=27
  ->  Index Scan using a3_idx_compose on commandes  (cost=0.42..339.37 rows=85 width=118) (actual
        time=0.113..0.330 rows=21.00 loops=1)
        Index Cond: ((client_id = 42) AND (ROW(created_at, id) < ROW('2026-09-12 08:04:02+00'::timestamp
              with time zone, '29042'::bigint)))
        Index Searches: 1
        Buffers: shared hit=27
Planning:
  Buffers: shared hit=159
Planning Time: 0.499 ms
Execution Time: 0.385 ms
```

Couvrant (statut, total)

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS
      created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
      AND (created_at, id) < ('2026-09-12 08:04:02+00'::timestamptz, 29042::bigint) ORDER BY
      commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit  (cost=0.42..2.14 rows=21 width=118) (actual time=0.142..0.161 rows=21.00 loops=1)
  Buffers: shared hit=7
  ->  Index Only Scan using a3_idx_couvrant on commandes  (cost=0.42..7.39 rows=85 width=118) (actual
        time=0.141..0.158 rows=21.00 loops=1)
        Index Cond: ((client_id = 42) AND (ROW(created_at, id) < ROW('2026-09-12 08:04:02+00'::timestamp
              with time zone, '29042'::bigint)))
        Heap Fetches: 0
        Index Searches: 1
        Buffers: shared hit=7
Planning:
  Buffers: shared hit=166
Planning Time: 0.488 ms
Execution Time: 0.203 ms
```

Couvrant (total)

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS
      created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
      AND (created_at, id) < ('2026-09-12 08:04:02+00'::timestamptz, 29042::bigint) ORDER BY
      commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit  (cost=0.42..84.16 rows=21 width=118) (actual time=0.139..0.293 rows=21.00 loops=1)
  Buffers: shared hit=27
  ->  Index Scan using a3_idx_couvrant_total on commandes  (cost=0.42..339.37 rows=85 width=118) (actual
        time=0.138..0.288 rows=21.00 loops=1)
        Index Cond: ((client_id = 42) AND (ROW(created_at, id) < ROW('2026-09-12 08:04:02+00'::timestamp
              with time zone, '29042'::bigint)))
        Index Searches: 1
        Buffers: shared hit=27
Planning:
  Buffers: shared hit=162
Planning Time: 0.541 ms
Execution Time: 0.374 ms
```

Annexe 5 : preuves des mesures

Ce que la slide 23 demande de conserver : version, résultats fonctionnels, temps des cinq répétitions, buffers. Tout ce qui suit provient des fichiers atelier3/resultats/resultats.json et des plans de atelier3/resultats/plans/.

5.0 Les trois requêtes exactes (client 42), telles qu'exécutées

cours

```
SELECT id, created_at, total FROM shopflow.commandes WHERE client_id = 42 ORDER BY created_at DESC, id
      DESC LIMIT 20
```

API, première page

```
SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS
      created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
      ORDER BY commandes.created_at DESC, commandes.id DESC LIMIT 21
```

API, page suivante par curseur

```
SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS
      created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
      AND (created_at, id) < ('2026-09-12 08:04:02+00'::timestamptz, 29042::bigint) ORDER BY
      commandes.created_at DESC, commandes.id DESC LIMIT 21
```

5.1 Environnement et état du laboratoire

Élément	Valeur
Version	PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu
Paramètres	work_mem = 4MB, shared_buffers = 128MB
Table commandes	100 000 lignes 1674 pages 13 713 408 octets
Index avant l'atelier 3	clients_email_key, clients_pkey, commandes_client_id_cle_idempotence_key, commandes_pkey, lignes_pkey, produits_pkey
Index après l'atelier 3	clients_email_key, clients_pkey, commandes_client_id_cle_idempotence_key, commandes_pkey, lignes_pkey, produits_pkey
Statistiques étendues	avant : 0, après : 0
Pages (total / visibles)	avant : 1674/1674, après : 1674/1674
Laboratoire restauré	**oui** (comparaison faite par le script)
Commande de mesure	python3 atelier3/benchmark.py (3 échauffements puis 5 mesures EXPLAIN (ANALYZE, BUFFERS) par combinaison)

5.2 Atelier 3 : les cinq mesures de chacune des 75 combinaisons (protocole du cours)

Durées en ms (Execution Time), dans l'ordre des 5 répétitions après 3 échauffements. « Empreinte » : md5 du résultat complet ; « = ref. » indique qu'elle est identique à celle de la base initiale pour la même requête et le même client. « Stable » : plan identique (mêmes nœuds et mêmes index) sur les 5 mesures.

Variante	Req.	Cl.	#1	#2	#3	#4	#5	Médiane	Buffers	Empreinte	= réf.	Stable
Initiale	cours	1	1,014	0,757	0,729	0,653	0,951	**0,757**	108	378faa2e6da2	oui	oui
	cours	42	0,470	0,525	0,538	0,575	0,606	**0,538**	99	5ae634c471f1	oui	oui
	cours	250	0,509	1,263	2,015	0,565	0,855	**0,855**	105	628efcdfee90	oui	oui
	cours	500	0,538	0,674	0,511	0,524	0,567	**0,538**	109	943014267f0c	oui	oui
	cours	1000	0,541	0,737	0,593	0,549	0,659	**0,593**	104	ce505a608836	oui	oui
	API	1	0,915	0,855	0,769	0,782	0,799	**0,799**	108	d60c48f2e88e	oui	oui
	API	42	1,035	0,882	0,786	0,627	0,711	**0,786**	99	21df3003a95c	oui	oui
	API	250	0,803	0,891	0,688	1,051	0,628	**0,803**	105	6d059dbcc1e7	oui	oui

Variante	Req.	Cl.	#1	#2	#3	#4	#5	Médiane	Buffers	Empreinte	= réf.	Stable
	API	500	1,222	0,884	0,860	0,723	0,922	**0,884**	109	4e97020cd521	oui	oui
	API	1000	0,710	1,076	0,771	0,742	0,805	**0,771**	104	300c87e77b17	oui	oui
	curseur	1	0,787	0,741	0,734	1,189	0,675	**0,741**	108	00f9a9fff24f	oui	oui
	curseur	42	0,729	0,784	0,649	0,747	0,755	**0,747**	99	e2a1d1ff6bb9	oui	oui
	curseur	250	1,067	1,331	1,335	1,210	0,668	**1,210**	105	b85c8192a892	oui	oui
	curseur	500	0,908	1,232	0,719	1,479	0,901	**0,908**	109	ae385fc44294	oui	oui
	curseur	1000	0,702	1,272	1,051	0,896	0,798	**0,896**	104	69b91bbcaa00	oui	oui
Simple (client_id)	cours	1	0,530	0,644	0,650	0,551	0,588	**0,588**	108	378faa2e6da2	oui	oui
	cours	42	0,565	0,486	0,792	0,528	0,534	**0,534**	99	5ae634c471f1	oui	oui
	cours	250	0,588	0,582	0,839	0,589	0,783	**0,589**	105	628efcdfee90	oui	oui
	cours	500	0,615	0,942	0,526	0,543	0,669	**0,615**	109	943014267f0c	oui	oui
	cours	1000	0,570	0,523	0,550	0,518	0,680	**0,550**	104	ce505a608836	oui	oui
	API	1	0,738	0,781	0,719	0,766	0,926	**0,766**	108	d60c48f2e88e	oui	oui
	API	42	0,662	0,733	0,935	0,847	1,306	**0,847**	99	21df3003a95c	oui	oui
	API	250	1,111	0,672	0,923	0,683	0,701	**0,701**	105	6d059dbcc1e7	oui	oui
	API	500	0,724	0,848	0,771	0,973	0,707	**0,771**	109	4e97020cd521	oui	oui
	API	1000	0,669	0,769	0,991	0,766	0,803	**0,769**	104	300c87e77b17	oui	oui
	curseur	1	0,729	0,924	0,743	0,741	0,795	**0,743**	108	00f9a9fff24f	oui	oui
	curseur	42	0,647	0,760	0,644	0,679	0,955	**0,679**	99	e2a1d1ff6bb9	oui	oui
	curseur	250	0,815	0,673	0,737	0,734	0,867	**0,737**	105	b85c8192a892	oui	oui
	curseur	500	0,713	0,764	0,726	0,715	0,700	**0,715**	109	ae385fc44294	oui	oui
	curseur	1000	0,719	0,803	0,690	0,728	1,122	**0,728**	104	69b91bbcaa00	oui	oui
Composé	cours	1	0,427	0,204	0,201	0,196	0,197	**0,201**	26	378faa2e6da2	oui	oui
	cours	42	0,510	0,184	0,199	0,220	0,222	**0,220**	26	5ae634c471f1	oui	oui
	cours	250	0,248	0,286	0,192	0,201	0,201	**0,201**	26	628efcdfee90	oui	oui
	cours	500	0,203	0,208	0,182	0,172	0,248	**0,203**	26	943014267f0c	oui	oui
	cours	1000	0,200	0,209	0,185	0,196	0,187	**0,196**	26	ce505a608836	oui	oui
	API	1	0,338	0,252	0,254	0,256	0,289	**0,256**	27	d60c48f2e88e	oui	oui
	API	42	0,407	0,464	0,248	0,251	0,321	**0,321**	27	21df3003a95c	oui	oui
	API	250	0,265	0,301	0,302	0,257	0,281	**0,281**	27	6d059dbcc1e7	oui	oui
	API	500	0,389	0,369	0,280	0,358	0,243	**0,358**	27	4e97020cd521	oui	oui
	API	1000	0,273	0,258	0,273	0,297	0,314	**0,273**	27	300c87e77b17	oui	oui
	curseur	1	0,328	0,334	0,280	0,281	0,302	**0,302**	27	00f9a9fff24f	oui	oui
	curseur	42	0,343	0,293	0,451	0,479	0,385	**0,385**	27	e2a1d1ff6bb9	oui	oui
	curseur	250	0,304	0,532	0,432	0,539	0,297	**0,432**	27	b85c8192a892	oui	oui
	curseur	500	0,369	0,270	0,274	0,275	0,303	**0,275**	28	ae385fc44294	oui	oui
	curseur	1000	0,522	0,318	0,288	0,300	0,333	**0,318**	27	69b91bbcaa00	oui	oui
Couvrant (statut, total)	cours	1	0,186	0,147	0,094	0,101	0,094	**0,101**	7	378faa2e6da2	oui	oui
	cours	42	0,091	0,095	0,097	0,095	0,099	**0,095**	7	5ae634c471f1	oui	oui
	cours	250	0,097	0,194	0,166	0,163	0,150	**0,163**	7	628efcdfee90	oui	oui
	cours	500	0,107	0,115	0,100	0,141	0,124	**0,115**	7	943014267f0c	oui	oui
	cours	1000	0,133	0,161	0,343	0,326	0,117	**0,161**	7	ce505a608836	oui	oui
	API	1	0,165	0,160	0,155	0,184	0,160	**0,160**	7	d60c48f2e88e	oui	oui
	API	42	0,200	0,161	0,158	0,150	0,217	**0,161**	7	21df3003a95c	oui	oui
	API	250	0,172	0,166	0,180	0,180	0,156	**0,172**	7	6d059dbcc1e7	oui	oui
	API	500	0,163	0,162	0,168	0,175	0,193	**0,168**	7	4e97020cd521	oui	oui
	API	1000	0,403	0,202	0,203	0,229	0,175	**0,203**	7	300c87e77b17	oui	oui
	curseur	1	0,207	0,165	0,267	0,201	0,294	**0,207**	7	00f9a9fff24f	oui	oui
	curseur	42	0,232	0,515	0,241	0,322	0,203	**0,241**	7	e2a1d1ff6bb9	oui	oui
	curseur	250	0,227	0,210	0,216	0,331	0,267	**0,227**	8	b85c8192a892	oui	oui
	curseur	500	0,242	0,207	0,218	0,204	0,190	**0,207**	7	ae385fc44294	oui	oui
	curseur	1000	0,198	0,251	0,225	0,225	0,191	**0,225**	7	69b91bbcaa00	oui	oui
Couvrant (total)	cours	1	0,119	0,092	0,102	0,099	0,099	**0,099**	7	378faa2e6da2	oui	oui
	cours	42	0,091	0,094	0,109	0,118	0,099	**0,099**	7	5ae634c471f1	oui	oui
	cours	250	0,119	0,103	0,283	0,142	0,104	**0,119**	7	628efcdfee90	oui	oui
	cours	500	0,098	0,112	0,129	0,098	0,107	**0,107**	7	943014267f0c	oui	oui
	cours	1000	0,106	0,100	0,160	0,106	0,128	**0,106**	7	ce505a608836	oui	oui
	API	1	0,256	0,318	0,297	0,282	0,360	**0,297**	27	d60c48f2e88e	oui	oui
	API	42	0,257	0,299	0,252	0,399	0,248	**0,257**	27	21df3003a95c	oui	oui

Variante	Req.	Cl.	#1	#2	#3	#4	#5	Médiane	Buffers	Empreinte	= réf.	Stable
	API	250	0,372	0,586	0,309	0,286	0,295	**0,309**	27	6d059dbcc1e7	oui	oui
	API	500	0,296	0,273	0,520	0,276	0,256	**0,276**	27	4e97020cd521	oui	oui
	API	1000	0,260	0,274	0,289	0,260	0,269	**0,269**	27	300c87e77b17	oui	oui
	curseur	1	0,305	0,376	0,460	0,324	0,295	**0,324**	27	00f9a9fff24f	oui	oui
	curseur	42	0,345	0,279	0,278	0,325	0,374	**0,325**	27	e2a1d1ff6bb9	oui	oui
	curseur	250	0,296	0,296	0,327	0,327	0,348	**0,327**	27	b85c8192a892	oui	oui
	curseur	500	0,270	0,300	0,293	0,282	0,266	**0,282**	27	ae385fc44294	oui	oui
	curseur	1000	0,530	0,292	0,609	0,301	0,308	**0,308**	28	69b91bbcaa00	oui	oui

Contrôle : 75 combinaisons sur 75 ont un résultat identique à la base initiale et un plan identique sur les 5 mesures.

5.3 Atelier 3 : mesure renforcée (51 tours en alternance), client 42 et client très actif

Requête	Variante	Client	Médiane	p95	Min	Tours	Buffers	Résultat = initiale
cours	Initiale	42	0,546	0,845	0,418	51	96	oui
cours	Simple (client_id)	42	0,548	0,860	0,419	51	96	oui
cours	Composé	42	0,161	0,339	0,129	51	26	oui
cours	Couvrant (statut, total)	42	0,094	0,186	0,076	51	7	oui
cours	Couvrant (total)	42	0,086	0,182	0,071	51	7	oui
API	Initiale	42	0,690	1,117	0,592	51	96	oui
API	Simple (client_id)	42	0,729	1,414	0,620	51	96	oui
API	Composé	42	0,274	0,485	0,218	51	27	oui
API	Couvrant (statut, total)	42	0,199	0,366	0,150	51	7	oui
API	Couvrant (total)	42	0,267	0,515	0,212	51	27	oui
curseur	Initiale	42	0,690	1,249	0,592	51	96	oui
curseur	Simple (client_id)	42	0,707	1,169	0,585	51	96	oui
curseur	Composé	42	0,342	0,562	0,240	51	27	oui
curseur	Couvrant (statut, total)	42	0,201	0,403	0,160	51	7	oui
curseur	Couvrant (total)	42	0,303	0,580	0,243	51	27	oui
cours	Initiale	7	6,619	11,3	5,838	51	287	oui
cours	Simple (client_id)	7	6,350	11,0	5,779	51	287	oui
cours	Composé	7	0,148	0,269	0,127	51	26	oui
cours	Couvrant (statut, total)	7	0,091	0,175	0,075	51	7	oui
cours	Couvrant (total)	7	0,103	0,168	0,081	51	7	oui
API	Initiale	7	23,3	42,9	21,0	51	287	oui
API	Simple (client_id)	7	23,3	40,4	21,2	51	287	oui
API	Composé	7	0,256	0,493	0,207	51	27	oui
API	Couvrant (statut, total)	7	0,189	0,347	0,152	51	7	oui
API	Couvrant (total)	7	0,276	0,439	0,225	51	27	oui

Durées en ms. p95 au rang le plus proche (49e valeur sur 51). Seuls la médiane, le p95 et le minimum de chaque série sont conservés, pas les 51 valeurs.

5.4 Atelier 3 : écriture, 20 000 insertions annulées (15 tours en alternance)

Variante	Index	WAL (octets)	Enreg. WAL	Médiane	Min	p95	Tours
Initiale	2	4 623 674	60 322	126,8	88,2	391,1	15
Simple (client_id)	3	5 927 578	80 327	180,9	132,5	353,8	15
Composé	3	6 240 760	80 322	194,0	139,5	593,3	15
Couvrant (statut, total)	3	6 570 048	80 322	219,0	152,1	443,7	15
Couvrant (total)	3	6 400 760	80 323	187,8	142,5	303,0	15

Durées en ms. Le volume de WAL est identique d'une exécution à l'autre ; les durées varient (voir le texte de la section G).

5.5 Optimisations O1 et O2 (copie) : les cinq mesures et l’équivalence des résultats

Variante	#1	#2	#3	#4	#5	Médiane	Empreinte du résultat
Référence (copie)	87,0	84,6	82,5	86,3	79,3	**84,6**	601f7d5aefe1 (= référence)
O1 : work_mem = 32MB	45,5	50,6	50,2	47,8	51,5	**50,2**	601f7d5aefe1 (= référence)
O2 : statistique sur date_trunc	56,6	44,9	48,6	45,5	45,2	**45,5**	601f7d5aefe1 (= référence)
O1 + O2	45,3	47,9	43,3	53,0	50,4	**47,9**	601f7d5aefe1 (= référence)

Durées en ms. Les quatre empreintes ont été recalculées directement (en ignorant la ligne « SET » qu’affiche psql) : toutes valent 601f7d5aefe1. Les mesures de O1 et O2 ont été faites sur la copie shopflow_opt, séquentiellement.

Annexe 6 : plans de l’atelier 4 (mesure n°5 de chaque série)

A. Initiale, file en_attente (vraie table)

```
-- SELECT id FROM shopflow.commandes WHERE statut = 'en_attente' ORDER BY created_at, id LIMIT 100
Limit  (cost=3296.75..3297.00 rows=100 width=16) (actual time=9.173..9.183 rows=100.00 loops=1)
  Buffers: shared hit=1680
  ->  Sort  (cost=3296.75..3321.14 rows=9753 width=16) (actual time=9.172..9.176 rows=100.00 loops=1)
        Sort Key: created_at, id
        Sort Method: top-N heapsort  Memory: 31kB
        Buffers: shared hit=1680
        ->  Seq Scan on commandes  (cost=0.00..2924.00 rows=9753 width=16) (actual time=0.630..8.066
              rows=10000.00 loops=1)
              Filter: (statut = 'en_attente'::text)
              Rows Removed by Filter: 90000
              Buffers: shared hit=1674
Planning:
  Buffers: shared hit=113
Planning Time: 0.359 ms
Execution Time: 9.207 ms
```

A. Partiel, file en_attente : Index Only Scan, 3 buffers

```
-- SELECT id FROM shopflow.commandes WHERE statut = 'en_attente' ORDER BY created_at, id LIMIT 100
Limit  (cost=0.29..3.43 rows=100 width=16) (actual time=0.032..0.047 rows=100.00 loops=1)
  Buffers: shared hit=3
  ->  Index Only Scan using a4_idx_partiel on commandes  (cost=0.29..306.58 rows=9753 width=16) (actual
        time=0.031..0.039 rows=100.00 loops=1)
        Heap Fetches: 0
        Index Searches: 1
        Buffers: shared hit=3
Planning:
  Buffers: shared hit=134
Planning Time: 0.512 ms
Execution Time: 0.070 ms
```

A. Complet, file en_attente

```
-- SELECT id FROM shopflow.commandes WHERE statut = 'en_attente' ORDER BY created_at, id LIMIT 100
Limit  (cost=0.42..4.22 rows=100 width=16) (actual time=0.042..0.057 rows=100.00 loops=1)
  Buffers: shared hit=4
  ->  Index Only Scan using a4_idx_complet on commandes  (cost=0.42..371.10 rows=9753 width=16) (actual
        time=0.041..0.049 rows=100.00 loops=1)
        Index Cond: (statut = 'en_attente'::text)
        Heap Fetches: 0
        Index Searches: 1
        Buffers: shared hit=4
Planning:
  Buffers: shared hit=145
Planning Time: 0.422 ms
Execution Time: 0.078 ms
```

A. Partiel, statut payee : le partiel ne sert pas (Seq Scan)

```
-- SELECT id FROM shopflow.commandes WHERE statut = 'payee' ORDER BY created_at, id LIMIT 100
Limit  (cost=5214.97..5226.37 rows=100 width=16) (actual time=13.230..15.329 rows=100.00 loops=1)
  Buffers: shared hit=1720
  ->  Gather Merge  (cost=5214.97..14368.64 rows=80316 width=16) (actual time=13.228..15.315 rows=100.00
        loops=1)
        Workers Planned: 1
        Workers Launched: 1
        Buffers: shared hit=1720
        ->  Sort  (cost=4214.96..4333.08 rows=47245 width=16) (actual time=11.286..11.292 rows=85.00
              loops=2)
              Sort Key: created_at, id
              Sort Method: top-N heapsort  Memory: 31kB
              Buffers: shared hit=1720
              Worker 0:  Sort Method: top-N heapsort  Memory: 31kB
              ->  Parallel Seq Scan on commandes  (cost=0.00..2409.29 rows=47245 width=16) (actual
                    time=0.431..6.786 rows=40000.00 loops=2)
                    Filter: (statut = 'payee'::text)
                    Rows Removed by Filter: 10000
                    Buffers: shared hit=1674
Planning:
  Buffers: shared hit=139
Planning Time: 0.512 ms
Execution Time: 15.375 ms
```

A. Complet, statut payee : sert tous les statuts

```
-- SELECT id FROM shopflow.commandes WHERE statut = 'payee' ORDER BY created_at, id LIMIT 100
Limit  (cost=0.42..4.20 rows=100 width=16) (actual time=0.039..0.053 rows=100.00 loops=1)
  Buffers: shared hit=4
  ->  Index Only Scan using a4_idx_complet on commandes  (cost=0.42..3041.97 rows=80317 width=16)
        (actual time=0.038..0.047 rows=100.00 loops=1)
        Index Cond: (statut = 'payee'::text)
        Heap Fetches: 0
        Index Searches: 1
        Buffers: shared hit=4
Planning:
  Buffers: shared hit=145
Planning Time: 0.434 ms
Execution Time: 0.076 ms
```

A. Requête paramétrée, plan générique forcé, index partiel : non utilisé

```
-- EXECUTE q_attente('en_attente') ; PREPARE q_attente(text) AS SELECT id FROM shopflow.commandes WHERE
      statut = $1 ORDER BY created_at, id LIMIT 100
Limit  (cost=4197.96..4198.21 rows=100 width=16) (actual time=10.712..10.723 rows=100.00 loops=1)
  Buffers: shared hit=1680
  ->  Sort  (cost=4197.96..4281.30 rows=33333 width=16) (actual time=10.711..10.715 rows=100.00 loops=1)
        Sort Key: created_at, id
        Sort Method: top-N heapsort  Memory: 31kB
        Buffers: shared hit=1680
        ->  Seq Scan on commandes  (cost=0.00..2924.00 rows=33333 width=16) (actual time=0.777..9.430
              rows=10000.00 loops=1)
              Filter: (statut = $1)
              Rows Removed by Filter: 90000
              Buffers: shared hit=1674
Planning:
  Buffers: shared hit=134 read=1
Planning Time: 0.651 ms
Execution Time: 10.750 ms
```

A. Requête paramétrée, plan spécifique forcé, index partiel : utilisé

```
-- EXECUTE q_attente('en_attente') ; PREPARE q_attente(text) AS SELECT id FROM shopflow.commandes WHERE
      statut = $1 ORDER BY created_at, id LIMIT 100
Limit  (cost=0.29..3.43 rows=100 width=16) (actual time=0.102..0.115 rows=100.00 loops=1)
  Buffers: shared hit=1 read=2
  ->  Index Only Scan using a4_idx_partiel on commandes  (cost=0.29..306.58 rows=9753 width=16) (actual
        time=0.101..0.108 rows=100.00 loops=1)
        Heap Fetches: 0
        Index Searches: 1
        Buffers: shared hit=1 read=2
Planning:
  Buffers: shared hit=134
Planning Time: 0.435 ms
Execution Time: 0.133 ms
```

B. Vraie table produits avec GIN : Seq Scan (200 lignes)

```
-- SELECT id, nom FROM shopflow.produits WHERE attributs @> '{"categorie":"livre"}'::jsonb ORDER BY id
Sort  (cost=6.91..7.04 rows=50 width=19) (actual time=0.089..0.091 rows=50.00 loops=1)
  Sort Key: id
  Sort Method: quicksort  Memory: 26kB
  Buffers: shared hit=6
  ->  Seq Scan on produits  (cost=0.00..5.50 rows=50 width=19) (actual time=0.017..0.059 rows=50.00
        loops=1)
        Filter: (attributs @> '{"categorie": "livre"}'::jsonb)
        Rows Removed by Filter: 150
        Buffers: shared hit=3
Planning:
  Buffers: shared hit=97
Planning Time: 0.494 ms
Execution Time: 0.120 ms
```

B. Vraie table, Seq Scan interdit (diagnostic) : le GIN peut servir @>

```
-- SET enable_seqscan = off ; SELECT id, nom FROM shopflow.produits WHERE attributs @>
      '{"categorie":"inexistant"}'::jsonb ORDER BY id
Sort  (cost=12.81..12.81 rows=1 width=19) (actual time=0.033..0.033 rows=0.00 loops=1)
  Sort Key: id
  Sort Method: quicksort  Memory: 25kB
  Buffers: shared hit=6
  ->  Bitmap Heap Scan on produits  (cost=8.79..12.80 rows=1 width=19) (actual time=0.014..0.014
        rows=0.00 loops=1)
        Recheck Cond: (attributs @> '{"categorie": "inexistant"}'::jsonb)
        Buffers: shared hit=3
        ->  Bitmap Index Scan on a4_idx_gin  (cost=0.00..8.79 rows=1 width=0) (actual time=0.008..0.008
              rows=0.00 loops=1)
              Index Cond: (attributs @> '{"categorie": "inexistant"}'::jsonb)
              Index Searches: 1
              Buffers: shared hit=3
Planning:
  Buffers: shared hit=97
Planning Time: 0.357 ms
Execution Time: 0.078 ms
```

B. Vraie table, Seq Scan interdit : le GIN ne sert pas ->>

```
-- SET enable_seqscan = off ; SELECT id, nom FROM shopflow.produits WHERE attributs->>'categorie' =
      'livre' ORDER BY id
Index Scan using produits_pkey on produits  (cost=0.14..18.14 rows=1 width=19) (actual time=0.021..0.068
      rows=50.00 loops=1)
  Filter: ((attributs ->> 'categorie'::text) = 'livre'::text)
  Rows Removed by Filter: 150
  Index Searches: 1
  Buffers: shared hit=4
Planning:
  Buffers: shared hit=95
Planning Time: 0.314 ms
Execution Time: 0.099 ms
```

B. 200 000 produits, valeur rare, sans index (Seq Scan)

```
-- SELECT count(*), sum(id) FROM a4_tmp.g_initiale WHERE attributs @> '{"categorie":"rare"}'::jsonb
Aggregate  (cost=5356.34..5356.35 rows=1 width=40) (actual time=45.080..45.082 rows=1.00 loops=1)
  Buffers: shared hit=2855
  ->  Seq Scan on g_initiale  (cost=0.00..5355.00 rows=267 width=8) (actual time=0.219..45.010
        rows=200.00 loops=1)
        Filter: (attributs @> '{"categorie": "rare"}'::jsonb)
        Rows Removed by Filter: 199800
        Buffers: shared hit=2855
Planning:
  Buffers: shared hit=66
Planning Time: 0.290 ms
Execution Time: 45.134 ms
```

B. 200 000 produits, valeur rare, GIN

```
-- SELECT count(*), sum(id) FROM a4_tmp.g_gin WHERE attributs @> '{"categorie":"rare"}'::jsonb
Aggregate  (cost=634.29..634.30 rows=1 width=40) (actual time=2.112..2.113 rows=1.00 loops=1)
  Buffers: shared hit=257
  ->  Bitmap Heap Scan on g_gin  (cost=116.89..633.49 rows=160 width=8) (actual time=1.300..2.092
        rows=200.00 loops=1)
        Recheck Cond: (attributs @> '{"categorie": "rare"}'::jsonb)
        Heap Blocks: exact=200
        Buffers: shared hit=257
        ->  Bitmap Index Scan on a4_idx_gin  (cost=0.00..116.85 rows=160 width=0) (actual
              time=1.270..1.271 rows=200.00 loops=1)
              Index Cond: (attributs @> '{"categorie": "rare"}'::jsonb)
              Index Searches: 1
              Buffers: shared hit=57
Planning:
  Buffers: shared hit=91
Planning Time: 0.364 ms
Execution Time: 2.194 ms
```

B. 200 000 produits, valeur à 24,9 %, GIN

```
-- SELECT count(*), sum(id) FROM a4_tmp.g_gin WHERE attributs @> '{"categorie":"livre"}'::jsonb
Aggregate  (cost=4110.21..4110.22 rows=1 width=40) (actual time=23.162..23.164 rows=1.00 loops=1)
  Buffers: shared hit=947
  ->  Bitmap Heap Scan on g_gin  (cost=378.93..3859.84 rows=50073 width=8) (actual time=5.756..20.245
        rows=49800.00 loops=1)
        Recheck Cond: (attributs @> '{"categorie": "livre"}'::jsonb)
        Heap Blocks: exact=888
        Buffers: shared hit=947
        ->  Bitmap Index Scan on a4_idx_gin  (cost=0.00..366.41 rows=50073 width=0) (actual
              time=5.657..5.658 rows=49800.00 loops=1)
              Index Cond: (attributs @> '{"categorie": "livre"}'::jsonb)
              Index Searches: 1
              Buffers: shared hit=59
Planning:
  Buffers: shared hit=91
Planning Time: 0.362 ms
Execution Time: 23.236 ms
```

B. 200 000 produits, ->> rare, index d'expression

```
-- SELECT count(*), sum(id) FROM a4_tmp.g_expression WHERE attributs->>'categorie' = 'rare'
Aggregate  (cost=685.22..685.23 rows=1 width=40) (actual time=0.865..0.866 rows=1.00 loops=1)
  Buffers: shared hit=203
  ->  Bitmap Heap Scan on g_expression  (cost=6.12..684.12 rows=220 width=8) (actual time=0.055..0.842
        rows=200.00 loops=1)
        Recheck Cond: ((attributs ->> 'categorie'::text) = 'rare'::text)
        Heap Blocks: exact=200
        Buffers: shared hit=203
        ->  Bitmap Index Scan on a4_idx_expr  (cost=0.00..6.07 rows=220 width=0) (actual
              time=0.026..0.026 rows=200.00 loops=1)
              Index Cond: ((attributs ->> 'categorie'::text) = 'rare'::text)
              Index Searches: 1
              Buffers: shared hit=3
Planning:
  Buffers: shared hit=96
Planning Time: 0.362 ms
Execution Time: 0.913 ms
```

B. 200 000 produits, colonne typée

```
-- SELECT count(*), sum(id) FROM a4_tmp.g_colonne_typee WHERE categorie = 'rare'
Aggregate  (cost=432.16..432.17 rows=1 width=40) (actual time=0.824..0.825 rows=1.00 loops=1)
  Buffers: shared hit=203
  ->  Bitmap Heap Scan on g_colonne_typee  (cost=5.40..431.52 rows=127 width=8) (actual
        time=0.054..0.802 rows=200.00 loops=1)
        Recheck Cond: (categorie = 'rare'::text)
        Heap Blocks: exact=200
        Buffers: shared hit=203
        ->  Bitmap Index Scan on a4_idx_typee  (cost=0.00..5.37 rows=127 width=0) (actual
              time=0.029..0.029 rows=200.00 loops=1)
              Index Cond: (categorie = 'rare'::text)
              Index Searches: 1
              Buffers: shared hit=3
Planning:
  Buffers: shared hit=96
Planning Time: 0.381 ms
Execution Time: 0.887 ms
```

B. Observation hors protocole : Parallel Seq Scan (22,274 ms) sur la table de la colonne typée

```
-- SELECT count(*), sum(id) FROM a4_tmp.g_colonne_typee WHERE attributs @> '{"categorie":"rare"}'::jsonb
Finalize Aggregate  (cost=5128.30..5128.31 rows=1 width=40) (actual time=18.570..22.203 rows=1.00
      loops=1)
  Buffers: shared hit=3086
  ->  Gather  (cost=5128.07..5128.28 rows=2 width=40) (actual time=18.424..22.188 rows=3.00 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=3086
        ->  Partial Aggregate  (cost=4128.07..4128.08 rows=1 width=40) (actual time=16.031..16.032
              rows=1.00 loops=3)
              Buffers: shared hit=3086
              ->  Parallel Seq Scan on g_colonne_typee  (cost=0.00..4127.67 rows=80 width=8) (actual
                    time=0.121..16.003 rows=66.67 loops=3)
                    Filter: (attributs @> '{"categorie": "rare"}'::jsonb)
                    Rows Removed by Filter: 66600
                    Buffers: shared hit=3086
Planning:
  Buffers: shared hit=98
Planning Time: 0.288 ms
Execution Time: 22.274 ms
```

C. Réservations (4 lignes) : Seq Scan

```
-- SELECT id FROM a4_tmp.reservations WHERE periode && tstzrange('2026-10-10 14:00+02', '2026-10-10
      16:00+02') ORDER BY id
Sort  (cost=1.07..1.08 rows=3 width=8) (actual time=0.044..0.045 rows=2.00 loops=1)
  Sort Key: id
  Sort Method: quicksort  Memory: 25kB
  Buffers: shared hit=4
  ->  Seq Scan on reservations  (cost=0.00..1.05 rows=3 width=8) (actual time=0.017..0.018 rows=2.00
        loops=1)
        Filter: (periode && '["2026-10-10 12:00:00+00","2026-10-10 14:00:00+00")'::tstzrange)
        Rows Removed by Filter: 2
        Buffers: shared hit=1
Planning:
  Buffers: shared hit=94
Planning Time: 0.612 ms
Execution Time: 0.142 ms
```

C. Réservations, Seq Scan interdit : le GiST sert &&

```
-- SET enable_seqscan = off ; SELECT id FROM a4_tmp.reservations WHERE periode && tstzrange('2026-10-10
      14:00+02', '2026-10-10 16:00+02') ORDER BY id
Sort  (cost=8.21..8.21 rows=3 width=8) (actual time=0.080..0.081 rows=2.00 loops=1)
  Sort Key: id
  Sort Method: quicksort  Memory: 25kB
  Buffers: shared hit=5
  ->  Index Scan using idx_periode on reservations  (cost=0.13..8.18 rows=3 width=8) (actual
        time=0.047..0.049 rows=2.00 loops=1)
        Index Cond: (periode && '["2026-10-10 12:00:00+00","2026-10-10 14:00:00+00")'::tstzrange)
        Index Searches: 1
        Buffers: shared hit=2
Planning:
  Buffers: shared hit=94
Planning Time: 0.834 ms
Execution Time: 0.111 ms
```

C. 200 000 périodes sans index

```
-- SELECT id FROM a4_tmp.r_sans WHERE periode && tstzrange('2026-03-01 14:00+00', '2026-03-01 16:00+00')
      ORDER BY id
Sort  (cost=3971.38..3971.42 rows=18 width=8) (actual time=26.709..26.711 rows=17.00 loops=1)
  Sort Key: id
  Sort Method: quicksort  Memory: 25kB
  Buffers: shared hit=1474
  ->  Seq Scan on r_sans  (cost=0.00..3971.00 rows=18 width=8) (actual time=1.249..26.694 rows=17.00
        loops=1)
        Filter: (periode && '["2026-03-01 14:00:00+00","2026-03-01 16:00:00+00")'::tstzrange)
        Rows Removed by Filter: 199983
        Buffers: shared hit=1471
Planning:
  Buffers: shared hit=74
Planning Time: 0.310 ms
Execution Time: 26.727 ms
```

C. 200 000 périodes avec GiST

```
-- SELECT id FROM a4_tmp.r_gist WHERE periode && tstzrange('2026-03-01 14:00+00', '2026-03-01 16:00+00')
      ORDER BY id
Sort  (cost=71.05..71.09 rows=18 width=8) (actual time=0.127..0.128 rows=17.00 loops=1)
  Sort Key: id
  Sort Method: quicksort  Memory: 25kB
  Buffers: shared hit=8
  ->  Bitmap Heap Scan on r_gist  (cost=4.42..70.67 rows=18 width=8) (actual time=0.100..0.102
        rows=17.00 loops=1)
        Recheck Cond: (periode && '["2026-03-01 14:00:00+00","2026-03-01 16:00:00+00")'::tstzrange)
        Heap Blocks: exact=1
        Buffers: shared hit=5
        ->  Bitmap Index Scan on a4_idx_gist  (cost=0.00..4.42 rows=18 width=0) (actual
              time=0.087..0.088 rows=17.00 loops=1)
              Index Cond: (periode && '["2026-03-01 14:00:00+00","2026-03-01 16:00:00+00")'::tstzrange)
              Index Searches: 1
              Buffers: shared hit=4
Planning:
  Buffers: shared hit=94
Planning Time: 0.412 ms
Execution Time: 0.163 ms
```

Annexe 7 : preuves des mesures de l’atelier 4

Tout provient de atelier4/resultats/resultats.json et de atelier4/resultats/plans/. Les valeurs brutes de chaque série de 51 tours et de 15 tours y sont conservées ; ce document n’en affiche que la médiane, le p95 et le minimum.

7.0 Requêtes exactes

A. file des commandes (statut variable : en_attente, payee, annulee)

```
SELECT id FROM shopflow.commandes WHERE statut = 'en_attente' ORDER BY created_at, id LIMIT 100
```

B. GIN sur la vraie table (valeurs : livre, materiel, inexistant ; variante ->>)

```
SELECT id, nom FROM shopflow.produits WHERE attributs @> '{"categorie":"livre"}'::jsonb ORDER BY id
```

B. 200 000 produits (la requête compte et additionne les id pour comparer les résultats)

```
SELECT count(*), sum(id) FROM a4_tmp.g_initiale WHERE attributs @> '{"categorie":"rare"}'::jsonb
```

C. 200 000 périodes

```
SELECT id FROM a4_tmp.r_gist WHERE periode && tstzrange('2026-03-01 14:00+00', '2026-03-01 16:00+00')
      ORDER BY id
```

7.1 Environnement et état du laboratoire

Élément	Valeur
Version	PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu
Paramètres	work_mem = 4MB, shared_buffers = 128MB
Index avant l’atelier 4	clients_email_key, clients_pkey, commandes_client_id_cle_idempotence_key, commandes_pkey, lignes_pkey, produits_pkey
Index après l’atelier 4	clients_email_key, clients_pkey, commandes_client_id_cle_idempotence_key, commandes_pkey, lignes_pkey, produits_pkey
Statistiques étendues	avant : 0, après : 0
Lignes avant / après	clients 1 000 / 1 000 ; produits 200 / 200 ; commandes 100 000 / 100 000 ; lignes 300 000 / 300 000
Laboratoire restauré	**oui** (comparaison faite par le script)
Commande de mesure	python3 atelier4/benchmark.py

7.2 Les cinq mesures brutes sur la vraie table (3 échauffements, durées en ms)

Famille	Variante	Requête	#1	#2	#3	#4	#5	Médiane	Buf.	Empreinte	= réf.	Stable
A	Initiale	en_attente	10,2	10,1	10,4	9,512	9,207	**10,1**	1680	7a826168f2fd	oui	oui
A	Initiale	payee	15,4	17,4	15,8	34,4	24,0	**17,4**	1720	1b318a28ebb9	oui	oui
A	Initiale	annulee	9,796	10,8	11,1	11,1	10,9	**10,9**	1680	9b0c6b53360d	oui	oui
A	Partiel	en_attente	0,073	0,070	0,183	0,064	0,070	**0,070**	3	7a826168f2fd	oui	oui
A	Partiel	payee	14,7	16,1	13,3	14,6	15,4	**14,7**	1720	1b318a28ebb9	oui	oui
A	Partiel	annulee	10,2	10,0	9,942	10,0	8,848	**10,0**	1680	9b0c6b53360d	oui	oui
A	Complet	en_attente	0,085	0,082	0,077	0,076	0,078	**0,078**	4	7a826168f2fd	oui	oui
A	Complet	payee	0,095	0,067	0,186	0,096	0,076	**0,095**	4	1b318a28ebb9	oui	oui
A	Complet	annulee	0,115	0,106	0,089	0,106	0,085	**0,106**	4	9b0c6b53360d	oui	oui
B	Initiale	@> livre	0,116	0,116	0,129	0,117	0,115	**0,116**	6	d6b7d96d9891	oui	oui
B	Initiale	@> materiel	0,133	0,131	0,132	0,141	0,128	**0,132**	6	075316ce230b	oui	oui
B	Initiale	@> absent	0,126	0,098	0,090	0,152	0,107	**0,107**	6	d41d8cd98f00	oui	oui
B	Initiale	->> livre	0,124	0,164	0,121	0,125	0,115	**0,124**	6	d6b7d96d9891	oui	oui
B	GIN	@> livre	0,123	0,110	0,148	0,117	0,120	**0,120**	6	d6b7d96d9891	oui	oui
B	GIN	@> materiel	0,127	0,142	0,137	0,136	0,145	**0,137**	6	075316ce230b	oui	oui
B	GIN	@> absent	0,117	0,112	0,192	0,118	0,129	**0,118**	6	d41d8cd98f00	oui	oui
B	GIN	->> livre	0,108	0,118	0,112	0,108	0,124	**0,112**	6	d6b7d96d9891	oui	oui
B	Expression	@> livre	0,123	0,146	0,159	0,120	0,120	**0,123**	6	d6b7d96d9891	oui	oui
B	Expression	@> materiel	0,148	0,133	0,230	0,136	0,133	**0,136**	6	075316ce230b	oui	oui
B	Expression	@> absent	0,100	0,105	0,112	0,120	0,106	**0,106**	6	d41d8cd98f00	oui	oui
B	Expression	->> livre	0,122	0,120	0,121	0,120	0,108	**0,120**	6	d6b7d96d9891	oui	oui

Contrôle : 21 combinaisons sur 21 ont un résultat identique à la base initiale et un plan identique sur les 5 mesures. Usage des index (idx_scan) : partiel 9, complet 27, GIN 0, expression 0.

7.3 Mesures renforcées (tables de travail, sans parallélisme, durées en ms)

Famille	Variante	Requête	Médiane	p95	Min	Tours	Buffers	Résultat = réf.
A	initiale	en_attente	9,899	11,9	8,344	51	846	oui
A	initiale	payee	22,2	26,2	19,7	51	846	oui
A	partiel	en_attente	0,071	0,141	0,059	51	3	oui
A	partiel	payee	21,9	25,1	19,5	51	846	oui
A	complet	en_attente	0,084	0,201	0,075	51	4	oui
A	complet	payee	0,082	0,140	0,067	51	4	oui
B	initiale	contient_livre	48,7	104,5	43,9	51	2855	oui
B	initiale	contient_rare	43,9	105,8	39,1	51	2855	oui
B	initiale	extrait_rare	30,9	74,0	27,3	51	2855	oui
B	gin	contient_livre	24,3	50,7	21,6	51	947	oui
B	gin	contient_rare	2,551	5,716	2,124	51	257	oui
B	gin	extrait_rare	32,8	80,3	29,3	51	2855	oui
B	expression	contient_livre	53,2	106,7	43,7	51	2855	oui
B	expression	contient_rare	46,3	73,0	39,9	51	2855	oui
B	expression	extrait_rare	1,104	3,590	0,877	51	203	oui
B	colonne_typee	contient_livre	49,1	154,8	44,5	51	3086	oui
B	colonne_typee	contient_rare	43,4	130,0	38,8	51	3086	oui
B	colonne_typee	extrait_rare	31,9	83,7	29,7	51	3086	oui
B	colonne_typee	colonne_rare	1,016	2,021	0,777	51	203	oui
C	sans_index	chevauchement	27,5	33,6	24,8	51	1474	oui
C	gist	chevauchement	0,168	0,294	0,154	51	8	oui

7.4 Écriture : lots annulés, 15 tours en alternance

Famille	Variante	Lot	WAL (octets)	Enreg. WAL	Médiane (ms)	Min	p95	Tours
A	initiale	insertion_20000	4 621 038	60 275	103,6	80,3	117,4	15
A	initiale	statut_en_attente_vers_payee	605 083	8 037	18,5	14,7	26,3	15
A	initiale	statut_payee_vers_en_attente	615 355	8 015	15,6	12,4	35,5	15
A	partiel	insertion_20000	4 768 498	62 291	112,0	91,1	126,7	15
A	partiel	statut_en_attente_vers_payee	605 083	8 037	16,5	12,8	23,3	15
A	partiel	statut_payee_vers_en_attente	769 824	10 102	22,3	16,0	42,7	15
A	complet	insertion_20000	6 305 626	80 607	183,2	131,3	207,2	15
A	complet	statut_en_attente_vers_payee	771 943	10 066	23,0	17,4	35,2	15
A	complet	statut_payee_vers_en_attente	797 436	10 042	21,9	18,3	42,2	15
B	initiale	insertion_20000	4 097 786	40 296	86,3	35,6	147,1	15
B	initiale	maj_attributs_20000	5 547 001	60 113	106,7	78,6	156,7	15
B	gin	insertion_20000	9 044 567	60 617	139,6	79,2	280,8	15
B	gin	maj_attributs_20000	10 120 555	80 364	139,8	123,6	322,6	15
B	expression	insertion_20000	5 515 841	60 514	120,5	71,5	244,6	15
B	expression	maj_attributs_20000	6 964 137	80 338	150,0	137,1	376,7	15
B	colonne_typee	insertion_20000	5 690 863	60 522	125,0	79,2	206,2	15
B	colonne_typee	maj_attributs_20000	7 153 583	80 367	160,5	135,7	252,9	15
C	sans_index	insertion_20000	3 029 330	40 170	55,6	32,7	87,3	15
C	gist	insertion_20000	4 669 330	60 170	434,9	343,6	539,4	15

Le volume de WAL est identique d’une exécution à l’autre ; les durées varient et ne servent pas à chiffrer précisément un surcoût.

Annexe 8 : preuves de l’atelier 4 bis

Tout provient de atelier4bis/resultats/ : resultats.json et transcript.txt (sortie brute de psql, avec les durées et les codes d’erreur) pour l’exécution des fichiers 01 à 09 ; volume.json pour la mesure à 3 millions de lignes. Les six traces demandées par la slide 43 y sont conservées.

8.1 Environnement et état du laboratoire

Élément	Valeur
Version	PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu
Exécution	2026-10-07 09:44:16 ; commande : python3 atelier4bis/run_atelier.py ; fichiers du cours : Atelier_4bis/01 à 09 (relus tels quels)
Volume	1 000 clients (identifiants 1 à 1000 ; 1001 et 1002 libres, utilisés pour les tests annulés)
Connexions	M (migration, autocommit), A, B et C : quatre processus psql distincts (application_name visible dans pg_stat_activity)
Index avant / après	clients_email_key, clients_pkey, commandes_client_id_cle_idempotence_key, commandes_pkey, lignes_pkey, produits_pkey (identiques après)
Tables avant / après	clients, commandes, lignes, produits (identiques après : les deux tables de travail sont supprimées)
Empreinte de clients	md5 aec47d5bdba0 avant et après ; copie aec47d5bdba0, référence aec47d5bdba0, copie après retour arrière aec47d5bdba0
Laboratoire restauré	**oui** (comparaison faite par le script ; les tables de migration sont supprimées à la fin, le script les recrée)

8.2 Trace 01 : SQL exécuté

Les fichiers Atelier_4bis/01_preparation.sql à 09_retour_arriere.sql, commande par commande, dans cet ordre. Les commandes sont envoyées telles quelles à psql : seule la mise en forme (découpage au point-virgule) est faite par le script.

Fichiers 01, 02, 03 (sans les commentaires)

```
-- 01_preparation.sql
CREATE TABLE shopflow.clients_migration_tp
  (LIKE shopflow.clients INCLUDING ALL);
INSERT INTO shopflow.clients_migration_tp (id,email,nom)
SELECT id,email,nom FROM shopflow.clients;
CREATE TABLE shopflow.clients_migration_reference_tp AS
SELECT id,email,nom FROM shopflow.clients_migration_tp;
SELECT current_database() AS base, version() AS version;
SELECT count(*) AS nb_clients FROM shopflow.clients_migration_tp;

-- 02_ajout_colonne.sql
SET lock_timeout = '2s';
ALTER TABLE shopflow.clients_migration_tp
  ADD COLUMN newsletter_ok boolean;
SELECT count(*) AS nb_null
FROM shopflow.clients_migration_tp WHERE newsletter_ok IS NULL;

-- 03_compatibilite.sql
SELECT id,email,nom FROM shopflow.clients_migration_tp ORDER BY id LIMIT 5;
SELECT id,email,COALESCE(newsletter_ok,false) AS newsletter_autorisee
FROM shopflow.clients_migration_tp ORDER BY id LIMIT 5;
BEGIN;
INSERT INTO shopflow.clients_migration_tp (id,email,nom)
VALUES (1001,'ancien@example.test','Client ancien');
INSERT INTO shopflow.clients_migration_tp (id,email,nom,newsletter_ok)
VALUES (1002,'nouveau@example.test','Client nouveau',false);
SELECT id,newsletter_ok FROM shopflow.clients_migration_tp WHERE id IN (1001,1002);
ROLLBACK;
ALTER TABLE shopflow.clients_migration_tp
  ALTER COLUMN newsletter_ok SET DEFAULT false;
SELECT count(*) AS nb_null
FROM shopflow.clients_migration_tp WHERE newsletter_ok IS NULL;
```

Fichiers 04, 05, 06 (sans les commentaires)

```
-- 04_remplissage_lot.sql
WITH lot AS (
  SELECT id FROM shopflow.clients_migration_tp
  WHERE newsletter_ok IS NULL ORDER BY id LIMIT 200
), maj AS (
  UPDATE shopflow.clients_migration_tp c
  SET newsletter_ok = false FROM lot
  WHERE c.id = lot.id AND c.newsletter_ok IS NULL
  RETURNING c.id
)
SELECT count(*) AS nb_mises_a_jour FROM maj;

SELECT count(*) AS nb_null_restants
FROM shopflow.clients_migration_tp WHERE newsletter_ok IS NULL;

-- 05_contrainte_progressive.sql
SELECT count(*) AS nb_null
FROM shopflow.clients_migration_tp WHERE newsletter_ok IS NULL;
ALTER TABLE shopflow.clients_migration_tp
  ADD CONSTRAINT clients_newsletter_nn_tp
  CHECK (newsletter_ok IS NOT NULL) NOT VALID;
SELECT conname,convalidated FROM pg_constraint
WHERE conrelid = 'shopflow.clients_migration_tp'::regclass
  AND conname = 'clients_newsletter_nn_tp';

-- 06_validation.sql
ALTER TABLE shopflow.clients_migration_tp
  VALIDATE CONSTRAINT clients_newsletter_nn_tp;
ALTER TABLE shopflow.clients_migration_tp
  ALTER COLUMN newsletter_ok SET NOT NULL;
SELECT conname,convalidated FROM pg_constraint
WHERE conrelid = 'shopflow.clients_migration_tp'::regclass
  AND conname = 'clients_newsletter_nn_tp';
SELECT column_name,is_nullable,column_default FROM information_schema.columns
WHERE table_schema='shopflow' AND table_name='clients_migration_tp'
  AND column_name='newsletter_ok';
```

Fichiers 07 (sans les commentaires)

```
-- 07_controles.sql
BEGIN;
INSERT INTO shopflow.clients_migration_tp (id,email,nom)
VALUES (1001,'defaut@example.test','Client défaut')
RETURNING id,newsletter_ok;
ROLLBACK;

UPDATE shopflow.clients_migration_tp SET newsletter_ok=NULL WHERE id=1;

SELECT count(*) AS nb_clients,
       count(*) FILTER (WHERE newsletter_ok IS NULL) AS nb_null,
       count(*) FILTER (WHERE newsletter_ok=false) AS nb_false
FROM shopflow.clients_migration_tp;
WITH differences AS (
 (SELECT id,email,nom FROM shopflow.clients_migration_tp
  EXCEPT ALL SELECT id,email,nom FROM shopflow.clients_migration_reference_tp)
 UNION ALL
 (SELECT id,email,nom FROM shopflow.clients_migration_reference_tp
  EXCEPT ALL SELECT id,email,nom FROM shopflow.clients_migration_tp)
)
SELECT count(*) AS ecarts_historiques FROM differences;
```

Fichiers 08A, 08B (sans les commentaires)

```
-- 08_verrous_session_A.sql
BEGIN;
SELECT id FROM shopflow.clients_migration_tp LIMIT 1;
COMMIT;

-- 08_verrous_session_B.sql
SET lock_timeout='2s';
ALTER TABLE shopflow.clients_migration_tp ADD COLUMN test_verrou integer;

ALTER TABLE shopflow.clients_migration_tp DROP COLUMN test_verrou;
RESET lock_timeout;
```

Fichiers 09 (sans les commentaires)

```
-- 09_retour_arriere.sql
SET lock_timeout='2s';
BEGIN;
ALTER TABLE shopflow.clients_migration_tp
  DROP CONSTRAINT clients_newsletter_nn_tp;
ALTER TABLE shopflow.clients_migration_tp
  ALTER COLUMN newsletter_ok DROP NOT NULL,
  ALTER COLUMN newsletter_ok DROP DEFAULT;
COMMIT;
ALTER TABLE shopflow.clients_migration_tp DROP COLUMN newsletter_ok;
SELECT count(*) AS colonne_restante FROM information_schema.columns
WHERE table_schema='shopflow' AND table_name='clients_migration_tp'
  AND column_name='newsletter_ok';
WITH differences AS (
 (SELECT id,email,nom FROM shopflow.clients_migration_tp
  EXCEPT ALL SELECT id,email,nom FROM shopflow.clients_migration_reference_tp)
 UNION ALL
 (SELECT id,email,nom FROM shopflow.clients_migration_reference_tp
  EXCEPT ALL SELECT id,email,nom FROM shopflow.clients_migration_tp)
)
SELECT count(*) AS ecarts_historiques FROM differences;
RESET lock_timeout;
```

8.3 Trace 02 : volumes et NULL avant, pendant et après

Moment	Lignes	NULL	false	Source
Après 01 (copie)	1 000	colonne absente	—	count(*)
Après 02 (colonne ajoutée)	1 000	1 000	0	WHERE newsletter_ok IS NULL
Après 03 (défaut false posé)	1 000	1 000	0	le défaut ne remplit pas l'existant
Pendant 04, après le lot 1	1 000	800	200	200 ligne(s) modifiée(s)
Pendant 04, après le lot 2	1 000	600	400	200 ligne(s) modifiée(s)
Pendant 04, après le lot 3	1 000	400	600	200 ligne(s) modifiée(s)
Pendant 04, après le lot 4	1 000	200	800	200 ligne(s) modifiée(s)
Pendant 04, après le lot 5	1 000	0	1 000	200 ligne(s) modifiée(s)
Pendant 04, après le lot 6	1 000	0	1 000	0 ligne(s) modifiée(s)
Après 06 et 07	1 000	0	1 000	comptage final

Taille de la table : 80 kB après la copie ; 152 kB avant le remplissage ; 216 kB après (chaque UPDATE crée une nouvelle version de ligne : l'ancienne devient une ligne morte qu'un VACUUM récupérera).

8.4 Trace 03 : compatibilité des lecteurs, défaut, rejet de NULL, écart historique

Insertions de test (03, annulées) : 1001 insérée sans la colonne → NULL, 1002 insérée avec false → false (avant le défaut). Après le défaut false (07, annulé) : l'insertion sans la colonne donne false. Écriture explicite de NULL après NOT NULL :

```
Time: 0.342 ms
ERROR:  23502: null value in column "newsletter_ok" of relation "clients_migration_tp" violates not-null
      constraint
DETAIL:  Failing row contains (1, etudiant1@example.test, Client 1, null).
SCHEMA NAME:  shopflow
TABLE NAME:  clients_migration_tp
COLUMN NAME:  newsletter_ok
LOCATION:  ReportNotNullViolationError, execMain.c:2213
```

Comparaison historique (EXCEPT ALL dans les deux sens, avec la table de référence) : **0 écart**. Les empreintes md5 de id|email|nom triés sont identiques : copie aec47d5bdba0, référence aec47d5bdba0, table clients d'origine aec47d5bdba0. Un comptage identique ne prouve pas l'égalité des valeurs, l'empreinte et EXCEPT ALL si.

8.5 Trace 04 : état de la contrainte avant et après validation

Moment	convalidated	Preuve
Après ADD CONSTRAINT ... NOT VALID (05)	false	insertion avec NULL refusée : erreur 23514 (violation du CHECK clients_newsletter_nn_tp)
Après VALIDATE CONSTRAINT (06)	true	VALIDATE en 38,2 ms sur 1 000 lignes
Après SET NOT NULL (06)	true (CHECK conservé)	2,3 ms ; colonne is_nullable = NO

Message affiché par PostgreSQL (client_min_messages = debug1) pendant SET NOT NULL : le CHECK valide suffit à prouver l'absence de NULL, la table n'est pas relue.

```
DEBUG:  00000: existing constraints on column "clients_migration_tp.newsletter_ok" are sufficient to
      prove that it does not contain nulls
LOCATION:  NotNullImpliedByRelConstraints, tablecmds.c:8116
Time: 2.258 ms
```

8.6 Trace 05 : verrou, erreur de B, COMMIT de A et réussite de B (chronologie)

t (s)	Session	Événement
0,003	A	BEGIN + SELECT : ACCESS SHARE détenu, transaction laissée ouverte
0,004	B	ALTER TABLE ... ADD COLUMN test_verrou : lancé (demande ACCESS EXCLUSIVE)
1,278	-	instantané pg_stat_activity / pg_locks pris pendant l'attente de B (B attend toujours)
2,009	B	retour de l'ALTER : 55P03
2,178	A	COMMIT : verrou ACCESS SHARE libéré
2,182	B	même ALTER relancé : réussi
2,400	B	DROP COLUMN test_verrou + RESET lock_timeout : nettoyage

Erreur de B : code 55P03, après 2,005 s mesurées côté script. Sortie brute :

```
Time: 2001.761 ms (00:02.002)
ERROR:  55P03: canceling statement due to lock timeout
LOCATION:  ProcessInterrupts, postgres.c:3437
```

Instantané pris pendant l'attente de B (pg_stat_activity puis pg_locks, requête sur clients_migration_tp) :

```
pg_stat_activity (session|état|type d'attente|attente|requête)
A|idle in transaction|Client|ClientRead|SELECT id FROM shopflow.clients_migration_tp LIMIT 1
B|active|Lock|relation|ALTER TABLE shopflow.clients_migration_tp ADD COLUMN

pg_locks (session|mode|accordé)
A|AccessShareLock|t
B|AccessExclusiveLock|f
```

Colonne test_verrou : absente après l'échec de B, présente après sa réussite, absente après le nettoyage (comptages 0 / 1 / 0).

Ajout : C derrière B (slide 31). B : lock_timeout 4 s ; C : SELECT count(*) lancé 1 s après B ; A garde sa transaction ouverte.

t (s)	Session	Événement
0,003	A	BEGIN + SELECT : ACCESS SHARE détenu
0,004	B	ALTER TABLE lancé (attend ACCESS EXCLUSIVE derrière A)
1,007	C	SELECT lancé (simple lecture, arrivée après B)
2,173	-	instantané pendant l'attente : B et C attendent, A non
4,007	B	ALTER abandonné : 55P03
4,012	C	SELECT terminé
4,017	A	COMMIT

```
session|type d'attente|attente|bloquée par un autre|requête
A|Client|ClientRead|f|SELECT id FROM shopflow.clients_migration_tp
B|Lock|relation|t|ALTER TABLE shopflow.clients_migration_tp AD
C|Lock|relation|t|SELECT count(*) FROM shopflow.clients_migrat
```

C a attendu **3,005 s** (durée mesurée côté script ; le chronomètre de psql donne 2999,9 ms) alors que A ne détient qu'un verrou de lecture compatible avec C : c'est la file d'attente de B qui la retient. Après l'abandon de B (55P03), C passe. Colonne test_verrou après ce scénario : 0 (aucune).

8.7 Trace 06 : retour arrière et contrôles

Contrôle	Attendu	Obtenu
Après le premier bloc (contrainte, NOT NULL, défaut retirés) : colonne encore là	1	1
… valeurs encore là (newsletter_ok = false)	1 000	1 000
… contrainte restante / is_nullable / défaut	0 / YES / aucun	0 / YES / (aucun)
Après DROP COLUMN : colonne restante	0	0
Écarts historiques (EXCEPT ALL, deux sens)	0	0
Rejeu de 02 sur la copie conservée / rejeu de 01	réussi / erreur « existe déjà »	réussi (1000 NULL) / erreur 42P07

DROP COLUMN détruit les valeurs de newsletter_ok : le retour de structure ne les reconstitue pas, seuls id, email et nom sont comparés.

8.8 Mesure à grande échelle : ancienne version → nouvelle version (3 000 000 lignes)

Tables de travail du schéma a4b_tmp (supprimé à la fin ; laboratoire vérifié : intact), PostgreSQL 18.6, mesurées le 2026-10-07 09:44:35. La « sonde » est une connexion séparée qui répète en boucle une lecture (SELECT … WHERE id = 1) ou un verrouillage de ligne (SELECT … FOR UPDATE sur l'id 1 500 000) et note la plus longue attente. Le WAL est lu sur tout le serveur (position d'écriture avant/après) : il inclut l'activité de fond (autovacuum, par exemple), d'où des valeurs élevées pour ADD COLUMN de la nouvelle variante.

Étape	Variante	Durée (ms)	WAL (octets)	Attente max de la sonde (ms)
ADD COLUMN	ancienne / nouvelle	4,8 / 2,5	18 016 / 1 669 864	—
SET DEFAULT false	ancienne / nouvelle	3,5 / 2,2	25 008 / 2 296 488	—
Remplissage	ancienne : un UPDATE	57 043	1 631 296 056	30 989 (verrou de ligne)
Remplissage	nouvelle : 60 lots de 50 000	156 265	1 985 809 168	416 (verrou de ligne)
CHECK … NOT VALID	nouvelle	7,9	48 304	—
VALIDATE CONSTRAINT	nouvelle	320,6	168	38,9 (lecture)
SET NOT NULL	ancienne (scan complet)	3 324	517 010 872	3 323 (lecture)
SET NOT NULL	nouvelle (sans scan)	38,4	7 048	—

Preuve de verrouillage (déterministe) : chaque ALTER est maintenu dans une transaction ouverte, une connexion concurrente (lock_timeout 500 ms) essaie ensuite de lire ou d'écrire.

Transaction maintenue ouverte	Verrou détenu	Lecture concurrente	UPDATE concurrent	INSERT concurrent
VALIDATE CONSTRAINT	ShareUpdateExclusiveLock	passe (1,4 ms)	passe (3,4 ms)	passe (2,0 ms)
SET NOT NULL	AccessExclusiveLock	**bloquée (55P03 après 500 ms)**	—	—

Résultat final identique : 3 000 000 lignes à false, 0 NULL, NOT NULL, défaut false, md5 b227f2377b62 = b227f2377b62. Lots de 50 000 lignes ici (200 dans l'atelier). Une seule exécution : les durées varient, pas l'ordre de grandeur.

Annexe 9 : preuves de l’atelier 7

Tout provient de atelier7/resultats/ : resultats_partie_a.json, mesures_partie_a.csv (CSV de l’aide du kit), api_journal.log (journal de l’API, une ligne JSON par appel) et api_extraits_par_traceid.txt pour la partie A ; offset_curseur_volume.json et les plans pour la mesure à 1 000 000 de commandes ; pgbouncer/ (synthese.csv, *_pgbench.txt, *_connexions.csv, *_observations.txt, configuration.txt de chaque campagne) et b_check.txt pour la partie B.

9.1 Environnement et état du laboratoire

Élément	Valeur
PostgreSQL ShopFlow	PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu
Node.js / API	v22.14.0 ; 01_server/api (server.mjs, commandes.mjs), POOL_MAX = 5, LAB_CLIENT_ID = 42 ; exécution du 2026-10-07 14:05:00
Laboratoire PgBouncer	PostgreSQL 17.11 (max_connections = 60), PgBouncer 1.18.0 (pool_mode = transaction, default_pool_size = 5, max_client_conn = 200, reserve_pool_size = 0), pgbench 17.11 ; base « pooling » séparée (1 000 produits), réseau Docker isolé
Avant / après	index : clients_email_key, clients_pkey, commandes_client_id_cle_idempotence_key, commandes_pkey, lignes_pkey, produits_pkey (identiques après) ; tables : clients, commandes, lignes, produits ; dates des commandes md5 8b3e4bf3d77c avant et après ; lignes md5 e989d5bec39b
Laboratoire restauré	**oui** (comparaison faite par le script ; la table de sauvegarde du kit est supprimée à la fin ; sauvegarde vide avant suppression : oui)

9.2 SQL émis par l’API (extraits de commandes.mjs)

```
-- OFFSET (page de 20 : LIMIT 21 pour hasNextPage)
SELECT <champs> FROM shopflow.commandes WHERE client_id=$1
ORDER BY commandes.created_at DESC, commandes.id DESC LIMIT $2 OFFSET $3
-- Curseur
SELECT <champs> FROM shopflow.commandes WHERE client_id=$1
AND (created_at,id)<($3::timestamptz,$4::bigint)
ORDER BY commandes.created_at DESC, commandes.id DESC LIMIT $2
-- N+1 : une fois par commande de la page
SELECT ... FROM shopflow.lignes WHERE commande_id=$1 ORDER BY lignes.produit_id
-- Groupé : une fois par page
SELECT ... FROM shopflow.lignes WHERE commande_id=ANY($1::bigint[])
ORDER BY lignes.commande_id,lignes.produit_id
```

Les ORDER BY qualifient les colonnes (commandes.created_at) : les alias de sortie id et created_at sont du texte, et un tri sur ces alias neutraliserait l’index (piège déjà rencontré à l’atelier 3). Les scripts SQL du kit (07_dates_identiques.sql, 07_restaurer_dates.sql) sont dans Kit_Jour4_Windows_Linux/02_Laboratoire/Jour4/ et rejoués sans modification.

9.3 N+1 contre groupé : appels uniques et traces de l’API (rattachées par TraceId)

Appel	Statut	SQL	HttpMs	sqlMs (API)	durationMs (API)	TraceId
N+1, 20 commandes	200	21	88,2	40,9	43,2	5aa4798c…
Groupé, 20 commandes	200	2	46,8	3,5	5,1	41f80089…
Curseur, page 2	200	2	42,6	2,1	2,7	bfa5e2ed…
OFFSET, page 2	200	2	42,4	2,0	2,5	c44b64ce…

Les lignes JSON complètes du journal de l’API (avec sqlCount, sqlMs, durationMs, poolWaiting) sont dans atelier7/resultats/api_extraits_par_traceid.txt. Ces appels uniques ne servent pas de mesure de gain : la première exécution d’un appel est plus lente (pool de connexions, caches) ; la comparaison repose sur les 30 tours ci-dessous.

9.4 Mesures répétées (30 tours alternés, 5 échauffements), durées en ms

Page	Mode	HttpMs médiane	p95	min	sqlMs médiane	durationMs médiane
20	N+1	55,3	64,2	51,1	14,2	14,6
20	groupé	43,9	51,1	40,0	2,2	2,5
50	N+1	68,5	80,0	65,3	29,0	29,5
50	groupé	44,2	52,8	40,2	2,6	3,0

p95 au rang le plus proche (29e valeur sur 30). Les 30 valeurs brutes de HttpMs de chaque série sont conservées dans resultats_partie_a.json (http_brut) ; les autres colonnes n'en gardent que la médiane.

9.5 Dates identiques (07_dates_identiques.sql) : sortie, pages et curseurs

```
2042|2026-09-13 23:54:02+00
1042|2026-09-13 23:54:02+00
42|2026-09-13 23:54:02+00
```

Page	Appel	Ids	hasNextPage	TraceId
1	/commandes?limit=2	2042, 1042	True	486259ea...
2	...&cursor;=	42, 86042	—	d6c69d85...

Page 2 par OFFSET (offset = 2) : réponse identique à celle du curseur. Page suivante après 1042 par created_at < date seule : 86042,83042 (42 absente) ; par (created_at, id) < (date, 1042) : 42,86042. nextCursor de la page 1 (signé, base64url) : eyJ2IjoxLCJjbGllbnRJZCI6IjQyIiwiZGF0ZSI6IjIwMjYtMDktMTNUMjM6…

9.6 Parcours, droits et restauration des dates

Contrôle	Attendu	Obtenu
Parcours : pages / total / uniques	5 / 100 / 100	5 / 100 / 100
Même ordre que PostgreSQL (ORDER BY created_at DESC, id DESC)	oui	oui
Commandes 2042, 1042, 42 : apparitions	1 / 1 / 1	1 / 1 / 1
Sans jeton	401	401
client_id=43 dans l'URL	400	400
Curseur falsifié (ajout)	400	400
Mode inconnu (ajout)	400	400
limit=101 / limit=0 (ajout)	400 / 400	400 / 400
dates_restaurees (07_restaurer_dates.sql)	3	3

```
3
42|2026-02-24 00:00:42+00
1042|2026-06-04 00:17:22+00
2042|2026-09-12 00:34:02+00
```

9.7 OFFSET contre curseur à 1 000 000 de commandes : mesures

Table de travail a7_tmp.commandes (supprimée ensuite) : 1 000 000 commandes d'un même client, 500 000 dates distinctes (deux commandes par date), table 66 672 kB, index (client_id, created_at DESC, id DESC) 39 656 kB. PostgreSQL PostgreSQL 18.6.

Décalage	Repère du curseur (date, id)	md5 de la page	OFFSET ms (7 mesures)	médiane	pages	Curseur médiane ms	pages
0	—	ef8a4de2ef83	0,185 0,161 0,155 …	0,171	24	0,154	24
20	(2026-12-14 05:10:00+00, 499990)	e5b3b25436df	0,165 0,174 0,193 …	0,180	44	0,177	24
1 000	(2026-12-13 21:00:00+00, 499500)	52fa2cb56a08	1,427 1,421 1,199 …	1,250	1 029	0,174	25

Décalage	Repère du curseur (date, id)	md5 de la page	OFFSET ms (7 mesures)	médiane	pages	Curseur médiane ms	pages
20 000	(2026-12-07 06:40:00+00, 490000)	cdddc0377d6a	19,8 22,4 21,1 …	21,8	20 122	0,167	24
100 000	(2026-11-09 12:00:00+00, 450000)	4e6aa812c3fd	99,7 103 109 …	99,7	100 516	0,168	24
500 000	(2026-06-23 14:40:00+00, 250000)	4cc8b3951a89	444 449 476 …	466	502 487	0,195	24
900 000	(2026-02-04 17:20:00+00, 50000)	d64595aa31d8	832 827 896 …	832	904 457	0,164	24
999 000	(2026-01-01 08:20:00+00, 500)	ff0a23a4a4f3	1003 944 934 …	944	1 003 945	0,177	24

Les 7 durées de chaque série sont dans offset_curseur_volume.json. Plans EXPLAIN (ANALYZE, BUFFERS) au décalage 100 000 (mesure n°7) :

Plan offset, décalage 100 000

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD HH24:MI:SS.US') AS created_at,
      statut, total::text AS total FROM a7_tmp.commandes WHERE client_id = 42 ORDER BY commandes.created_at
      DESC, commandes.id DESC LIMIT 21 OFFSET 100000
Limit  (cost=7160.63..7162.13 rows=21 width=118) (actual time=101.352..101.370 rows=21.00 loops=1)
  Buffers: shared hit=100516
  ->  Index Scan using idx_hist on commandes  (cost=0.42..71602.49 rows=1000000 width=118) (actual
        time=0.101..98.254 rows=100021.00 loops=1)
        Index Cond: (client_id = 42)
        Index Searches: 1
        Buffers: shared hit=100516
Planning:
  Buffers: shared hit=138
Planning Time: 0.455 ms
Execution Time: 101.396 ms
```

Plan curseur, décalage 100 000

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD HH24:MI:SS.US') AS created_at,
      statut, total::text AS total FROM a7_tmp.commandes WHERE client_id = 42 AND (created_at, id) <
      (TIMESTAMPTZ '2026-11-09 12:00:00+00', 450000::bigint) ORDER BY commandes.created_at DESC, commandes.id
      DESC LIMIT 21
Limit  (cost=0.42..2.02 rows=21 width=118) (actual time=0.095..0.125 rows=21.00 loops=1)
  Buffers: shared hit=24
  ->  Index Scan using idx_hist on commandes  (cost=0.42..68095.97 rows=899044 width=118) (actual
        time=0.093..0.122 rows=21.00 loops=1)
        Index Cond: ((client_id = 42) AND (ROW(created_at, id) < ROW('2026-11-09 12:00:00+00'::timestamp with
              time zone, '450000'::bigint)))
        Index Searches: 1
        Buffers: shared hit=24
Planning:
  Buffers: shared hit=137
Planning Time: 0.399 ms
Execution Time: 0.174 ms
```

9.8 PgBouncer : campagnes brutes (synthese.csv)

2026-10-07T12-06-44-389Z_persistant

Essai	Clients	Mode	Code	Transactions	TPS	Latence moy. (ms)	Pic connexions échantillonné	Éch.
1_direct	40	persistant	0	5200	261,8	152,5	40	17
1_pool	40	persistant	0	1884	93,8	422,2	5	15
2_pool	40	persistant	0	1900	94,7	418,2	5	15
2_direct	40	persistant	0	5160	260,6	153,0	40	17
3_direct	40	persistant	0	5187	260,7	152,9	40	17
3_pool	40	persistant	0	1882	93,7	422,6	5	15

2026-10-07T12-09-38-553Z_reconnexion

Essai	Clients	Mode	Code	Transactions	TPS	Latence moy. (ms)	Pic connexions échantillonné	Éch.
1_direct	10	reconnexion	0	802	39,8	189,2	10	17
1_pool	10	reconnexion	0	836	41,6	202,2	5	15
2_pool	10	reconnexion	0	601	29,7	257,5	5	12
2_direct	10	reconnexion	0	456	22,6	301,5	10	12
3_direct	10	reconnexion	0	420	20,8	320,4	10	11
3_pool	10	reconnexion	0	269	13,3	509,8	5	5

2026-10-07T12-13-34-674Z_reconnexion

Essai	Clients	Mode	Code	Transactions	TPS	Latence moy. (ms)	Pic connexions échantillonné	Éch.
1_direct	10	reconnexion	0	540	26,8	270,5	10	13
1_pool	10	reconnexion	0	639	31,8	253,7	5	12
2_pool	10	reconnexion	0	718	35,6	225,4	5	13
2_direct	10	reconnexion	0	782	38,8	195,4	10	17
3_direct	10	reconnexion	0	737	36,5	206,0	10	16
3_pool	10	reconnexion	0	670	33,1	246,2	5	12

2026-10-07T12-12-39-799Z_saturation

Essai	Clients	Mode	Code	Transactions	TPS	Latence moy. (ms)	Pic connexions échantillonné	Éch.
1_direct	80	persistant	1	non mesurable	non mesurable	non mesurable	0	2
1_pool	80	persistant	0	1609	90,7	860,0	5	10

Pic échantillonné : nombre de sessions du rôle atelier relevé environ une fois par seconde (pg_stat_activity) ; ce n'est pas un maximum exact. Le pic « 0 » de l'essai direct à 80 clients vient de ce que l'essai s'est arrêté en 2,9 s sur un refus de connexion : l'échantillonnage n'a rien vu, et ses TPS et sa latence sont « non mesurables », pas nuls.

Refus en direct à 80 clients (extrait de 1_direct_pgbench.txt) et SHOW POOLS pendant la charge PgBouncer (extrait de 1_pool_observations.txt)

```
pgbench (17.11 (Debian 17.11-1.pgdg12+2))

pgbench: error: connection to server at "db" (172.25.0.2), port 5432 failed: FATAL:  remaining connection slots are reserved for
      roles with the SUPERUSER attribute
pgbench: error: could not create connection for client 68

 database  |   user    | cl_active | cl_waiting | cl_active_cancel_req | cl_waiting_cancel_req | sv_active | sv_active_cancel |
       sv_being_canceled | sv_idle | sv_used | sv_tested | sv_login | maxwait | maxwait_us |  pool_mode
 pooling   | atelier   |        26 |          0 |                    0 |                     0 |         0 |                0 |
       0 |       4 |       0 |         0 |        0 |       0 |          0 | transaction
 database  |   user    | cl_active | cl_waiting | cl_active_cancel_req | cl_waiting_cancel_req | sv_active | sv_active_cancel |
       sv_being_canceled | sv_idle | sv_used | sv_tested | sv_login | maxwait | maxwait_us |  pool_mode
 pooling   | atelier   |        12 |         69 |                    0 |                     0 |         5 |                0 |
       0 |       0 |       0 |         0 |        0 |       0 |     592542 | transaction
 database  |   user    | cl_active | cl_waiting | cl_active_cancel_req | cl_waiting_cancel_req | sv_active | sv_active_cancel |
       sv_being_canceled | sv_idle | sv_used | sv_tested | sv_login | maxwait | maxwait_us |  pool_mode
 pooling   | atelier   |        14 |         66 |                    0 |                     0 |         5 |                0 |
       0 |       0 |       0 |         0 |        0 |       0 |     739627 | transaction
```

Lecture : **cl_waiting** = clients qui attendent une connexion serveur (66 à 69 avec 80 clients ; 25 à 29 avec 40 clients) ; **sv_active = 5** : jamais plus de 5 connexions serveur. La première ligne de chaque observation (cl_waiting = 0) est prise au tout début de la charge, quand les clients se connectent encore. Dans SHOW POOLS la ligne « pooling / atelier » est la bonne, pas la console « pgbouncer ».

Les clients du chemin direct gardent leur session ouverte (40 sessions pour 40 clients) ; le pool en garde 5. Les 3 valeurs de chaque campagne et leur médiane sont dans les tableaux ci-dessus ; le test avec reconnexion (option -C) a été exécuté **deux fois** parce que la première campagne était instable (débit décroissant d'un essai à l'autre) ; la deuxième l'est moins, et aucune des deux ne permet de conclure.

Annexe 10 : preuves de l’atelier 8

Tout provient de atelier8/resultats/ : resultats_atelier8.json, mesures_atelier8.csv (CSV de l’aide du kit), api_journal.log et api_extraits_par_traceid.txt (journal de l’API, une ligne JSON par appel) ; mesure_persistante.json et api_journal_persistante.log pour la mesure sur connexion persistante.

10.1 Environnement et état du laboratoire

Élément	Valeur
PostgreSQL / Redis / Node	PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu ; Redis 8.10.2 (maxmemory-policy allkeys-lru, save « 3600 1 300 100 60 10000 ») ; Node v22.14.0
Produit et clé	produit 42 (« Produit 42 », stock 100) ; clé shopflow:produit:v1:42 ; TTL 60 s (variable CACHE_TTL_SECONDS passée à l’API ; .env inchangé : oui)
Exécution	2026-10-08 09:30:28 ; python3 atelier8/run_atelier8.py (aide du kit jour4_http.mjs) ; mesure persistante : python3 atelier8/mesure_persistante.py
Avant / après	produit 42 : 57.50 / stock 100 avant, 57.50 / stock 100 après ; table produits md5 98aee3515f87 avant, 98aee3515f87 après ; dates des commandes md5 8b3e4bf3d77c (identique) ; index et tables identiques
Laboratoire restauré	**oui** ; clé Redis supprimée ; Redis démarré : oui ; API arrêtée

10.2 Code du cache (produits.mjs, extraits)

```
// getProduit : GET Redis ; hit -> retour sans SQL ; sinon SELECT, puis SET avec expiration (EX = TTL)
const value = await cache.get(key);                    // Redis indisponible -> cacheState = 'indisponible'
const r = await query('SELECT ... FROM shopflow.produits WHERE id=$1', [id]);
await cache.set(key, JSON.stringify(data), ttl);
// updatePrix : UPDATE (autocommit) puis DEL de la clé ; invalidation = 'ok' ou 'echouee'
const r = await query('UPDATE shopflow.produits SET prix=$2::numeric WHERE id=$1 RETURNING ...', [id, prix]);
try { await cache.del(key); } catch { invalidation = 'echouee'; }
```

L’UPDATE est en autocommit : sa réussite valide l’écriture avant l’invalidation. Avec une transaction explicite, il faudrait invalider après le COMMIT.

10.3 Les cinq paires miss / hit de la fiche (étape 2)

Paire	Miss : HttpMs	durationMs	SQL	Hit : HttpMs	durationMs	SQL
1	60,6	4,0	1	50,3	1,2	0
2	46,4	4,2	1	44,6	1,5	0
3	42,7	3,4	1	42,0	1,2	0
4	50,1	3,1	1	44,8	1,1	0
5	47,8	2,8	1	42,5	1,2	0

HttpMs vient de l’aide du kit : il inclut le traitement HTTP du client Node.js, qui lance un processus par appel. durationMs vient du journal de l’API (rattaché par TraceId). La première lecture d’une série est parfois plus lente.

10.4 Appels de l’atelier : valeur, cache, SQL, durées et TraceId

Moment	Prix	Cache	SQL	HttpMs	durationMs	TraceId
Miss initial	57.50	miss	1	53,6	4,7	989627b7…
Hit suivant	57.50	hit	0	50,4	2,3	775396e5…
UPDATE SQL sans DEL (copie ancienne)	57.50	hit	0	43,4	1,0	109f82c6…
Après DEL	19.90	miss	1	43,4	3,9	873ddadb…
PATCH 29.90	29.90	—	1	55,7	6,7	dc23a08a…
GET après PATCH	29.90	miss	1	41,0	2,3	6341aee1…
GET suivant	29.90	hit	0	40,0	0,887	4a02cd0b…
Après expiration (61 s)	29.90	miss	1	53,0	13,8	d4c5b091…
Redis arrêté	29.90	indisponible	1	41,2	1,5	efb2db3e…
PATCH, Redis arrêté (invalidation échouée)	34.90	—	1	—	—	db9e1c20…
Relecture après retour de Redis	29.90	hit	0	40,2	0,822	18cd840c…
Course : API après le SET de A	34.90	hit	0	42,8	0,938	ca7af0c7…

Les lignes JSON complètes du journal de l’API sont dans atelier8/resultats/api_extraits_par_traceid.txt. La durée du PATCH en panne n’est pas reportée (appel de contrôle, non chronométré).

10.5 TTL, expiration et sorties SQL

Contrôle	Attendu	Obtenu
TTL de la clé après chargement (TTL = 60)	entre 55 et 60 (pas 5)	60
TTL après deux lectures (étape 2)	entre 0 et 60	60
TTL après chargement, puis après 61 s (étape 5)	positif, puis −2	60, puis -2
UPDATE shopflow.produits SET prix=19.90 WHERE id=42 RETURNING id,prix	42 | 19.90	42|19.90
Prix PostgreSQL après l’UPDATE direct	19.90	19.90
PATCH 29.90 : corps de la réponse	invalidation = ok	{"data": {"id": "42", "prix": "29.90"}, "invalidation": "ok"}
PATCH 34.90 avec Redis arrêté : corps de la réponse	invalidation = echouee	{"data": {"id": "42", "prix": "34.90"}, "invalidation": "echouee"}
Prix PostgreSQL après ce PATCH / clé Redis après redémarrage	34.90 / ?	34.90 / présente (copie ancienne servie)
Restauration : PATCH du prix initial	invalidation = ok	{"data": {"id": "42", "prix": "57.50"}, "invalidation": "ok"}

Le serveur Redis sauvegarde son contenu à l’arrêt (réglage save ci-dessus) : la clé a donc survécu au redémarrage, ce qui rend l’échec d’invalidation observable. Sans cette persistance, la copie aurait disparu avec Redis.

10.6 Mesures répétées

Série	Tours	Durée requête (ms) médiane	p95	min	durationMs API médiane
Miss, connexion persistante	100	2,7	4,3	2,3	2,1
Hit, connexion persistante	100	0,988	1,9	0,801	0,671
Miss, aide du kit	30	43,0	49,5	40,1	2,4
Hit, aide du kit	30	42,0	50,3	39,7	0,979
Repli (Redis arrêté), aide du kit	30	42,2	49,4	38,2	1,4

5 paires d’échauffement avant chaque série. p95 au rang le plus proche (95e valeur sur 100 ; 29e sur 30). Les valeurs brutes de HttpMs (30 et 100 par série) sont dans les fichiers JSON ; les autres colonnes n’en gardent que la médiane. Le prix n’a pas été modifié pendant la mesure persistante (vérifié). Le rapport miss / hit est de ×2,74 sur la durée de la requête et de ×3,10 sur le temps dans l’API.

10.7 Rafale (stampede) : 20 lectures simultanées

État de la clé	Requêtes	Miss	Hit	SELECT au total	Durée médiane (ms)
Absente	20	6	14	6	42,5
Présente	20	0	20	0	33,4

20 requêtes lancées en même temps par un seul processus Node (atelier8/stampede.mjs). Toutes ont répondu 200. Le nombre de miss dépend du moment où la première copie est rangée dans Redis : ce n’est pas un chiffre reproductible à l’unité près, seul le fait que plusieurs requêtes rechargent la même clé l’est. Une mesure, non répétée.

Jour 5 : monitorer et prouver les gains

Observation, tuning, courbe de charge (Atelier 9), export analytique : chaque point par une mesure réelle

Protocole commun. Tests lourds sur une **copie jetable** du laboratoire (conteneur a9-pg, même image PostgreSQL 18.6, mêmes réglages, **mêmes données : empreintes md5 de commandes et de lignes identiques**) ; le laboratoire n'est modifié par aucun test (lectures et EXPLAIN annulés seulement). Machine : 8 cœurs, 7,6 Go, WSL2, PostgreSQL, API et générateur de charge sur la même machine. Courbe de charge : 1, 5, 10 et 20 clients, 10 s de mesure après 3 s d'échauffement, 3 répétitions en ordre alterné, **un seul changement par série**. Sorties brutes et scripts : dossier atelier9/ ; détail : annexe 11.

Slides	Point demandé	Preuve (section de ce chapitre)
4-5	métriques, logs, traces, tableau de bord	tableau de bord mesuré pendant 20 clients ; TraceId API → SQL
6-7	pg_stat_statements, temps cumulé	top des requêtes par temps cumulé, charge réelle de l'API
8-9	pg_stat_activity, requêtes lentes	instantané avec verrou ; journal lent : le N+1 n'y figure pas
10-11	API qui attend, tri sur disque	file du pool mesurée (pool 5 / 10 / 20) ; tri disque 3 040 kB → mémoire
12-15	méthode, work_mem, shared_buffers, connexions	work_mem alterné 9 tours ; effective_cache_size : aucun changement de plan
16-17	VACUUM, ANALYZE, transactions longues	VACUUM bloqué par une transaction longue (200 000 lignes mortes)
18-20	benchmark SQL / HTTP, pgbench, charge ouverte et fermée	séries A, B, D de l'Atelier 9
21	Atelier 9 : courbe de charge	courbes débit / p95, avant/après index et N+1, pool, saturation
22-23	lecture avant/après, conclusion limitée	tableau avant/après (%, facteur, erreurs, CPU) et conclusion bornée
24-29	OLTP / analytique, contrat, export, Lakehouse, Mesh	rapport concurrent, export CSV vérifié, contrat, implémenté ou conçu
30-31	pièces du dossier, grille	tableau pièces → preuves, auto-relecture

1. Observer : tableau de bord, traces, requêtes coûteuses

Tableau de bord réel : GET /commandes?limit=20 en N+1, 20 clients, médiane des 3 essais (une ligne par niveau de la slide 5).

Niveau	Observation	Valeur mesurée	Question de la slide
API	débit terminé, p50, p95, erreurs	246,9 req/s ; 79,0 / 98,0 ms ; 0 erreur	contrat respecté ? oui, 0 erreur
Pool (API, 5 connexions)	attente maximale (échantillon 1 s)	15 requête(s) en attente	le service attend-il une connexion ? oui
PostgreSQL	sessions actives (max)	4 actives sur 5 connexions	quel travail, quelle attente ?
Machine	CPU moyen / max, charge 1 min	50,7 % / 57,2 % ; 4,81	quelle ressource se sature ?

Niveau	Observation	Valeur mesurée	Question de la slide
Redis	hits / misses	non sollicité par /commandes ; hit 0 SQL, miss 1 SELECT (Atelier 8)	le cache réduit-il la charge ? ailleurs : oui

pg_stat_statements (copie avec l'extension ; charge : 1 000 pages N+1 + 200 pages groupées de l'API, 10 rapports mensuels, 10 exports ; requêtes normalisées, étiquetées d'après leur nombre d'appels). Requête de la slide 6, tri par temps cumulé :

Requête (normalisée, tronquée)	Appels	Total ms	Moyenne ms
Rapport mensuel (agrégation, atelier 1)	10	1116,8	111,676
Export journalier (slide 28)	10	650,7	65,066
N+1 : lignes d'UNE commande (WHERE commande_id = $1)	20 000	368,6	0,018
Page de commandes de l'API (une par appel HTTP)	1 200	301,4	0,251
Groupé : lignes de toute la page (= ANY($1))	200	37,3	0,186

Lecture. Par temps cumulé le rapport mensuel domine (10 appels × 111,7 ms = 1117 ms). La requête de lignes du N+1 fait **20 000 appels** de 0,018 ms : 369 ms cumulés, donc invisible dans le haut du classement. Pourtant l'API mesure 14,2 ms de sqlMs pour une page de 20 (21 requêtes, ≈ 0,68 ms chacune : **×38** le temps vu par PostgreSQL) : le coût du N+1 est dans les allers-retours, que pg_stat_statements ne compte pas. Le signal est le **nombre d'appels** (20 000 contre 200 pour la requête groupée). Surcoût de l'extension (agrégation mensuelle, 4 MB, 9 tours alternés) : 83,4 ms sans, 84,5 ms avec (×1,01).

Journal des requêtes lentes (slide 9)	Laboratoire	Copie
log_min_duration_statement	-1 (désactivé)	20ms
log_destination / logging_collector / pg_current_logfile()	stderr / off / NULL	journaux du conteneur (docker logs)
Lignes « duration: » pendant ce test (≥ 20 ms)	—	48 : rapports, exports, EXPLAIN ; requête de lignes du N+1 : absente
Journal complet de la copie, tous les tests	—	1 249 lignes ; requêtes de lignes (N+1 ou groupé) : 0

Extraits : 2026-10-08 10:05:53.493 [158] duration: 137.321 ms statement: SELECT date_trunc('month', created_at) AS mois, statut, co ‖ 2026-10-08 10:05:56.049 [229] duration: 52.304 ms statement: SELECT (created_at AT TIME ZONE 'UTC')::date AS jour, count

pg_stat_activity (requête de la slide 8, pendant qu'une transaction ouverte bloque un ALTER) :

```
 pid |        state        | wait_event_type | wait_event | bloqueurs | application_name
-----+---------------------+-----------------+------------+-----------+------------------
 353 | idle in transaction | Client          | ClientRead | {}        | A
 355 | active              | Lock            | relation   | {353}     | B
(2 rows)
```

2. Tuning : work_mem, cache, connexions, VACUUM

Requête (9 tours alternés)	work_mem	Médiane (ms)	p95	Tri	Pages temp. écrites
Slide 13 : SUM(total) GROUP BY client_id	4 MB	30,7	33,1	quicksort Memory 60 kB	0
	32 MB	31,2	41,2	quicksort Memory 60 kB	0
Agrégation mensuelle (Atelier 1)	4 MB	84,0	90,7	external merge Disk 3040 kB	381
	32 MB	53,5	79,3	quicksort Memory 26 kB	0

Lecture. La requête de la slide 13 ne déborde pas (60 kB, 1 000 groupes) : work_mem n'est pas sa cause, 32 MB ne change ni son plan ni son tri. L'agrégation mensuelle déborde (external merge Disk 3040 kB) ; à 32 MB le tri tient en mémoire (×1,6). **Coût mémoire :** work_mem est un budget par opération : le plan de l'agrégation compte 4 opérations de tri/hachage, et max_connections = 100 : pire cas théorique 32 MB × 4 × 100 = 12,5 Go (calcul, non mesuré) sur une machine de 7,6 Go : on règle donc localement (SET LOCAL), jamais globalement sans ce calcul.

shared_buffers / effective_cache_size : 128MB alloués, 4GB estimés (ce n'est pas une seconde allocation). Plan de SELECT id … WHERE client_id BETWEEN 1 AND k pour k = 20, 50, 100, 150, 200, 300, 500, 800 avec effective_cache_size = 1 MB, 128 MB, 4 GB, 16 GB : **aucun changement de plan** (Bitmap Heap Scan partout). Réglage non justifié ici : il n'est pas modifié.

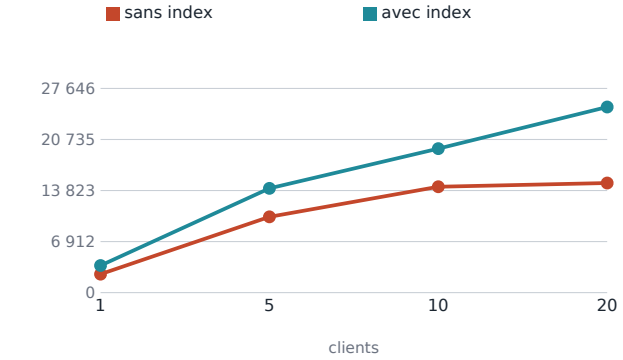
VACUUM et transaction longue (table de 200 000 lignes, tout mis à jour)	Lignes supprimées	Mortes non supprimables	Taille
Après l'UPDATE (200 000 lignes mortes, transaction ouverte depuis 3 s)	—	200 000 (statistiques)	14 160 kB
VACUUM pendant que la transaction longue est ouverte	0	200 000	14 160 kB
VACUUM après son COMMIT	200 000	0	14 160 kB (espace réutilisable, non rendu)

Table initiale : 7 080 kB ; l'UPDATE double la taille. **Une transaction oubliée empêche tout nettoyage** (slide 17). Laboratoire : autovacuum = on, track_counts = on ; clients : dernier autovacuum jamais, dernier analyze auto jamais, mortes 0. Connexions du laboratoire au moment du relevé : active 1, idle 1 sur 100 (dont 3 réservées).

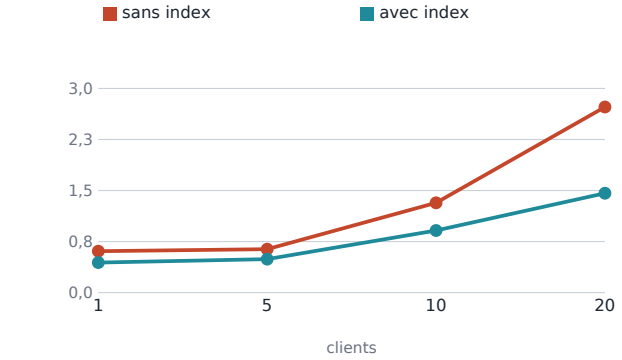
3. Atelier 9 : la courbe de charge

Même consultation, 1 / 5 / 10 / 20 clients, médiane de 3 essais, un seul changement par série. A : SQL (pgbench, script de la slide 19, client_id aléatoire de 1 à 1 000), sans index puis avec l'index (client_id, created_at DESC, id DESC). B : HTTP (charge fermée, GET /commandes?limit=20), N+1 puis chargement groupé.

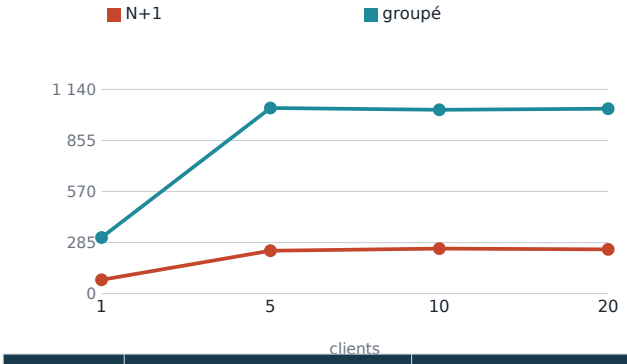
A. SQL : transactions par seconde



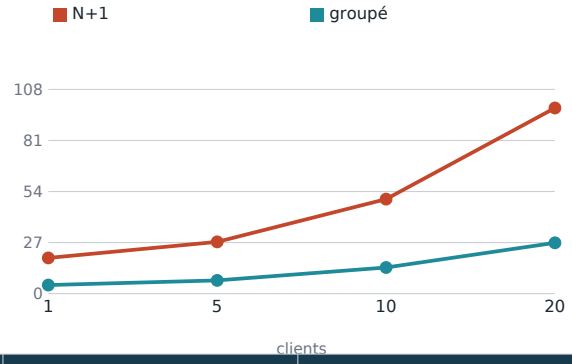
A. SQL : latence p95 (ms)



B. HTTP : requêtes terminées par seconde



B. HTTP : latence p95 (ms)



Cl.	A sans index : tps / p50 / p95 (ms)	A avec index : tps / p50 / p95 (ms)	B N+1 : req/s / p50 / p95 (ms)	B groupé : req/s / p50 / p95 (ms)
1	2 499 / 0,356 / 0,615	3 669 / 0,241 / 0,446	77,0 / 12,0 / 18,8	313,2 / 3,0 / 4,5
5	10 262 / 0,452 / 0,645	14 118 / 0,317 / 0,497	239,4 / 19,8 / 27,3	1036,1 / 4,5 / 7,0
10	14 336 / 0,512 / 1,3	19 496 / 0,401 / 0,922	251,3 / 38,1 / 49,9	1026,1 / 9,0 / 13,8
20	14 840 / 1,1 / 2,8	25 133 / 0,646 / 1,5	246,9 / 79,0 / 98,0	1032,1 / 18,2 / 26,8

Lecture. HTTP N+1 : le débit plafonne dès 5 clients (239 → 251 → 247 req/s) : les clients supplémentaires n'ajoutent que de l'attente (p95 27 → 98 ms, file du pool jusqu'à 15 requêtes). Le groupé plafonne plus haut (≈ 1032 req/s) avec un p95 de 26,8 ms à 20 clients. SQL : le débit croît jusqu'à 10 clients puis plafonne sans index (14 336 → 14 840 tps), l'index le relève (25 133 tps à 20 clients). Erreurs : 0 dans toutes les séries. Le SQL (pgbench) et le HTTP (loadgen) n'ont pas la même unité ni le même scénario : leurs résultats restent séparés (slide 18).

C. Le pool de l'API (N+1, 20 clients, 2 essais par taille). Un seul changement : POOL_MAX.

Pool	Essai	Req/s	p95 (ms)	Attente pool (max)	Sessions PG actives (max)	CPU moyen
5	1	216,0	125,0	15	2	50,1 %
	2	137,4	234,2	15	2	53,3 %
10	1	237,6	112,9	10	4	52,5 %
	2	254,9	105,1	10	2	54,3 %
20	1	275,7	97,2	0	2	55,0 %
	2	299,0	83,5	0	5	56,0 %

Lecture. Un pool de 20 supprime l'attente dans le pool (0) et relève le débit (216 et 137 → 276 et 299 req/s) : les sessions PostgreSQL actives montent jusqu'à 5 et la machine reste à ≈ 56 % de CPU : la base n'est pas saturée ici, l'attente n'a pas été déplacée vers elle. Les deux essais à pool 5 diffèrent (137 contre 216 req/s, alors que la série B donne 238 à 253) : série **indicative**, 2 essais. **Corriger le N+1 vaut mieux qu'agrandir le pool** : groupé avec 5 connexions = 1032 req/s, contre ≈ 287 req/s en N+1 avec 20.

D. Charge fermée contre charge ouverte (N+1, pool 5). Fermée : 20 clients, chacun attend sa réponse. Ouverte : arrivées à cadence fixe, latence comptée depuis l'instant prévu d'arrivée.

Charge	Cadence offerte (req/s)	Fin. / s	p95 (ms)	Erreurs 503	Requêtes en vol (max)	File du pool (max)
Fermée, 20 clients (série B)	—	246,9	98,0	0	20	15
Ouverte 50 % du débit fermé, essai 1	123	123,0	21,5	0	26	0
Ouverte 50 % du débit fermé, essai 2	123	123,0	18,8	0	8	0

Charge	Cadence offerte (req/s)	Fin. / s	p95 (ms)	Erreurs 503	Requêtes en vol (max)	File du pool (max)
Ouverte 90 % du débit fermé, essai 1	222	222,0	358,5	0	90	72
Ouverte 90 % du débit fermé, essai 2	222	7,0	9132,7	2 150	1 567	1 476
Ouverte 130 % du débit fermé, essai 1	321	16,9	5232,2	3 041	1 278	1 203
Ouverte 130 % du débit fermé, essai 2	321	27,8	6969,5	2 932	1 490	1 099

Lecture. La charge fermée donne 0 erreur et un p95 modéré : elle **masque** la saturation, parce que les clients ralentissent avec le service. En charge ouverte, à 50 % du débit fermé tout va bien ; à 90 % le comportement est instable (un essai tient avec un p95 élevé, l'autre s'effondre) ; à 130 % presque tout échoue en 503 (réponse « Service de données indisponible » : le pool de l'API abandonne après 1 s d'attente, connectionTimeoutMillis = 1000 dans server.mjs) et le débit terminé tombe **sous** le débit fermé. Les deux modèles ne se comparent pas sans le préciser (slide 20) ; chaque essai est présenté séparément, aucune médiane n'est calculée sur un comportement bimodal.

E. Rapports analytiques sur la même instance (slide 24). pgbench, requête d'historique, 10 clients, avec l'index ; « avec rapports » : 2 sessions exécutent en boucle l'agrégation par jour de l'export (≈ 65 ms chacune).

Série (3 essais alternés)	tps	p50 (ms)	p95 (ms)	p99 (ms)	Rapports terminés / essai
Historique seul	10 929	0,562	2,2	5,1	—
Historique + 2 rapports en boucle	10 005	0,382	3,6	11,2	156 / 171 / 236
Écart	−8 %	inchangé	+64 %	×2,2	

Le rapport ralentit la **queue** de latence des commandes (p95, p99) et le débit, pas la médiane : un rapport lourd concurrence l'OLTP sur la même instance. Séparer le calcul (export) déplace ce coût et ajoute un délai de fraîcheur.

4. Lire un résultat avant/après (20 clients, médiane de 3 essais)

Changement	Indicateur	Avant	Après	Réduction / gain	Facteur
Index composé (SQL)	p50 (ms)	1,1	0,646	−42 %	×1,7
	p95 (ms)	2,8	1,5	−46 %	×1,9
	débit (tps)	14 840	25 133	+69 %	×1,7
	erreurs / CPU moyen	0 / 85,9 %	0 / 84,9 %	—	—
Chargement groupé (HTTP)	p50 (ms)	79,0	18,2	−77 %	×4,3
	p95 (ms)	98,0	26,8	−73 %	×3,7
	débit (req/s)	246,9	1032,1	+318 %	×4,2
	erreurs / CPU moyen	0 / 50,2 %	0 / 50,1 %	—	—

Réduction = (avant − après) / avant ; facteur = avant / après (ou après / avant pour un débit) : deux façons d'écrire le même rapport (slide 22). Les erreurs sont constantes (0) ; le CPU moyen de la machine ne bouge pas pendant que le débit HTTP est multiplié : le gain vient du travail évité (21 → 2 requêtes par page), pas d'une ressource supplémentaire.

5. OLTP et analytique : export minimal et contrat de données

Requête de la slide 28 (lecture seule sur le laboratoire), CSV :
atelier9/export/commandes_journalieres.csv (240 lignes, md5 e17cb331709a).

Contrôle	Attendu	Obtenu
Granularité : une ligne par date UTC	240 jours distincts	240 lignes, 240 jours distincts (2026-01-02 → 2026-10-27)
Σ nb_commandes = commandes payées	80 000	80 000 ✔
Σ montant = Σ commandes.total des payées (exact)	23750126.25	23750126.25 ✔
Valeurs vides ou négatives	0	0
Reproductibilité : deux exécutions	même empreinte	e17cb331709a = e17cb331709a ✔
Piège : agréger après une jointure aux lignes	—	Σ = 71250378.75 (×3,0), 240 000 lignes au lieu de 80 000
Fuseau : commandes payées dont le jour UTC ≠ jour Paris	règle à écrire	4 896 sur 80 000 (6,1 %)
Annulation après publication (copie, annulée ensuite) : 2026-06-01	la journée change	333 / 137287.50 → 332 / 136950.00 (commande 223)

Contrat (slide 27)	Engagement	Preuve ou statut
Granularité / champs	une ligne par date UTC ; jour (date), nb_commandes (entier), montant (décimal exact)	CSV ci-dessus
Définition de « payée »	statut = 'payee' ; en_attente et annulee exclues ; remboursements non modélisés dans le schéma	statuts : annulee 10000, en_attente 10000, payee 80000
Qualité	montant exact (numeric), lignes uniques, aucune jointure aux lignes	invariants ✔ ; piège chiffré ci-dessus
Fraîcheur	publication quotidienne ; la journée en cours est provisoire	non testable : dates des données jusqu'au 2026-10-27 (synthétiques)
Correction d'une journée publiée	la journée est republiée en entier (remplacement), pas de cumul	démonstration d'une annulation tardive ci-dessus
Responsable / évolution	équipe Commandes (rôle du scénario) ; version v1 et période de compatibilité à fixer	conception, non implémentée

Concept (slides 24-29)	Dans ShopFlow	Statut
Export OLTP → analytique, contrat	requête d'export + CSV + contrat ci-dessus	**implémenté** (extraction ponctuelle)
Séparer le calcul des commandes	mesuré : un rapport concurrent dégrade p95 et p99 (série E)	**mesuré**
Lakehouse (Parquet, Delta Lake)	plateforme de stockage analytique	**conçu, non déployé** (non demandé)
Ingestion incrémentale fiable	—	**conçu, non implémenté** : l'extraction ponctuelle ne la prouve pas

Concept (slides 24-29)	Dans ShopFlow	Statut
Data Mesh	produit commandes_journalieres : responsable, champs, qualité, fraîcheur	**défini sur le papier** ; gouvernance partagée non mise en place

6. Conclusion limitée et pièces du dossier

Conclusion testable. Sur cette machine (8 cœurs, 7,6 Go, WSL2), ce jeu (100 000 commandes, 1 000 clients) et cette charge (1 à 20 clients, 10 s) : l'index composé fait passer le débit SQL de 14 840 à 25 133 tps (×1,7) et le p95 de 2,8 à 1,5 ms à 20 clients ; le chargement groupé fait passer le débit HTTP de 247 à 1032 req/s (×4,2) et le p95 de 98 à 26,8 ms, sans erreur. **Ce que ces mesures n'établissent pas :** « PostgreSQL devient ×1,7 plus rapide » ; un gain en production ; l'effet sur l'import (l'index coûte +35 % de WAL à l'insertion, Atelier 3) ; le comportement avec des clients très actifs (non testé ici) ou avec plusieurs instances d'API.

Résultat (slide 2)	Nature	Raison mesurée
Index composé (SQL, 20 clients)	**optimisation reproductible**	3 essais : 24 008 à 25 570 tps contre 14 570 à 15 555 sans index : intervalles disjoints
Chargement groupé (HTTP, 20 clients)	**optimisation reproductible**	3 essais : 978 à 1082 req/s contre 238 à 253
Taille du pool (série C)	**variation de mesure** possible	2 essais à pool 5 : 137 et 216 req/s ; la tendance (0 attente à 20) est nette, l'ampleur ne l'est pas
Charge ouverte à 90 % (série D)	**instable**	un essai tient (p95 359 ms), l'autre s'effondre (503) : pas de médiane
Index : coût d'écriture	**déplacement du coût**	WAL +35 % à l'insertion (Atelier 3) : le gain de lecture se paie en écriture
Rapport concurrent (série E)	**déplacement du coût**	p95 des commandes +64 % ; l'export déplace le calcul mais retarde la donnée
Cache Redis (Atelier 8)	**déplacement du coût**	gain de lecture, copie possiblement périmée (57,50 servi pour 19,90 en base)

Annexe 11 : preuves du Jour 5

Tout provient de atelier9/ : scripts p1_diagnostics.py, p1b_workmem.py, p2_charge.py, p3_export.py, loadgen.mjs, common.py ; sorties dans atelier9/resultats/ (p1_diagnostics.json, p1b_workmem.json, p2_charge.json, p3_export.json, journaux) et atelier9/export/commandes_journalieres.csv. Les journaux de latence par transaction de pgbench ne sont pas conservés (volume) : seules leurs statistiques le sont.

11.1 Environnement

Élément	Valeur
Machine	Intel(R) Core(TM) i5-10310U CPU @ 1.70GHz ; 8 cœurs ; 7827 Mo ; noyau 6.18.33.2-microsoft-standard-WSL2 (WSL2)
PostgreSQL	PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu (laboratoire api-postgres-1 et copie a9-pg, même image)
Copie jetable a9-pg	créée avec 01_schema.sql et 02_donnees.sql du projet ; pg_stat_statements chargé (shared_preload_libraries = pg_stat_statements) ; log_min_duration_statement = 20 ms ; supprimée à la fin
Identité des données	md5 de commandes 7505112bb794… et de lignes e989d5bec39b… identiques entre laboratoire et copie (vérifié avant les tests)
Réglages communs	work_mem 4 MB, shared_buffers 128 MB, effective_cache_size 4 GB, max_connections 100, autovacuum on
API et charge	01_server/api (server.mjs, commandes.mjs), POOL_MAX 5 sauf série C, port 3001 vers la copie ; loadgen.mjs (Node, fetch), pgbench 18.6 exécuté dans le conteneur (script de la slide 19, protocole simple)
Laboratoire	non modifié : 6 index, 0 statistique, tables inchangées (empreinte md5 des commandes vérifiée après les tests)

11.2 pg_stat_statements : requête de la slide 6 et résultat (tri par temps cumulé, 10 premières lignes de la charge de l’API)

```
SELECT query, calls, round(total_exec_time::numeric, 1) AS total_ms, round(mean_exec_time::numeric,
      1) AS moyenne_ms FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;
```

Requête	Appels	Total ms	Moyenne ms	Lignes
SELECT date_trunc($1, created_at) AS mois, statut, count(*)	10	1116,8	111,676	300
SELECT (created_at AT TIME ZONE $1)::date AS jour, count(*)	10	650,7	65,066	2 400
SELECT commande_id::text AS commande_id,produit_id::text AS	20 000	368,6	0,018	60 000
SELECT id::text AS id, to_char(created_at AT TIME ZONE $3,$4	1 200	301,4	0,251	25 200
SELECT commande_id::text AS commande_id,produit_id::text AS	200	37,3	0,186	12 000

Les rapports mensuels (10 appels) sont des premières exécutions non échauffées : leur moyenne est plus élevée que celle des mesures alternées de work_mem.

11.3 Journal lent (extraits) et pg_stat_activity

```
2026-10-08 10:05:53.493 [158] duration: 137.321 ms statement: SELECT date_trunc('month', created_at) AS mois, statut,
      count(*) AS nb_commandes, sum(total) AS chiffre_affa
2026-10-08 10:05:56.049 [229] duration: 52.304 ms statement: SELECT (created_at AT TIME ZONE 'UTC')::date AS jour,
      count(*) AS nb_commandes, sum(total) AS montant FROM sh


SELECT pid, state, wait_event_type, wait_event, pg_blocking_pids(pid) bloqueurs FROM
      pg_stat_activity WHERE datname = current_database() AND pid <> pg_backend_pid();
```

11.4 work_mem : tours alternés (ms) et plans de l’agrégation mensuelle

Requête	work_mem	9 tours (ms)	Médiane	Tri	Pages temp.
slide 13	4MB	30,3 29,6 30,1 29,8 30,7 32,9 32,8 30,9 33,1	30,7	quicksort Memory 60 kB	0
slide 13	32MB	31,9 33,4 31,2 30,5 29,2 30,6 41,2 38,3 29,6	31,2	quicksort Memory 60 kB	0
agrégation	4MB	81,2 83,5 88,3 90,0 81,3 79,6 84,0 86,8 90,7	84,0	external merge Disk 3040 kB	381
agrégation	32MB	53,6 52,2 50,5 53,5 50,9 54,3 64,5 50,1 79,3	53,5	quicksort Memory 26 kB	0

Plan de l’agrégation mensuelle, work_mem = 4MB (extrait)

```
GroupAggregate  (cost=13279.82..15742.50 rows=80845 width=54) (actual time=63.824..94.008 rows=30.00 loops=1)
  Group Key: (date_trunc('month'::text, created_at)), statut
  Buffers: shared hit=1680, temp read=380 written=381
  ->  Sort  (cost=13279.82..13529.82 rows=100000 width=20) (actual time=63.489..74.689 rows=100000.00 loops=1)
        Sort Key: (date_trunc('month'::text, created_at)), statut
        Sort Method: external merge  Disk: 3040kB
        Buffers: shared hit=1680, temp read=380 written=381
        ->  Seq Scan on commandes  (cost=0.00..2924.00 rows=100000 width=20) (actual time=0.600..28.527 rows=100000.00
              loops=1)
              Buffers: shared hit=1674
Planning:
  Buffers: shared hit=152
Planning Time: 0.372 ms
```

Plan de l’agrégation mensuelle, work_mem = 32MB (extrait)

```
Sort  (cost=11726.70..11928.82 rows=80845 width=54) (actual time=51.391..51.394 rows=30.00 loops=1)
  Sort Key: (date_trunc('month'::text, created_at)), statut
  Sort Method: quicksort  Memory: 26kB
  Buffers: shared hit=1680
  ->  HashAggregate  (cost=3924.00..5136.68 rows=80845 width=54) (actual time=50.925..51.333 rows=30.00 loops=1)
        Group Key: date_trunc('month'::text, created_at), statut
        Batches: 1  Memory Usage: 2089kB
        Buffers: shared hit=1674
        ->  Seq Scan on commandes  (cost=0.00..2924.00 rows=100000 width=20) (actual time=0.628..26.822 rows=100000.00
              loops=1)
              Buffers: shared hit=1674
Planning:
  Buffers: shared hit=152
```

11.5 effective_cache_size : plans par sélectivité

client_id ≤	1 MB	128 MB	4 GB	16 GB
20	Bitmap Heap Scan	Bitmap Heap Scan	Bitmap Heap Scan	Bitmap Heap Scan
50	Bitmap Heap Scan	Bitmap Heap Scan	Bitmap Heap Scan	Bitmap Heap Scan
100	Bitmap Heap Scan	Bitmap Heap Scan	Bitmap Heap Scan	Bitmap Heap Scan
150	Bitmap Heap Scan	Bitmap Heap Scan	Bitmap Heap Scan	Bitmap Heap Scan
200	Bitmap Heap Scan	Bitmap Heap Scan	Bitmap Heap Scan	Bitmap Heap Scan
300	Bitmap Heap Scan	Bitmap Heap Scan	Bitmap Heap Scan	Bitmap Heap Scan
500	Seq Scan	Seq Scan	Seq Scan	Seq Scan
800	Seq Scan	Seq Scan	Seq Scan	Seq Scan

11.6 VACUUM VERBOSE (extraits)

```
pendant la transaction longue :
INFO:  vacuuming "shopflow.shopflow.a9_dead"
INFO:  finished vacuuming "shopflow.shopflow.a9_dead": index scans: 0
tuples: 0 removed, 400000 remain, 200000 are dead but not yet removable
frozen: 0 pages from table (0.00% of total) had 0 tuples frozen

après son COMMIT :
INFO:  vacuuming "shopflow.shopflow.a9_dead"
INFO:  finished vacuuming "shopflow.shopflow.a9_dead": index scans: 1
tuples: 200000 removed, 200000 remain, 0 are dead but not yet removable
frozen: 0 pages from table (0.00% of total) had 0 tuples frozen
```

11.7 Mesures brutes de la courbe de charge (chaque essai)

Série	Clients	Variante	Essai	Débit	p50 ms	p95 ms	p99 ms	Erreurs	CPU %	Pool att.	PG act.
A SQL	1	sans index	1	2 499	0,356	0,615	0,759	0	15,3	—	1
A SQL	1	sans index	2	2 421	0,360	0,641	0,790	0	15,8	—	1
A SQL	1	sans index	3	2 540	0,348	0,612	0,747	0	15,0	—	1
A SQL	1	avec index	1	3 669	0,241	0,427	0,541	0	14,9	—	1
A SQL	1	avec index	2	3 695	0,234	0,446	0,561	0	17,5	—	1
A SQL	1	avec index	3	3 501	0,251	0,458	0,554	0	14,6	—	1
A SQL	5	sans index	1	13 385	0,351	0,504	0,763	0	60,9	—	5
A SQL	5	sans index	2	10 212	0,452	0,698	1,2	0	60,3	—	4
A SQL	5	sans index	3	10 262	0,458	0,645	1,0	0	58,1	—	5
A SQL	5	avec index	1	13 553	0,327	0,553	0,979	0	56,5	—	5
A SQL	5	avec index	2	15 338	0,302	0,451	0,737	0	57,2	—	5
A SQL	5	avec index	3	14 118	0,317	0,497	0,872	0	63,2	—	4
A SQL	10	sans index	1	14 951	0,496	1,3	2,3	0	90,6	—	9
A SQL	10	sans index	2	14 336	0,512	1,3	2,5	0	89,0	—	8
A SQL	10	sans index	3	10 766	0,552	2,2	6,2	0	88,6	—	6
A SQL	10	avec index	1	19 496	0,401	0,960	1,9	0	71,8	—	8
A SQL	10	avec index	2	19 404	0,424	0,922	1,6	0	71,1	—	7
A SQL	10	avec index	3	21 361	0,376	0,807	1,3	0	80,5	—	5
A SQL	20	sans index	1	14 570	1,1	2,8	5,9	0	82,7	—	9
A SQL	20	sans index	2	14 840	1,1	2,8	5,5	0	85,9	—	9
A SQL	20	sans index	3	15 555	1,1	2,5	4,6	0	86,2	—	8
A SQL	20	avec index	1	24 008	0,669	1,6	3,6	0	83,5	—	7
A SQL	20	avec index	2	25 570	0,633	1,4	2,8	0	87,8	—	8
A SQL	20	avec index	3	25 133	0,646	1,5	2,9	0	84,9	—	7
B HTTP	1	N+1	1	69,1	12,7	23,2	33,7	0	19,1	0	1
B HTTP	1	N+1	2	77,1	12,0	18,7	21,5	0	19,0	0	1
B HTTP	1	N+1	3	77,0	11,9	18,8	25,3	0	19,2	0	1
B HTTP	1	groupé	1	312,2	2,9	4,5	6,0	0	19,9	0	1
B HTTP	1	groupé	2	321,8	3,0	4,3	5,1	0	19,1	0	1
B HTTP	1	groupé	3	313,2	3,0	4,6	5,3	0	19,4	0	1
B HTTP	5	N+1	1	288,1	16,7	21,8	26,8	0	49,0	0	3
B HTTP	5	N+1	2	232,5	20,3	29,0	39,2	0	48,8	0	2
B HTTP	5	N+1	3	239,4	19,8	27,3	36,1	0	49,0	0	3
B HTTP	5	groupé	1	924,9	4,7	9,0	17,8	0	44,7	0	3
B HTTP	5	groupé	2	1036,1	4,5	7,0	11,6	0	46,3	0	3
B HTTP	5	groupé	3	1091,4	4,3	6,4	9,1	0	46,7	0	4
B HTTP	10	N+1	1	252,5	38,1	48,8	54,7	0	50,7	5	3
B HTTP	10	N+1	2	234,5	40,0	57,2	72,6	0	50,3	5	2
B HTTP	10	N+1	3	251,3	37,6	49,9	64,7	0	50,5	5	3
B HTTP	10	groupé	1	1067,6	8,9	13,5	17,2	0	50,4	3	2
B HTTP	10	groupé	2	1015,7	9,0	15,0	20,7	0	50,0	5	3
B HTTP	10	groupé	3	1026,1	9,2	13,8	17,6	0	49,6	4	3
B HTTP	20	N+1	1	253,2	75,8	98,0	111,8	0	50,2	15	2
B HTTP	20	N+1	2	246,9	79,1	96,3	102,3	0	50,7	15	4
B HTTP	20	N+1	3	238,5	79,0	109,3	140,0	0	50,1	15	2
B HTTP	20	groupé	1	1081,5	17,7	24,6	29,9	0	50,4	14	2
B HTTP	20	groupé	2	1032,1	18,2	26,8	34,2	0	50,1	14	1
B HTTP	20	groupé	3	977,8	18,6	29,1	45,1	0	48,9	14	4

Série	Réglage	Essai	Débit	p50 ms	p95 ms	p99 ms	Erreurs	Pool att.	PG act.
C pool	POOL_MAX 5	1	216,0	86,1	125,0	190,4	0	15	2
C pool	POOL_MAX 5	2	137,4	133,5	234,2	272,2	0	15	2
C pool	POOL_MAX 10	1	237,6	79,8	112,9	125,8	0	10	4
C pool	POOL_MAX 10	2	254,9	75,1	105,1	120,3	0	10	2
C pool	POOL_MAX 20	1	275,7	70,5	97,2	119,5	0	0	2
C pool	POOL_MAX 20	2	299,0	66,2	83,5	91,9	0	0	5
D ouverte	123 req/s	1	123,0	12,8	21,5	24,8	0	0	2
D ouverte	123 req/s	2	123,0	12,7	18,8	23,7	0	0	1
D ouverte	222 req/s	1	222,0	38,6	358,5	392,9	0	72	3
D ouverte	222 req/s	2	7,0	3894,2	9132,7	11193,7	2 150	1 476	2

Série	Réglage	Essai	Débit	p50 ms	p95 ms	p99 ms	Erreurs	Pool att.	PG act.
D ouverte	321 req/s	1	16,9	3807,6	5232,2	7072,6	3 041	1 203	3
D ouverte	321 req/s	2	27,8	5646,9	6969,5	8877,2	2 932	1 099	3
E OLTP	seul	1	8 487	0,382	5,0	14,2	0	—	6
E OLTP	seul	2	10 929	0,628	2,2	5,1	0	—	6
E OLTP	seul	3	13 222	0,562	1,7	3,6	0	—	6
E OLTP	avec rapports	1	10 005	0,365	3,6	12,0	0	—	8
E OLTP	avec rapports	2	9 912	0,382	3,8	11,2	0	—	7
E OLTP	avec rapports	3	18 643	0,415	1,0	3,2	0	—	8

Durée : 10 s par essai après 3 s d’échauffement. Séries A et E : pgbench -n -M simple, 4 threads au plus ; B, C, D : loadgen.mjs. Pool att. : plus grande file d’attente du pool de l’API vue par l’échantillonneur (1 s) ; PG act. : sessions PostgreSQL actives (max). Campagne complète : 16.1 min, du 2026-10-08 12:07:43.

11.8 Export et reproductibilité

```
SELECT (created_at AT TIME ZONE 'UTC')::date AS jour, count(*) AS nb_commandes, sum(total) AS montant FROM
      shopflow.commandes WHERE statut = 'payee' GROUP BY 1 ORDER BY 1


GroupAggregate  (cost=9868.78..11888.91 rows=69541 width=44) (actual time=51.238..69.979 rows=240.00 loops=1)
  Group Key: (((created_at AT TIME ZONE 'UTC'::text))::date)
  Buffers: shared hit=1677, temp read=200 written=201
  ->  Sort  (cost=9868.78..10069.57 rows=80317 width=10) (actual time=51.152..58.540 rows=80000.00 loops=1)
        Sort Key: (((created_at AT TIME ZONE 'UTC'::text))::date)
        Sort Method: external merge  Disk: 1600kB
        Buffers: shared hit=1677, temp read=200 written=201
        ->  Seq Scan on commandes  (cost=0.00..3325.59 rows=80317 width=10) (actual time=0.020..25.089 rows=80000.00
              loops=1)
              Filter: (statut = 'payee'::text)
              Rows Removed by Filter: 20000
              Buffers: shared hit=1674
Planning:
  Buffers: shared hit=145
Planning Time: 0.433 ms
Execution Time: 70.465 ms
```

Commandes pour tout rejouer : docker run (copie a9-pg, voir en-tête de p1_diagnostics.py) ; python3 atelier9/p1_diagnostics.py ; python3 atelier9/p1b_workmem.py ; python3 atelier9/p2_charge.py (≈ 16 min) ; python3 atelier9/p3_export.py. La copie a9-pg est jetable : elle ne contient aucune donnée du laboratoire modifiée et n’est pas nécessaire pour lire ce document.